```
import threading
from flask import Flask, render_template, request, jsonify, send_file, redirect,
session
from flask_cors import CORS
import os
import time
import io
import base64
import hmac
import hashlib
import hmac as hmac_lib
import hashlib as hashlib_lib
import json as json_lib

app = Flask(__name__)


# ── PWA: install files must be served from the site root ──
_PWA_BASE = os.path.dirname(os.path.abspath(__file__))

# Manifest + transparent splash/app icons are built into app.py itself (no separate
files to upload).
import base64 as _pwa_b64
from flask import Response as _PwaResp

_PWA_MANIFEST_JSON = json_lib.dumps({
    "name": "Whole AI",
    "short_name": "Whole AI",
    "description": "AI coding and building assistant. Review code, build websites and
apps from your phone.",
    "start_url": "/",
    "scope": "/",
    "display": "standalone",
    "orientation": "portrait",
    "background_color": "#F0E8FF",
    "theme_color": "#F0E8FF",
    "icons": [
        {"src": "https://www.wholeai.space/icon-transparent-192.png", "sizes":
"192x192", "type": "image/png", "purpose": "any"},
        {"src": "https://www.wholeai.space/icon-transparent-512.png", "sizes":
"512x512", "type": "image/png", "purpose": "any"},
        {"src": "https://www.wholeai.space/icon-maskable-512.png", "sizes":
"512x512", "type": "image/png", "purpose": "maskable"}
    ]
}, indent=2)
```

```
_PWA_ICON_192 = _pwa_b64.b64decode(

"iVBORw0KGgoAAAANSUhEUgAAAMAAAADACAYAAABS3GwHAAAWfmNhQlgAABZ+anVtYgAAAB5qdW1kYzJwYQAR
ABCAAACqADibcQNj"

"MnBhAAAAFlhqdW1iAAAAR2p1bWRjMm1hABEAEIAAAKoAOJtxA3VybjpjMnBhOjBjYjJlY2JiLTUxMjgtNDYz
MyO4Mjg1LTkzMGJm"

"YmZhMmIwYwAAAAOTanVtYgAAAClqdW1kYzJhcwARABCAAACqADibcQNjMnBhLmFzc2VydGlvbnMAAAAAuGp1
bWIAAABEanVtZGNi"

"b3IAEQAQgAAAqgA4m3ETYzJwYS5pbmdyZWRpZW50LnYzAAAAABhjMnNonsjRmpciIfnA77zCntKQWQAAAGxj
Ym9yo2lkYzpmb3Jt"

"YXRpaW1hZ2UvcG5namluc3RhbmNlSUR4LHhtcDppaWQ6M2ZkZjNjYzEtOWM2MiO0NTMzLWEwMWItMmFhMzdl
MzBlZjdhbHJlbGF0"

"aW9uc2hpcGhwYXJlbnRPZgAAAeJqdW1iAAAAQWp1bWRjYm9yABEAEIAAAKoAOJtxE2MycGEuYWN0aW9ucy52
MgAAAAAYYzJzaGC/"

"9/OwjEBFqyuOo/wn2eEAAAGZY2JvcqJnYWN0aW9uc4KiZmFjdGlvbmtjMnBhLm9wZW5lZGpwYXJhbWV0ZXJz
oWtpbmdyZWRpZW50"

"c4GiY3VybHgtc2VsZiNqdW1iZj1jMnBhLmFzc2VydGlvbnMvYzJwYS5pbmdyZWRpZW50LnYzZGhhc2hYIHrd
PAl9jmqxRWSJbQU2"

"1fWJeza9KacF8ncPnwJ2NcH6pGZhY3Rpb254HWNvbS5hbnRocm9waWMuY2xhdWRlLnByb3ZpZGVkanBhcmFt
ZXRlcnOheB9jb20u"

"YW50aHJvcGljLm9yaWdpbi1jb25maWRlbmNlZ3Vua25vd25rZGVzY3JpcHRpb254ZkNsYXVkZSBwcm92aWRl
ZCB0aGlzIGZpbGUg"

"YXQgdGhlIHJlcXVlc3Qgb2YgYSB1c2VyIGFuZCBtYXkgaGF2ZSBjcmVhdGVkIG9yIG1vZGlmaWVkIHRoZSBm
aWxlIGNvbnRlbnRz"

"Lm1zb2ZOd2FyZUFnZW50oWRuYW1lZkNsYXVkZXJhbGxBY3Rpb25zSW5jbHVkZWT1AAAAyGp1bWIAAABAanVt
ZGNib3IAEQAQgAAA"

"qgA4m3ETYzJwYS5oYXNoLmRhdGEAAAAAGGMyc2gRyCC8c76joppqr+UcbWNoAAAAgGNib3KlY2FsZ2ZzaGEy
NTZjcGFkTQAAAAAA"

"AAAAAAAAAABkaGFzaFggxQS8khsxs2No6BpHbMDvgreu/t5eh7vdUi1QmvVza6VkbmFtZW5qdW1iZiBtYW5p
ZmVzdGpleGNsdXNp"

"b25zgaJlc3RhcnQYIWZsZW5ndGgZFooAAAI+anVtYgAAACdqdW1kYzJjbAARABCAAACqADibcQNjMnBhLmNs
YWltLnYyAAAAAg9j"

"Ym9ypWNhbGdmc2hhMjU2aXNpZ25hdHVyZXhNc2VsZiNqdW1iZj0vYzJwYS91cm46YzJwYTowY2IyZWNiYi01
MTI4LTQ2MzMtODI4"
```

```
"NSO5MzBiZmJmYTJiMGMvYzJwYS5zaWduYXR1cmVqaW5zdGFuY2VJRHgseG1wOmlpZDoxNWEwNTFjNi1hMzA2
LTQxZWMtYTcwNC1l"

"OWE4NDJhM2FjZmRyY3JlYXRlZF9hc3NlcnRpb25zg6JjdXJseC1zZWxmI2p1bWJmPWMycGEuYXNzZXJ0aW9u
cy9jMnBhLmluZ3Jl"

"ZGllbnQudjNkaGFzaFggget08CX2OarFFZIltBTbV9Yl7Nr0ppwXydw+fAnY1wfqiY3VybHgqc2VsZiNqdW1i
Zj1jMnBhLmFzc2Vy"

"dGlvbnMvYzJwYS5hY3Rpb25zLnYyZGhhc2hYIIo0EYi7Rxk4RY7ZotENe2Z/8dVJHe7oqQql86g9z0bXomN1
cmx4KXNlbGYjanVt"

"YmY9YzJwYS5hc3NlcnRpb25zL2MycGEuaGFzaC5kYXRhZGhhc2hYIG54lQt1/2OxTA3P2qoaok4axF0Ti0rV
mTOE/1OKjWaEdGNs"

"YWltX2dlbmVyYXRvcl9pbmZvo2RuYW1lb0FudGhyb3BpYyBGaWxlc2d2ZXJzaW9uZTEuMC4wa3NwZWNWZXJz
aW9uZTIuNC4wAAAQ"

"OGp1bWIAAAAoanVtZGMyY3MAEQAQgAAAqgA4m3EDYzJwYS5zaWduYXR1cmUAAAAQCGNib3LShFkCEqIBJhgh
WQIKMIICBjCCAY2g"

"AwIBAgIUQOWgCu7COdC+uIP6BkIFPWdVEwAwCgYIKoZIzj0EAwMwSTEXMBUGA1UEChMOQW50aHJvcGljLCBQ
QkMxLjAsBgNVBAMT"

"JUFudGhyb3BpYyBDb250ZW50IENyZWRlbnRpYWxzIFJvb3QgQ0EwHhcNMjYwODA3MTg0MzU2WhcNMjgwODA2
MTk0MzU2WjBEMRcw"

"FQYDVQQKEw5BbnRocm9waWMsIFBCQzEpMCcGA1UEAxMgQW50aHJvcGljIENsYXVkZSBDb250ZW50IFNpZ25p
bmcwWTATBgcqhkjO"

"PQIBBggqhkjOPQMBBwNCAASYegpry1AYBRTVNL1CpTlbROnY3dey+UrsF9C3phYrATN3ZHf93Mo8RQN0KOUu
On19P4oWNFWe5n2/"

"She9N7eTo1gwVjAOBgNVHQ8BAf8EBAMCB4AwFQYDVR0lBA4wDAYKKwYBBAGD6F4CATAMBgNVHRMBAf8EAjAA
MB8GA1UdIwQYMBaA"

"FM5R4gSBTmRbI/jjxM+aPpzB11zCMAoGCCqGSM49BAMDA2cAMGQCMDFzHRSeAXrSy1WOzkbhPZ6Km2wGTmZ/
2gK18k8BQGXyqz88"

"Rdrz6CTX9flAnYNVxgIwcF9c3fVhqmJKpi+UhasNUMko69cyX6STPfta3Q8EjyzDjzoyrol46FP6VFHhvUcJ
oWNwYWRZDZ4AAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAA"
```

"AAAA

```
AAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAA
```

```
"AAAA
```

"Oz8ldFSliONS+n7GoP8PRZk6yWrMUKiTC3Au8Ajtil+naTPqQ8Os/f9IlJEos/sGZijS5sRPgGNMhuL3MoRjUdZOeTCmMlKc9cA9"

"FMq2wOooeZUjle2eKHuaF2MqoSbPVuAxRuf1O9GddDhU/9cdLepXGzwW85DmyZieSFG2Ac8RFOk4gyujFL1OJ1md3F7zAvOOyfxc"

"zEuaNxNxJ6mdNQSF2wbsIczgLlB/VEVKXE73PvAa8B/Csodj8fuNhOURpwCnAhtK6RYYrFMr2V8AdhJml5VHY9rQGP8m4ABFs2cQ"

"j5968r2EsfrdwBmOz+KWWRvP2R3T7KW95hikRlAeDsS8ea7ALENDnbMEzz+I8qeKfwj4NXB+hfvp6Obdz4/XSucd6nbElZc9Fe5n"

"MkRNldup3+ZXx7YFvA78HNiSXFtrhmZpnOnuhLzzLMvX+GyJ136donapUxsoT7eX8mwyRx3DK6nv+VNPfDeh/S7mGE1TYw3tynpq"

"vNcgtYHydmW8ljvFmaMNJ58mrMdfpPpojRTvDQqFgvGtziyvPr0y3ruOESh/Rwh5Vv5NpsgD3kc976/zngK2x2us1LLk1BC2RxkG"

"kf2+eA3XApmiB7+Lel4O7VBuitdYjfaO7rmJ+h135XVXvIaNIDPU2VxLGB5MO7JVFOfvFGP1q6k8uvcGgkxVDVkd6AOEMujXOTdT"

"hrznNdRTmhbwInBiTD8JnlMynEiQLZW1ijFfE9N7VCgjNA7/DMsnrzodOucY7cuNJ4V0ufYxqk2Y6Zxn8LxAVkhZdlPd++uc62La"

"SfSWkuk66udrdOw7SUZtxoQe8vOs31DSSOkeiukmUfmFZHuIakagFa73x3Q2gClHY95nEBamKWhVr6bPIiFMyTkx7SQriWQ7hyBz"

"v9li5f99QpmA5wWmGnnI66k2bKjf74rpJln5hWS8i3p5vD6mm+QazgyJvNuD9O/8yjvOA2fTnFWUGtI8m6KJO6sWUGf4wSS9mUIO"

"yrEZeItCyfu1/R+I6ZqkGJL1Afr3BVQGbxHKBjIaEWrSQx0W5fUCwsrKJao96HtpjvcXqgXurXCuAvhuIZSN0mdBNhmlUPaL42ev"

"XVEtQlv6KPAwxeRSU9BE2MOEPMzG/7uhslDZuAaYYrptUkmRQvyTsHJSIcqbQosg8xFCHqDaNsgqZTNV5GQA8uBnxc9eXk7K/kT8"

"bGI5SWbloZcBqyxUNk2q7YaiiQ92EGYICrCBYtNKv51ZEJYaN52n4meV/GpDfqvP+VNDLgYgNlNtpEPl8nL8bGIUBcmsPPR61ukI"

"2eYxyTOR5GIAesCfoP9LKuT9PiSOoZvOEUJeVAv2Yh2hjMA1wFShh/lJihGRfg/4PeDd+HeTOsBCMr9L+5tlOiHjmCWUkb6benIx"

"gEFQqMKmo1CMpgO5GUAdrzZNHjDXfPclNwN4n+rNmY3xgGYqhWRO89GPFqGMsiEXAOjbw/PO7hBKcTbQHuCqqWyh2L/czZDVJ5qn"

"2f2e2uRmAO8Qlgb0Y4lQNqfH/5tcA5xO9YC4RwllBDaAqeQoIZgUVFsboO0wTTYA5aHf2icIZVPFQUwNuRiAhviWKCaGqng4LQ5r"

"ojeUzBf3PKv93JcJZdRv8dzUkIsBQOERn4mfvR6wyuVSwuTQIs2qBWYIMq8j5AF6P2uVhcqmSXkdipwMQOyL

```
n/2WQiwBZwI7aO5+"

"gB2EPKhP0w2Vxb4e5OwlTXqow6I28F5CqPCqa+SvpnmLwzTKdXX8v8reh+OEsul3vmkwM/HYT7HJpd9WwcMU
b1ZpghFIzk0Ub7Hv"

"tfVTZbCf5uRxZORUA0Dh9RXjp5enUzv6ZOBaCk856SiP1xJk79d/URkollAT8mgGRA/3IvrXAPp9CXiFMJs6
6UFk1VfZSJC5StBf"

"/X5RvIYNYMpRXNCn6R8xIf39lph+kuPmSLZbqJe3p3F8OGyQktxAe2CoXn2BBeAD4LyYdhK9pGQ6jyBrlQC5
yvsNMe0kG7cZEfJy"

"W4G36R84KvWU+wjx9CfNW6pWW0uQsYr3V77fpnib/CTlyYwRebrbqFYLpAp1Z0y7lslQmBmKdw/fSTXlT/N8
W0xr758R6syeRlj9"

"WPWVo1KaG+N1VtsIUuW/kerGrM7xu4QyaNpEnxkBajPfRD3FKbebV6s5pGYPtPdn6hjyTTH9JPZpzJiR1/s4
8BLV3xOWBtW9Nbne"

"SipReq9bKZo9VZRftd1LhLxP+tCuGSNSpMuoXguU+wR/oQglMqqXY3cjfWn25njvqm3+sve/LF7H3j9zpFB3
UN8IdO6LwOWla47S"

"EMpvi7+c4oV4g8h7RyKnyRy1pddTrBGq41HTc++heHG2ri1jqLsxXUqfptse79Hp3lXl3B/zOmlDuWYVkbc+
i2JuoEp/oNyubhF2"

"VP2OsBS5zCxBqXXMdvmuzI54zaO0j+LUle9tivifHvUxbUjxdlLEBKqjZGWPvAQ8QhihOZdiuLIKa2OaG+I1
0s5tHa8v5VdsoJ2l"

"vGaPq8B25giK8gPgTxQrJet4SyndXOm7l4BnCW33VwghCxWx7QTCbOyZwBcIyv852p/PAvWbLan81wB/psij
MR0pv0W+bnMjrQEW"

"qNdJ7dRprTq82atZ5rfBm1pIUa4iLCyTMg6qyDIiGYQUOz3S76vOTPcynFaU/apSnoyphBTmG8CrFIo1jGKO
+0hnql+Nsqd5MaYW"

"UpzPAHsYvCO6Ekcq054oc5oHYwZCIyYzwM2EWPtSuEH6BqM+1LRqRdlupugoe7THjIR0IutC4FHaPe9qGEKq
+KOoO4VRRq/uNGMh"

"bU5cCzzPcoUcZx9BI0upwT0fZekkozEjJ60N1gM/Bp5keXt8nuFHczR6pNGh9Lcn473XR1ns9c2KkravZwiz
rH+kiMXTq7lSpUPb"

"qVl1ON5jJ+0TYm7rD4hngodDXjd9ldJJwDeBXcAlhJndjw14/eOEmeMnCHF7HgXeTH5XwN/WgNc3ZmRoYVuZ
7cB3gN9SNGt6dWxb"

"wF0xzfYO1+t2H2MmAi2vLndGv0dQ7l4zyvrtZ6WOWiXqGnvEeMRg9MiTQ7FpfRH4bI1rnEnh6dUBNmPABjBe
yiM5VdEIUpWXW5sh"

"8LCZyRobgMkaG4DJGhuAyRobgMkaG4DJGhuAyRobgMkaG4DJGhuAyRobgMkarwXqjtb6D7sCU2Vcx9koMO4o
nk+6BNuUsAF0ZpYi"

"puawaCXnsRpp/kcRMGtUePNMB2wAy9Eb4rcCXwK2DHk9GdOlyfV73RvgAuD7SdpBaREiSu8HXkvuYSMwy0g3
lf8KeIOVD3UyruO/"
```

```
"wG8IWzNn8Maaj3BBFKwhNBF+D/wwfpd630G9pjzuGqr3A/QO32G8dadN838Dvkv7hntjPmoK/oigFB8y2TFA
6x5LFBHt9IpX7ys2"

"H6FmwX7qhzBpyqFO/UFCLCE3hfA8AIQyaAGfAj5PvaZKk1C+thH2J7ewAUzlgx6Ucb/idFLQi/sMeTzwfsgT
vg4cYnrHytUXeBP4"

"V/Jd1tgAghLMEtr9fyCUyTSGIZkn5O2vhLmBWWwAbgNG1CFcR3i5xKUUQ4WjYpBhOFGhYF3PA18jzAt4GNSO
IWewGbiP1R+1GfXx"

"IHBazKNr/ohrgHbSiaeLgK8Tgt0OgybYvgx8O/7dTQH1215CIFylHYZ3gMeBx0ryGNORccXZv45gXFVig/5y
DPf3uH8HPBy2HCnj"

"qILR6sXUJ9RIs5FiOfQoOuSaCDMlbADdGaXC6NVJVdGrkGA6R6QmBneGTNbYAEzW2ABM1tgATNbYAEzW2ABM
1tgATNbYAEzW2ABM"

"1tgATNbYAEzW2ABM1tgATNbYAFYOr8WfQGwAK4c2u/TDa/dXEBvA+JHSP0X/XVn6fd+YZTJmxdAWyzlC6MUW
cJzu2yEPEnaEeQuj"

"mRpUO15IeFFGi7DTaz4eikW6COyK5zp4rZkqZASXUNQE6fECsLt0rhkzrmJXFoUkmQO+BeyI3x0gBOT6gOHf
CmPMRNPLu7vZs8K4"

"Blgdym+gVOBahyo0xhhjjDHGGGOMMcYYY4wxxhhjjDHGGGOMMcYYY4wxxhhjjDHGGGOMMcYYY4wxxhhjjDHG
GGOMMcYYY8zU8X9R"
    "hGEr6x/yUAAAAABJRU5ErkJggg=="
)
_PWA_ICON_512 = _pwa_b64.b64decode(

"iVBORw0KGgoAAAANSUhEUgAAAgAAAAIACAYAAADOeNT6AAAWfmNhQlgAABZ+anVtYgAAAB5qdW1kYzJwYQAR
ABCAAACqADibcQNj"

"MnBhAAAAFlhqdW1iAAAAR2p1bWRjMm1hABEAEIAAAKoAOJtxA3VybjpjMnBhOmEzZGYwMmNiLTM1NzEtNGZj
OS1iODI1LTBjNzQ2"

"ZWZjNzdhZgAAAAOTanVtYgAAAClqdW1kYzJhcwARABCAAACqADibcQNjMnBhLmFzc2VydGlvbnMAAAAAuGp1
bWIAAABEanVtZGNi"

"b3IAEQAQgAAAqgA4m3ETYzJwYS5pbmdyZWRpZW50LnYzAAAAABhjMnNog73h8XvWc1N9ZaoMnEgQngAAAGxj
Ym9yo2lkYzpmb3Jt"

"YXRpaW1hZ2UvcG5namluc3RhbmNlSUR4LHhtcDppaWQ6MGZmMmJhZWItNzBhZi00YTIyLTk5NDgtNzUxMThm
MTRlYmI1bHJlbGF0"

"aW9uc2hpcGhwYXJlbnRPZgAAAeJqdW1iAAAAQWp1bWRjYm9yABEAEIAAAKoAOJtxE2MycGEuYWN0aW9ucy52
MgAAAAAYYzJzaKm1"

"6tn3LQ2GemPO3GZOjHIAAAGZY2JvcqJnYWN0aW9uc4KiZmFjdGlvbmtjMnBhLm9wZW5lZGpwYXJhbWV0ZXJz
oWtpbmdyZWRpZW50"

"c4GiY3VybHgtc2VsZiNqdW1iZj1jMnBhLmFzc2VydGlvbnMvYzJwYS5pbmdyZWRpZW50LnYzZGhhc2hYIMpB
DVzMuVog6qdCCHak"
```

"8uMvDCKyVsilVw/QKMaYtO6WpGZhY3Rpb254HWNvbS5hbnRocm9waWMuY2xhdWRlLnByb3ZpZGVkanBhcmFtZXRlcnOheB9jb2Ou"

"YW50aHJvcGljLm9yaWdpbi1jb25maWRlbmNlZ3Vua25vd25rZGVzY3JpcHRpb254ZkNsYXVkZSBwcm92aWRlZCBOaGlzIGZpbGUg"

"YXQgdGhlIHJlcXVlc3Qgb2YgYSB1c2VyIGFuZCBtYXkgaGF2ZSBjcmVhdGVkIG9yIG1vZGlmaWVkIHRoZSBmaWxlIGNvbnRlbnRz"

"Lm1zb2ZOd2FyZUFnZW5oWRuYW1lZkNsYXVkZXJhbGxBY3Rpb25zSW5jbHVkZWT1AAAAyGp1bWIAAABAanVtZGNib3IAEQAQgAAA"

"qgA4m3ETYzJwYS5oYXNoLmRhdGEAAAAAGGMyc2jM6zp6g4mrVDX6L2uMLEuDAAAAgGNib3KlY2FsZ2ZzaGEyNTZjcGFkTQAAAAAA"

"AAAAAAAAAABkaGFzaFgg4+UplNmPt/5DxVywRX4q3YaXj3LltQCRtx2/xoB8JN1kbmFtZW5qdW1iZiBtYW5pZmVzdGpleGNsdXNp"

"b25zgaJlc3RhcnQYIWZsZW5ndGgZFooAAAI+anVtYgAAACdqdW1kYzJjbAARABCAAACqADibcQNjMnBhLmNsYWltLnYyAAAAAg9j"

"Ym9ypWNhbGdmc2hhMjU2aXNpZ25hdHVyZXhNc2VsZiNqdW1iZj0vYzJwYS91cm46YzJwYTphM2RmMDJjYi0zNTcxLTRmYzktYjgy"

"NS0wYzc0NmVmYzc3YWYvYzJwYS5zaWduYXR1cmVqaW5zdGFuY2VJRHgseG1wOmlpZDoxMTM0MDczMS1iZTM0LTQwNTUtYWM5My1k"

"MDU3ZDA5MDFhY2VyY3JlYXRlZF9hc3NlcnRpb25zg6JjdXJseC1zZWxmI2p1bWJmPWMycGEuYXNzZXJ0aW9ucy9jMnBhLmluZ3Jl"

"ZGllbnQudjNkaGFzaFggykENXMy5WiDqp0IIdqTy4y8MIrJWyKVXD9Aoxpi3TpaiY3VybHgqc2VsZiNqdW1iZj1jMnBhLmFzc2Vy"

"dGlvbnMvYzJwYS5hY3Rpb25zLnYyZGhhc2hYIN8NrVhYZFNFehVgptFne63G+dUDm5Et9Ox1xMSlvVqiomN1cmx4KXNlbGYjanVt"

"YmY9YzJwYS5hc3NlcnRpb25zL2MycGEuaGFzaC5kYXRhZGhhc2hYIO15NGjOTYMe+Dx5jeTpxA1o0kxNqKrp8JKZtwqj8LMJdGNs"

"YWltX2dlbmVyYXRvcl9pbmZvo2RuYW1lb0FudGhyb3BpYyBGaWxlc2d2ZXJzaW9uZTEuMC4wa3NwZWNWZXJzaW9uZTIuNC4wAAAQ"

"OGp1bWIAAAAoanVtZGMyY3MAEQAQgAAAqgA4m3EDYzJwYS5zaWduYXR1cmUAAAAQCGNib3LShFkCEqIBJhghWQIKMIICBjCCAY2g"

"AwIBAgIUQOWgCu7COdC+uIP6BkIFPWdVEwAwCgYIKoZIzj0EAwMwSTEXMBUGA1UEChMOQW50aHJvcGljLCBQQkMxLjAsBgNVBAMT"

"JUFudGhyb3BpYyBDb250ZW50IENyZWRlbnRpYWxzIFJvb3QgQ0EwHhcNMjYwODA3MTg0MzU2WhcNMjgwODA2

```
MTkOMzU2WjBEMRcw"

"FQYDVQQKEw5BbnRocm9waWMsIFBCQzEpMCcGA1UEAxMgQW50aHJvcGljIENsYXVkZSBDb250ZW50IFNpZ25pbmcwWTATBgcqhkjO"

"PQIBBggqhkjOPQMBBwNCAASYegpry1AYBRTVNL1CpTlbROnY3dey+UrsF9C3phYrATN3ZHf93Mo8RQN0KOUuOn19P4oWNFWe5n2/"

"She9N7eTo1gwVjAOBgNVHQ8BAf8EBAMCB4AwFQYDVR0lBA4wDAYKKwYBBAGD6F4CATAMBgNVHRMBAf8EAjAAMB8GA1UdIwQYMBaA"

"FM5R4gSBTmRbI/jjxM+aPpzB11zCMAoGCCqGSM49BAMDA2cAMGQCMDFzHRSeAXrSy1WOzkbhPZ6Km2wGTmZ/2gK18k8BQGXyqz88"

"Rdrz6CTX9flAnYNVxgIwcF9c3fVhqmJKpi+UhasNUMko69cyX6STPfta3Q8EjyzDjzoyrol46FP6VFHhvUcJoWNwYWRZDZ4AAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
```

"AAAA

"AAAA

```
AAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAA
```

```
"jlX9Ozj6dwO82uXRXnkHAAfS/3KblyX5SBaNzl773DvGPfp58pHsHg1wpwAAB9KfR+73+g33T/voTws8mt2j
AdYGALCvGja+Osn7"

"s3cPOzHto59X7+/ysp+3APB9/SH/1yd5MKOBOetfzaM/GvBgl6eJKQEAevorxt+RRcPRX2XuWM2jn5fvyIK7
BABmruaGrOjy3ix6"

"j1b4r8+xlcUoznu7vE6sCwCYrWoArk1ybxY9RUP+63f0pwTu7fI8EQQAzE4tCLspydezaPzHbqgcR3tUHn+9
y/vE4kCA2agK//VJ"

"HklrEKzyn89Ref1IFosDBQEAa64q+tuz6A2a75/fUXl+Oq0sJIIAgLXVb/x3YrHf3I/+4kBBAMCaWm785/S1
vdu948x5jv7vjp3u"

"oa5NTQkIAgDWzBwa/xrNOJM2rHO6F/6FRef6e+t47QQBzI4dsVh3x9Mq99uT3JXWgK3DbnDVcO2kvZf9bmc7
mUWA8LnucS+bSV7Z"

"PR5L8qx9/mYFFrWD4rpcz8Okv5rkt7MoO7CWVv1DC/tZt8a/Gvzk7AZ/K+3WtoeSPJDk2OnuS/JEFo3+dlow
sJ9npV2jCgYuTbtd"

"7rlJXp3khiQvOMf5k91bKq8aQQCzsqoVIZzPRWnD1m9J8gdZ3ca/GqXlhvW7ST6b5BNpDf7fJHm4e30/52uc
t8/z82ckuT7Jj6UF"

"BLckubF7vf83drLa13szyS8m+cMsyhIAE1e9O5vS7vXeyuqt9q/59/5rX0nywSQnkrzoHO/9WFqv9Xh3HTbT
GuE6zqf/u/X/6++d"

"K3h4UZemD3Zp7Kf5QtchjHXtt9LKTmOWZMdAgImrivpVWWzysyoNUC1G6y+yeyzJ+5L8XJIrl97rRnY3zkP0
tmskos67fM4rO9L6"

"vrSO7/fepnxUmXkkrSwlggCAyarG6eqOYfGdrMYOf/1V6HXcn7Z24bql91g98inNs1dAsNxAXpf2Hu7P7ve2
KoFA5ckDaWVqFac0"

"ANZe9YaT5KNpFfcq7O3fb/hPJbknyU9nd2PaH8qfuv7UQdlMe0/3pL3Hvd77VI8qQx/t3steox4AjKga/zvT
KuwnM37jcb6Gv3rB"

"p5LcnbaYrm9VGv1z2ev2xBvT3msFAnuNfkztqLJ0Z/ce7BEAMBHVyLwlu3ttUzyWG7zlhn+VevsHtdeoQAUC
ewVEUzyqTL2lS7/1"

"AAAjq3nZF2ex4n+qDUm/4f/TJLf23kdtvrPuao+BcmvatdjrGk3pqI2UHkkra6u83wHAyuv3LD+Z6TYg/V7/
19JumStzafiXLQcC"

"J9KuTeXhFIO4ysNPZj1HagBWRjUgd6ZVzFMc+u8HJL+ftotecnYDOFf9AOgFaddor2s3laPK2J299AMwoKp4
b81OG/9KO9eSvLWX"

"dovIzta/Jm/NYjRgyvlaUziCAICB1PzrFWl739f87NgNQx399HwkybVdug0Z769/18C1adduJ9Nb11HpeSit
DNofAGAg1Vt8T1qF"

"PKWh4n5a3rlHmjm//rV6Z/a+tmMflZb37JFmAI7A8tD/FBuFR5K8uUvnscxzkd+F6l+3N2exrfMU89tUAMAR
q6H/KzO9of+aF/5O"

"ktdO6b3oCK7B3NQ1fE3ate1f67GP/lTAlTEVAHBkqof1rkyrIah03Jvkqi6NhoQPT13Lq9Ku8RTz/l1dGoOC
AByyGg5+ado2slO5"
```

"V7wagPemLQhLNAJHoa7pFWnXeipBQO3xcCqtbCamfAAOVTUAH0ireKcwF1wN0F29dKr8j07/2t6V6QQBVRY/0KVNAAhwSKpCfWOm"

"0/hXGqrxn+uOfkPrb6BUQcCUysMbu7QJAgAuUC38uyTJpzKNhX/LPX/39w+rv1/AVEYCakHgp9LKqgWBABeoKvq3ZRq9veXvh9f4"

"j6MfBHwOOwgCqmy+rUuXUQCAp6nf+/9Cxu/9VwV/b5Kr4x7/sdX1vzqLuwPGDBBrFOALMQoAcEGm1PuvwONbSa7p0qXxH1/lwTVp"

"edPPK6MAACtoozsuzvi9/7rV6/EkN3fpU7FPR+XFzWl5NOYtov1RgIuzKMcAHFBV6rdl/N5/zS2f6NJkk5/pqTw5kd15NuYowG1d"

"mgSLAE9BDe3el0UPfMzK/O4uPRr/6aq8uTvjBo01AnFflx5TRQAHVD2mN6UNqY419F/nvT/JM2PF/9TVnQHPTMuzfh6OUXa20spw"

"YhQA4ECqsvxwxuvJ1ajDk1l8uY9KfPoqj16TlndjrQeoMvvhpXQBcA41XPqSJN9Nq7zHqMBrDvmOLj2G/ldH5dUd2Z2XQweQ22ll"

"+CVdekwFAOxjCpV3DRt/Ou3raA39r5aaCrgoi68QHmMqQBAJcEB1y9Rlad+xPsatf3XO0zH0v8r6UwGns7g9b+hAcjutLF8WtwQC"

"nFNV2j+b8XptNXf77qU0sXoq796d3Xk7dBCwk1am+2kCoKfmSD+UcSrsmrd9JMlVsZXrqqutpK9Ky9Mx1pNUGf5QlybrAACWVEP7"

"/CSPZdEgj1FZv71Li97a6qs8fHvGCyp30sr087u0CCoBemqB1G0Zp6Lub+F6PHr/66JGAY5nvC2ll3cGtBgQoKeGRse699+XuKyv"

"sb9UanlPANMAAJ3qaV+b5GR2D5OO2fv/fHyN6zrqf6305zP8KECV5ZNpZbzSBKMSiTIF1UP7mSRXplXOQ1aQO935fivJqbTPxc6A"

"5+do7aTl6am0PN7IsPm7kVamr0wr44kRJiZAAMAUVGX8pqXnQ537WNp3yd+TRWXNeqmg8p60vB46yBuzjANMUvX0n5nkqxl++L92"

"a/utLh0WaK2vytvfyu68H3Ia4KtpZT0xDQDMXI1CvS7Df3FL3Rf+r0lekcVcMeup1na8Ii3Ph94XoL5g6nW99MBoFEDGVmXwDWnz"

"okMOv2+nNQh/kbY4bKN7jfVU+f35tDwfOr+30sr4G7rn6l9GpQAytpoLfWP3OMaw6O9F738uahTg90Y4d5XtKuvWAQCzNeb8f53n"

"n5JcvpQe1lfl8eVpeT9GmbMOgEnQ42FMVfm9Im2b1J0MVyHWVMNHkjyeNjSrR7b+dtLy+vG0vE+Gm3aq2w+fn1bm6zUYhQCAMVX5"

"uyXDz/9XxXvPgOdkWu7pHodshGsdwC3dc3Uwo1H4GFP1uF+3728dzXk3k/y/JPd1r1n8Nx+V1/ellYExRn+qzBt1YjQCAMa0k1YG"

"b+ieD9UTqwbgL5N8PXb+m5sqd19PKwPJcAFglfEbotwxMgEAY6lbsC7PIgAYqjxWpfunA5+X6ag8rzIwVENc57OhrezXrYkwOBUf"

"Y+n3hH4gwy4A3EzbkOVj3XPD//NTef6xtLIw1N78tRDwBzL8yBfsIgBgLFXpvSRti9ahGuEKNB5N8sXea8xL

```
5fkXO8rCkF8QtJ1W"

"5l/SPRcAMAoBAGPp3wKYDFv5JskDabeCmYedp1oH8HhaWUiGDUITtwIyMgEAY6lK8Ie7x6EqwTrvJ9MqfJ+B
+TqWVgY+2TOfKhCs"

"sl5lXwDKKFR+jKUqvesHPm9Vvp9eSgfzU3lfZWHonvj1S+mAQQkAGEPNt16W5JreaOM4lvY1sF/unqt856vy
/stpZWKo+rDK+jVp"

"n4EhF8DC9wkAGMtOkkuTXNc9H6ICrIr2e0ke7r3GPFXeP5xWJoZaCFhl/bqOz4AyyCgEAIxp6O1/q6L9apIn
otdFKwNPpJWJZNjG"

"uLYFhlEIABhDlbtXJnl2htsMpSr3b2XYHh/T1B8R+lbvtaNWm2A9O+0zkKiLGYFCx5guyji98H/oHo0AUGXg
H/b9raM79OUjnBeS"

"CAAY16mBz1e9uwe7RwEAVQaqTAw9IjTOZwC+TwDAGKrSfWn3OHSle/nA52P6hi4TVebrMyAYZXACAMZUe6EP
vQHLF/f9LeaoysTQ"

"G1LdsO9vwRESADCmJwc+33IAYAEg/e8ESIbviQ/9GYDvEwAwprGGPS8e6bxM11hlwtA/oxEAMEd6/ixTJpgd
AQAAzJAAAABmSADA"

"HJl3ZZkywewIABjTWPOuVl6zbKwyYe0BoxEAMKahV17bfIVlY29K5Y4URiMAYEwPdY9Db77y0n1/izkaOgCo
Mv/Qvr8FR0gAwBjG"

"3nzl8YHPx/QNXSZsSsXoBACM6ZKBz1eV7su6R5UuVQaqTAwdjA79GYDvEwAwptMZpxH+oe5RAECVgR/a97eO
7tynRzgvJBEAMI7t"

"7vFzSR5NK4dDNMbVu7smyWXdOS0EnK+NtDJwWVqZqNeO2k5amX807TOQLD4TMBgBAGPaSrI54Pmqcr8uyaUx
AkArA5emlYlk2IBw"

"M+0zAKMQADCWjSRPJPlq93yoEYDq8V3fe415qry/PsOOCFVZ/2raZ0AZZBQCAMZQFe33knyr99oQtpNclOTF
3XOV73xV3r84rUwM"

"NQxfZf1baZ+BjRiNYgQCAMZSle/DA5+3KtrXLqWD+am8r7IwdCP88FI6YFACAMZSld7fd49Db8Dy42nl3+Kr
+dpOKwM/3j0fekOq"

"KvsCAEYhAGAsVQl+vnscqhKsMv/qJJenNQIq4PnZSMv7y9PKQjJcfVjlrcq+4X9GIQBgLFXpfSnJmQxb+e4k
eXZ8J8Cc9b8D4NkZ"

"9pbQY2ll/kvdcwEAoxAAMJaq9B5K8kiGXQi1leR4kp/snvsczE/l+U+mlYWhbserQOORLL4HQADAKFR8jKU2
Q3k8i4pwqPn46un9"

"1MDnZToqz6sMDNX7r/M+lFb2h9oEC84iAGBMNQ87dE+oyv0bk7wg1gHMTZW7F6SVgWS4urA/8qXcMSoBAGOq
yu9vRzjvVpIrktzU"

"veazMB+V1zellYGtDN8QV5kXADAalR5jquHQT2T4bYGrJ/YLA56TafmF7nHIIfja/vcT3XPTT8AsVe/nmWnb
ou6kVYg7Axx1nn9K"

"uxWsnx7WV+Xx5Wl5POaZ+2pame+nBwZnBIAx7aT1iP41yX3da0MuBNxK8vwkP989H3IEgnFspuX1z6fl/ZDD
/1W27Osr85uxAJAR"
```

```
"CQAYW1W+f9k9jlEh/lIWPTTWW/XCf2mEc1fZrrKu9w/MWgWhr0vbHGWo4dhq8LfTemOvSKuQBcXr61haHr8i
Lc8r/4csb2fSynql"

"B0ajADK26hV9Ick3MuyGQDUN8Iwkv5LF3gSsp7rn/lfS8nzI4f/aAOgbaWW9XgOYtZp7/52OSvF0hh8F+GaS
56RVOoZm1O/l63PS"

"8nro3n+V6d/p0mO9CaPT22EKqsH986XnQ517O8k1abeF1cJE1kstuPuFtLweehOeMcs4wGRVZXhtkpPZfcvU
EMdWd77PJ7kki7li"

"1kOt7bgkLY+3O/J8yFGmnbSyfW0vTQBkMRr14bTK8kyGq6D753tblw6jAOuj8vJtGbdsfbhLh5FXgJ7j3eNt
GaeSrlGAL3RpMQqw"

"Hqr3fzwtb4fu/ffL8m1dmqqsA5BFY/v8JI9l99Dp0BX127u0GAVYfZWHb884gWWV4cfSynYisAQ4Sw2Nfijj
Vdbbad/VflWMAqy6"

"6v1flZanQ6/875fhD3VpMvwPsIfqrf1sWqU59FBtv8J+91KaWD2Vd+/OOAFlvwz/7FKaAOipe7Uvy+L7OocO
AuqcP5O8pkuXSnv1"

"VJ69Ji0va43H0I3/dlpZviz2mGBiDEcxJTtpFff3kvxuFvfoD6kq6ONJ3pvkoqXXmb7Kq4vS8vD4OutDqb0G
fjetTPvyH4B9VFD6"

"kiTfzTjztjtZ7Nx2R5ceK7dXR+XVHdmdlOOPJG2nleGXdOnR4QI4jxq+HWtPgKrAzyR5MqYCVkl/6P/JDP8F
U3Us3/uv7AAcQFWW"

"b0qbRx1jMeBO77z3J3lmFt8lzzRtpOXRM9PyrJ+HY5SdrbQynAgAAA6shkvvy6I3PkZFXue9u0uPqYDpqry5
O7vzbowys51WdhND"

"/wBPSfWYbsu4lflOFnPIJ7o0CQKmp/LkRHbn2ZhB421dmvT+AZ6CumXq4oy3hWsdNQLxeJKbu/Sp1Kej8uLm
tDwaa95/J7u3lL44"

"bv0DeFrG/hKX5Yp9J8m30r5ONjGOOwWVB9ek5U0/r8bs/b+tS5dAEeBp6H+N69ijAP3K/d4kV3dpEwSMp67/
1Wl5MoUgsXr/vlYa"

"4AJNaRRgJ4u55Y/2OqeSH16t+E9aXvTzZuwA8W1duvT+AS5AfxTgUxl/FKDf0NzVpVEQMKx+439XptH4V+//
U9H7Bzg0Vdm/Mbt7"

"WlPo7fWDANMBR+9Yzm78p1Qe3tilTe8f4JBUhfqBTKfSXx4JSAQBR6l/bafS8++XxQ90adP4AxyiqvxfmuRU
xr3Va68g4L1JrujS"

"qAE4fHVNr0i71lNp/OsW0VNpZTMRBAIcumoE3pXpNAD9dNyb5KoujTYLOjx1La/KYrX/1PL+XV0aBX8AR6AW
BF6Z9h3rU1gQuNwQ"

"fDqLLw+qrxLm6atr+Jq0azulxr8W/j2UViYt/AM4QtXDujWtEp7CWoA6Ki2PJHlzlO57BTw9/ev25rRrOtX8
vrVLp94/wBGrIeH3"

"ZLqNwk6Sd+6RZs6vf63emb2v7dhHpeU9e6QZgCNSUwFXZHpTATtL6flIkmu7dNsvYH/9+/uvTbt2O1kMtY+d
r3XOh/6viKF/gEEt"

"TwVMZV64f1Savpbkrb2O6y2erX9N3pp2zaaer4b+AUZSFe+dmW5jOR+2/v0kL+jS3N/QZs76Gyi9IOOa7XXt
pnJUGbuzl34ABlZD"
```

"xptJPpnpNhp1r/hOWs/2RO89zHUHweUA6EQWvf6p7PFwrmDuk1mUO0P/ACOp+dcXp60Un9p88V4NyE6SP81iCDmZTyCw3PDfmnYt"

"9rpGUzpqXccjaWWt1qEAMKJqUN6SVllPcSqg35DOG7m7k9y49F7WrWfZH6kpN6a9937DP9XArV+m3tKl39A/wETUIrI7OyrqJzN+"

"o7Hf0W/wTuXsQCBZ/UCgv6q/VMN/KnsHRFM8qizd2b0HizgBJmQji4p5Kt8Pf9BAoP59Ksk9SX46uxvOVRoV2Ku3v5n2nu7JouFf"

"fu9TPaoMfbR7L8ezGvkAMCs1L3t1kgeyOo3MXr3g+5PcnuS6pfe4mdYITWn++VhampZ7+9elvYf7c3bQM+Xh/uUA5YG0MuV+f4AJ"

"q0boVVlsHzulTYIOEgj0G8fHkrwvyc+l7TffV6MeFRAM0ThVkFXnXT7nlWlpfV9a2vd7b1M+qsw8klaWEvP+AJNXFfVNWdwZsCpB"

"QL8BWh4V+EqSD6bdMveic7z3fuPcnzqo43z6v1v/vx9k7OVFXZo+2KWxn+YzWc1rXyv+b+reo8aftWM4i3V1Udr87VuS/EFahb6K"

"Q7g7ab3m5dvOvpvks0k+kTZE/TdJHu5e38/5pg62z/PzZyS5PsmPJXl1klvSFvY9Y+lv7GS1r/dmkl9M8odZlCVYK6v24YSn4nha"

"D/T2JHdldYOAUg1rcnaPdCvJ19P2p38gybeT3JfkiSSfy2J/hJPnOcezsrhX/5VJLk3rBT83rcG/IW3Hvr3On6z2/fH9xv9Xk/x2"

"FmUI1s6qVoRwUOsWBJRqrHay9+12fSez2MimgoG9VKNfmxI9a5+/WcP61eCvy/XU+AOskbo98PYs5qVXZTHaQY9q4M+kDVefzoXP"

"v5/r763jtav1Frenca8/wJqYQxCwXwNXx5nzHP3fHTvdGn8ADsVyEFC95rEbIcc4R380Q+MPsOb6QUDt8iYImN9ReX46Gn+A2aiK"

"/vVZbBaOCjsGOg7nqLx+JK0MJBp/gNmoCv+mtNvnqjc4duPkONqj8vjrWWzyo/EHmJm6de7aJPdm0UDMZQHcnI7tLBr/e7s8T+zw"

"BzBb1QBckeS9WTQW1gWsz9Ff7PfeLq8TjT/A7PV3rntHFj1FUwKrf/Tz8h1ZWNXdCgE4ZP0tbF+f5MEsGg5TAqt39If8H8xisd86"

"7FoIwBGoBWFXJ3l/Fg2KuwRW5+jn1fu7vOznLQDsqT83/OtJHk1rTIwGTPvo9/of7fKumO8H4ED6UwIvS/KR7N3DdEzj6OfJR7o8"

"Swz5A/A09YeN+6MBF/pFO47DOeoLi3Zydq/fkD8AF+RYdo8G/HEWDZBpgXGO/nD/Tpcn/V6/Vf4AHJr+PPJbk3wmAoGxG/7PdHmx"

"Vx4BwKHp9y4vTvKbSb6ZRYMOp68ZHrrh78/zf7O79hfvkS8AcGT6Pc0XJrk7yakIBI664T/VXesXniMvAODIbWT3QrMbIxA46ob/"

"xt71Ph4r/AEY0UZ29OIrEHgiuwMBdw2c/+iv6t/pruFyw78ZDT8AE3IsuwOBlye5I8k3cnbP1qjA/tfkG921e3nvem7GPD8AE7Yc"

"CDwvye1JPp7dDd+ZzDcYqEZ/eVOlj3fX6nm966fhB2ClLAcCSXJLkruy+86BuQQD52r0v9ldk1uWrpWGH4CV

```
VmsE+vPWz0tyIsk9"

"SU5md4O4lXav+6oHBNXgn87Z6x9Odu/9RHb39ve6VgCw8vYaFbgurSH8oyTfydkNaTWiUw8I+g3+Xt+X8J3u
PZ7o3nOf3j4MTJQN"

"46ie7nZ3lKvTvrv+TUl+Im31+zOW/m8/CNjI7i8uGspB0vDdJJ9N8tdJ/jxtfv+fez+vzXtqdAAYkAAAxlcN
4XIwkCQ/mORHk7wh"

"LTB4WZJnDZq6gzuZ5MG0hv6vknwqyT8u/c5+7xUYkAAApqV60hvZu2f83CSvTPLiJK/ujhuSXJPkkgzzmd5O
8pUkX0ryQHd8Ocnn"

"knx76XdrpKOmCPT0AeAAas3AfrvePTvJ32WxgPCo5/mfSPIj50hL7Y5oTh8mzgcUpmO7u3fHqx718bQvwjmW
5Pq0UYCdHP1neitt"

"pOGN3bkuzaLB38hi0eJWDPHDpB0//68AE1K9/GSxiPCGLBbTDfXFOC/N7vv6gRVjBABWV00JvLp7HGJ+vc75
qu5RLx9WlAAAVt/W"

"+X9lLc4JHCIBAKy+Me7mcQcRrDgBAADMkAAAAGZIAAAAMyQAAIAZEgAAwAwJAABghgQAADBDAgAAmCEBAADM
kAAAAGZIAAAAMyQA"

"AIAZEgAAwAwJAABghgQAADBDAgAAmCEBAADMkAAAAGZIAAAAMyQAAIAZEgAAwAwJAABghgQAADBDAgAAmCEB
AADMkAAAAGZIAAAA"

"MyQAAIAZEgAAwAwJAABghgQAADBDAgAAmCEBAADMkAAAAGZIAAAAMyQAAIAZEgAAwAwdHzsBMJKNrH4AvJlk
J+O8j43u/Me6f6+y"

"nd4BsyEAYE6OdcdWWmW/NW5yLlil/3sjnPt0d/5Vv4Z9m93jdgQDzIAAgDk4llahb3dHkjwnySuzqPRX0WZa
A/zy7vkQPfE6x3OT"

"/Lu0a7t97l9fCQ8l+Xp2BzObWa/gBs6y6kN3cD79ivzVSf5Tkn+b5N8kedZYiWJSvpvk4ST3Jvn9JH+WVmYq
cDQawFoSALCuNrpj"

"O8mNSf57kp/J7h7/qvdcS73XIa1Tw7i8huIzSf5bkj/p/XxdygrAWusv8HtHklNZNFg1d72d3Yu/HPM9ttPK
RJWNev23k1yZZtUX"

"jAKsvX7jf1cWFfyZjN/QOFbj2MqivPxNkqvSCAIAJqyG+KvxfzJ6+46ndzzZPd6b5OqMM9UCwAHUXS13ZHcF
7nA83aPK0EfTRgBW"

"+a4RgLVUFfMb0nr8p6Pn7zico4KA30gjCGAtGM5iHdTQ7CVJPpbktWmNvzlbDkMFAo8nuSnJF7O4wwRWlgqS
dVC3ad2a1vjXPdxw"

"GKqxvzLJr6UFA8oXK08hZh3sdI+/1vs3HKb63oW3pm0gdSZGUFlxAgBWXfXOnpPk9VmPL/lhejbSAoBrk/xo
95pyxkpTgFl1VYZr"

"a9/t6JlxNGrO/8e6R+WMlSYAYNVVJXxzLMziaFVZu6l7NN3EShMAsC5+YOwEMBtV1gQArDQBAOvizNgJYDaU
NdaCAIB1YXMWhqKs"

"sRYEAKy6GoZ9oHu0MIujUmXts92j+pOVpgCz6qpS/kzalq3KNEelgsu/HTUVcEhUlqy6uu3v75P8Q9wJwNE5
ljb///HuuXLGShMA"
```

```
"sA42O3r/v9c9VzFz2La6x48leSiL7adhZQkAWAdVEf/vtECgdm2Dw7KTVq7uii+aYkOoxKyD7bRRgC8l+Z/dv7f2/R9wcFtJjqf1"

"/v8gi6kAWGlWTLMu6iuBLO+rqF+VVnG7ZYsLUaNLTyS5Jcn9MfzPmjACwLqoIf/HkvzHJP8SIwFcmOOsvvr3v6Q1/pvR+ANMUvX4"

"bOrySFoF/mT36HAc9DjT+/ftaY4HgEmrivqmJA+mVeJbSU5nOatzOJaP7bSG/3T3/LEkv5xG48/asQaAdVXD/1cn+R9JTvR+tpVW"

"wa9L+d/I8NN51WCugyoL/fUif5bkvyb5dFrjb9EfwArpN4q3JnlfkpMZv6fpmOZxOsn/SfKfs2ARKWtrXXpAcC7VO67FgNelTQ28"

"NsnNSS5Kq/xX8bNQ+x1cn+SHM8z7qHP8S5L/e8TnOmo73eMDaQv87k3yue61uqtkXUY5AGZrM+vbm/uNLHqwR91LrsVxHx3knQ1v"

"eSoA1paFLcxFjQAcy6J3V43aqqq56UtGOvexrMc98TVVtNOdbh1lFgQAzM2qN1Z91WCNEcTsZHEtV/2arnr64WmxERAAzJAAAABm"

"SAAAADMkAACAGRIAAMAMCQAAYIYEAAAwQwIAAJghAQAAzJAAAABmSAAAADMkAACAGRIAAMAMCQAAYIYEAAAwQwIAAJghAQAAzJAA"

"AABmSAAAADMkAACAGRIAAMAMCQAAYIYEAAAwQwIAAJghAQAAzJAAAABmSAAAADMkAACAGRIAAMAMCQAAYIYEAAAwQwIAAJghAQAA"

"zJAAAABmSAAAADMkAACAGRIAAMAMCQAAYIYEAAAwQwIAAJghAQAAzJAAAABmSAAAADMkAIDVtzOTcwKHSAAAq29zJucEDpEAAFZX"

"9cLv7x43BjznA92jOgRWlA8vrL6Hk2xn2M/zlwc8FwCwZCPJJUm+kNY73+oej+rYTvJEkpd359eJgBXlwwurbTPJqSQf6p5vH+G5"

"trrHB5L8XVr9cZTnAwDOoYL4V6T1zLfSGuWj6P2f6R5v685pISAAjKga4v+V1kCfzuE3/jW18HdJLk4LPIZYdAgAnMOx7nh+km+k"

"jQAc5lqA7bSgYjvJT3Xn1PsHgAmoBvmnsxiuP6wg4FT3eEd3juNH/WYAgIOrIOD2nD1v/3SH/ev/35U25K/nDwATVA3OLyd5LIs1"

"AU9lNKCG/Ov5nd3f3Ih5fwCYrBqif22ST+bshr2mB7Z7x5mcHSh8Lcl/6P6WRX8AsAJqJOCitNGA+3PwEYCvpM33X7POt4A1IqKH"

"9dXfqGczyc1J/n3ayMANSa7ofnYmyefStve9J8lfpEOf1P+rDYCANSIAgPW2kRYILDfilyW5tPv3dpKTSz/fzGJDIWANCQBgHioQ"

"2Mhi3r+vP8ev4YcZEADAPC1/9jX4AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAElFTkSuQmCC"
```

```
)

_PWA_ICON_MASK = _pwa_b64.b64decode(

"iVBORw0KGgoAAAANSUhEUgAAAgAAAAIACAIAAAB7GkOtAAA0TUlEQVR4nO3da3AU15XA8ds9o9EDkCzxsmQT
8xLIwoAVEGDwq2xL"

"Mpstsl6zlLOpmBhb4QOVKmqLbDaxU9FWKskHb4oqbyrlYBscVyq2d/HaZHcrWJaN7QDGgJEl8RIPY0eFYEEv
BLIeM913P5yoMxEg"

"BJqe6en+/z6kMBE9rb4959x77u3bSgEAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
mjRJxT0RDQDpwTAM"

"p/e6Zs2a06dPOzEupTE2DTiX6PTpO2vWrJFrGA6HqQgBSANOuX/x4sV1dXVOXAtsrf96ydyA/Lmurm7x4sVD
LiwAeJF0/MPhcE1N"

"jSx0CdoKn0RxVgpFo9Gamhrnwqa6hQHgMs76xcWLF+/Zs0eiGDWfUXIu4J49e2QowDpRAN7iVCd+/OMf0/FP
rPihwI9//OMhFxwA"

"UknqEoWFhdu3b5eYRcc/4ZxLun379sLCQkU5CEBqOYsUq6qqWltbNR1/NzlDgdbW1qqqKhW30BYAksqpQtTU
1EiEouOfBM5Frqmp"

"GdIQAJAMUn/Iz89//fXXtdaWZfl+HzfvcK7266+/np+frygHAUgaCTdlZWUHDhzQWg8MDKQ6JAaRXPYDBw6U
lZUpcgCAJJBAs2zZ"

"so6ODq11/K6WSDK5+B0dHcuWLVPkAACuysjIUEpVV1dL6KHon3LSBNFotLq62mkgAEgkZ0vn6upqrbVt2xT9
PcKyLFl5JTmAraQB"

"JJKz3HPjxo3xEQce4eTjjRs3KpaHAkgUp++/adMmzbZuXuVsIbdp0ybFOADA6A2J/mm34Ee6xrFYLBaLRQdZ
V+H8gPx8Og5OpIHI"

"AQBGy6n8SPRPiwU/0hGWIJ6QWQrJH3LAtMgH0kySA6gFYXjcHLgy6fvHYrFNmzZVV1fHYjHPrjLUWktovrzP
e+HChQsXLpw8ebKr"

"q6uhocEwDOcPWmvnx+Q/58+ff9NNNzl/mDFjRl5eXl5e3pDPsixLUqNnY6s01gsvvPCd73wnHA5LIkz1ScGL
PHoHI+UyMjJkJ3rZ"

"4NOD6wtt27Zte8j2yG1tbYcOHaqvr29sbPzs88OHz586dKl3t7eGzh+dnb22LFjSOtLp0+fPm/evKysjlz
5kyYMGH4E/AIabJ/"

"/dd/rampkaZM9RkBSBPS2ZcVn17b3O2KPPHlnba2tm3btq1fv37RokVDQKBQeFLqK+B+4YkzPy8tb
tGjR+vXrt23b1tbW"

"5pyAlIm8dpWkFiRrQz07egPgLRIsVq9erT224lPmaZ3/bGlp2bJly4oVK4oVK8aOnHcMIwbLtTIv3US
w5CUMH78+BUrVmzZ"

"sqWlpcU5JZlhTsW1uQJnbejq1asVOQDANcman/Ly8p6eHu9s8Rbfv+7u7n7llVdWrFiRm5vrnLYEfbfXvTj5
ID4Z5Obmrlix4pVX"

"Xunu7pYzjH+pb2pJC/b09JSXlyv2DQUwDIlrO6ZNk31+vBD946s9DQ0NGzZsuPXW5OTltJNSiZjJRnEh9Rb
b711w4YNDQ0NcrZS"

"FOrdldPOaWitOzo6pk2b5jQxAPwVWdmSk5Ozb98+7YF9fuJDf21tbUVFhRNtUxj3LzckE4RCoYqKitraWjlz
L6QBOYF9+/bl5OR4"

"efESgJSRGvHLL7+sU73kP37iQUJ//El6Nn4ZhhFfZ49PAynfOkka9OWXX1ZMBgAYQoLC+vXrdUof942vnseH
flnG49nQH08GBE6l"

"JT4NpHalkDTr+vXrFTkAgEPKFOuWLJFHXlMVpJzQ39TUtHz5cjk3Cf0pvTw3KD4NLF++vKmpacivmWSyMDQW
iy1ZskQxIQxADS52"
```

```
"zM/PP3nyZKoqFc6i9a6urmeeeSYSiah0Dv3xnDQQiUSeeeaZrq4unbpHK6S2dvLkyfz8/NEskAXgE1IN2Lp1
q05R59RJOdu3b581"

"a5aclQ9Cfzzn15k1a9b27duH/OLJJE28detWRSEICDgJTOvWrdMpmvh1Ov7f/e535ZRubJrX+/3Z+Cni7373
u85QIFXXfN26dcp3"

"iRbASMmKwBkzZnR3dyd/ftKZ7925c6d0/G9gUx3vx/0hnN9x1qxZO3fu1KmYGZYr393dPWPGDGfDVwDBIr2/
uro6nfTij/NxGzdu"

"lG3mrrccER/6MzIypkyZcu+991ZVVaVFWUNOMiMjQ96wlqrrX1dXpxgEAAGUwuKPRJ+Ojo5Vq1bJyVxXJzS+
yz9p0qTy8vJVq1Y9"

"9dRT1dXVTz311MSJE4f8jDc5v/KqVavkuesk5wAKQVBsBx1MpmlqradNm/bpp5+OGTMmmYUU2aq+vr7+ySef
rK+vv67d6p1N/E3T"

"nD59+owZM2655ZZwOCxbsNm2nZOTs3fv3k8//dQ0Tdu2Xf5VRst54OJZWdlLL71UVlaWzJcuSBro6em58847
T5O6ZRiG968YEo4E"

"EEShUMiyrP/8z/9cuXKlZVlJ6wBKgNu3b19VVVVnZ2c4HI7FYiP5h07oz8rKKi4unjFjxqRJk2zbloebJIFp
rTMyMs6cOfOHP/xB"

"KTXCpJJychHy8/Pffvvt8vLyZOYAafqtW7f+wz/8g9wSyflceAfzP4EjX/XKysqURP/Nmzc/+OCDnZ2d0vm9
5r9ygrtpmqWlpX//"

"939/1113FRQU9PX1DQwMxL+ZyzAMy7IKCgqysrLSJforpWKxWCgU6uzsfPDBBzd3vs3jzypDh6ciesXLmysrIy
mXcCvIMRQLBIPA2H"

"w42NjbNmzZLAmoTPjX9JoVJqhCUap+M/ffrOO++8c/z48fK9qvVrGzbzsrKeu+9906ePJkWVSCHc7ZJfgGn
bduGYRw7dmzevHO0HnO"

"JEQSPhcewQggWCTQfO9735s9e7azjAJHxof/eXJ2GuGZonvWuvc3NzKysoHHnggPz+/v7/fsqxr7mdZWFiY
yLNPCmmLUCj0ne98"

"54UXXkjaOEDaYvbs2d/73veSdj/AOxgBBIjEzZtuuunEiRP5+fkkOocncuXPnzp2bnZ3d398/ODhIXL178
398/wrMNhUIXL178"

"r//6r3SsaEtFy7KsZI4DpDk6OztnzpzZ1dWl5f8pKCgQMb+bn/iDUf/3Nzc5cuXL1L1myJBwBR2J9KBTKyMjQWnd1dW3btm1gYGCUgK
yUXLz8//5JNPpk2bmZ2b"
```

"5vYMrfN48Pz58xkEBAcjgEBwuv+ZmZlud/GcROxXrVol6/1HEv3HjRtXVVU1derU3t7e68pP8nEZGRmZmZm9vb1Hjhx5++23f//7"

"3OejOUT8NikjkwGdnZ2rVq3q6elxe4GmtEJmZiaDgEChmf1Puv8lJSX19fVJ6P5LzXrNmjVbtmy55jSmxJ3x48c//PDDWVlZ11X2"

"kYAYiURs2/6///u/EydO/OlPf+rt7U3A7+AZcgGfeOKJzZs3uzOZ4AwCysrKjh49yiAgCBgB+J8kgKeeeiozM9PtxT9S+t+yZct1"

"Rf/ly5dnZmaOPPpLqIpEIhkZGSOtLdu3b//f//3f5ubm6x09eJ8Efed6ujoZIBE/MzPzqaeeYjlQQPjnq4Irkmg4ceLEpqYmt3fK"

"lDp1Y2PjOqVL+/r6hl/2Ex/9I5GIPN87wk/JyMgwTbOlpaWpqam1tTX+gIn5TbxEFgVlZWXt3r173rx5rk4G

yAU8f/783Llzz58/"

"r3gmwO9I8j4XCoWO1t/61rcmTZpkWZZ70d8pIHz729/u6elRw8aOG4v+csDs7OwLFy689957b7/9dmtrq9Pl92uokt+rp6fn29/+"

"trNxtOufJfspTZo06Vvf+pbWmt2BfI8E4HNSk3niiSfcHtTLbmI/+9nPnE2er/aTXO/9Jbmft3KvUvf9JFk/9J6+y9U9J7+6VPb

"OfvE+TXOO6QR6+vrf/azn7m9bacUDJ944okLD9FylEC8jMJFg8++GBdXZ2rpQM5eH19/eLFi23bvmxJJ"

"XROdHR9++OG5c+eUfws+VyOFINMOP/7447KysiSO5kMPPfTuv3/6p39blZIpA8ei8WefPJ"

"RUUjif5y8Ozs7OPHj//3f//3uXPn5J8EKvqrwd83Go0++eSTbm/bW+RgLwLannFhYW3n333a7Wf2TF+q4rvhO4zy"

"cNP8+fNnz5795ZdfjiiT6hOKKhSCSyc+fOHTt2yKZAgV2bKEW2+vr6X//X//X7619d8umI0JEnffffdhYWFrs4bIeVI"

"AL4lM3jf/OY3eGH/OY3c3Nz"

"3fsaS7ng7NmzP/rRj4ZHJ4/4ldid2Fh4cKS/eV1j3V1dYcS/eV1YcPH/b3TO8IydX+0Y9+dPbsKuQ25u"

"7je/+U3FG4N9jQTg"

"WxIsVq5cq1fe7Yc+ilPjf7gBz/MLPPP03djn8SSq93Z2fhvDH/zA1VkQuWFWrlyZphvDH/zA1VkQuWFWrlyZphvq"

"YYQY3PmTfG9nzZrV1NQkT/+68SnO+6TmzFZGRkNDQ3/ngBsWTtTtU8ZGRkNDQ2u"

"vtNNFvXXOnTv32LFj"

"NIFfMQLwJwkKX//61yORiHurOKRD+UCjkRP8gF/2vSK55f3//z3/+c1cH"

"AZZlRSKRr3/962rw"

"doL/0K7+JPWfRx99VLlW/5GPaG5ufu78zepzu+ezPkzcHMgFLq"

"0UcfpfvvYyQAH5IWvT/84Q/dXtTRhxGsJ5C8rNK4+6t1+BBvWJUHy/v"

"7MyZM8vvKytx7nvN8888C7Kz888mHrsT8n8rpgv6JRfUi"

```
9R+JCEePHh2++y/h"

"afLkybNmzbpm8ce27aysrP379588eZJe5/CcQcDRoOfdu1ZSBXrkkUcUVSCfolF9SMLBww8/rFyr/Ogn9Be/
+IW8tXH4Tuidd945"

"kkWfWVlZn3/+efq+yjGZpFbW39//i1/8wr1BgNw8ciPRIr5EAvAbmTUtKioqKytT7nTcJPq0tbW99dZbsmb8
ameitZ46deqUKVMG"

"BgaGSUVSp7p48eKHH36oiDUjI892vPXWW21tbS6Vy+TmKSsrKyoqcnsjcaQECcBv5Eu7ePHivLw8l57/ksO+
+eab1ww9oVDoq1/9"

"6khik2EYH3zwQV9fH1FmhJwO/Oabbw6ThkdDDpuXl7d48WJFFciPaFG/kQB63333Kdeem5USzYsvvjhMsJbu
f2FhYUFBwfDdfyn9"

"Hzt27MyZMOz8Xi/DMF588UX3JmmlOeR2Ijf7DwnAbyzLMk3z7rvvVu5O2eT4+/bt279///AdT8MwSktL1bUC
RzgcvnTpOieffMKS"

"/+slQ7H9+/fv27dPls8m/CPkFrr77rtdOj5SiwTgK9KDvuWWW0pKSpQ7XTbpEr7xxhvD9Dql+19QUDCS6n84
HD5w4IDP3uWbNDIa"

"e+ONN5Q7Az5pu5KSkltuuYX3RPoPzekr8nW9/fbbx4wZ49KsnTymu23bNnWt2drbb799JFs+tLW1NTc388zX
jZEm2LZt28DAgBsP"

"fMiwbMyYMbfffruiCuQ7JABfke/nOqVLlTtraaTmUF9ff+LEiWEWa8qazqlTpOaj0Wsu/mlsbCTO3zAZh5O4
caK+vt6lqWBpZbmp"

"SAA+QwLwFfmu3nXXXcrN+s/WrVuHqf/I3O+ZMiU7O3uYeKS1jkQibW1tpO6dovs/GpKJt27dqtysAslNxSSN
z5AA/ENG6zk5Oe6N"

"1uU9JDt27FBXjzVSKZ4zZ478wPBvB2tsbCSmjJJc4R07dsibeRJ+fKeumJOTw9MAPkMC8A/5Zo4fP378+PHH
hQQgX/4TJO4cPHhw"

"mGqDFHb6+vpisVh2dnYkErniq9tDodCFCxe++OILFfjXvIyS1OUOHjx44sQJN1ZSuX1fIYVIAP4h38w5c+a4
1FOTyPLRRx/J9g/D"

"/GQOGt2+fftbb721e/fulpaWaDSalZUVnwls287IyDh16lQsFmNhyejJthAfffSRcqFK44ws58yZoOgA/sJ3
zz/km3nnnXcqd2q1"

"cnyp/4wkCnR1dRO8ePDtt99+88O3d+zY8fnnnOejOczMzEgkEgqF+vv7x8/ruj+J8L1Ns31kttJbiOSgJ+E
U3OCSBiJpEVFRS4d"

"3zTNWCzW2NioRpZgnEjx5ZdfHj9+/pPjx42PGjCxjO4yZcqU9vb2rq4upn8TQpqjsbExFou59wpfubVo
L8CLpJby7rvvaq1j"

"sZhOKNu2tdbnzp3Ly8tT19kNNAY5fzN27Njc3NzrPQ6uRi5jXl7euXPnnPnnMZKILmd3n33XcWOQP5CW/qEdKWz
s7MLCwuVzPASqnm"

"5uaLFy9e7449ThxxMsGlS5e6u7sV3ckE0Vqbpnnx4sXm5mblzjSAUqqwsDA7O1vaHHR6qQAPxDax2JRGSc
nvCvqETq+vr60ew7"
```

```
"NiQTJPQEgO6eBqivr1cupFVprKKiokgkQs72ExKAT7j9FZXjHzt2LCFHkzSQkEMhnjSQS88Aute9QKqQAHxC
vpO33HKLS4NOOeCR"

"IOcUdRtPkkaRBnKj9aXAeMstt7hxfKQKCQDXJhmlt7e3tbVVkQA8SRqltbW1t7eXtVUYIRKATOinbMmSJcq1
hwAGBgZIAJ7lJIDh"

"99++YXJTyQ3GCMA3SAC+EolE3Dis28EFCZGEJO3SDYZUIQH4ysDAgBuHlWjSOtJCecGznDJdSOuLci0BuHSD
IVVIAD4hX/gJEya4"

"9xE5OTnuHRyJ4mozyQ1GD8A3SAA+Id/JsrIy5VqJ9sSJE+4dHKMnTSPN5NLB5QYjAfgGCcBXYrGYG4eVLzwJ
wOPiE4BLMdqlGwyp"

"QgLwFVejMxOAacHVZiL9+wwJACPFwD8t0EwYORIAAAQUCQAAAooEAAABRQLwFVfrvxSX0wL3AEaOBOAr4bCL
7/jMyMhw7+BIFFeb"

"ydUbDMlHAvAJWZ/X3t7u3sEXLVqk3NlpDgkhTSPN5NJ6TbnBWAzqGyQAn5DvpEsvhBL0/tKCS83kvBJOkQB8
hATgK6FQyNWD8833"

"OMMw3L4H3Ds4ko8E4Csu1X/lJcClpaW5ubm2bZMGPMgwDNu2c3NzS0tL1WCTJRzzQD5DAvAJGaEfP35cuTZC
z8jIYDcIj4tEIi7F"

"aLmp5AZjLZBvkAB85eTJk8qdV8JK77K4uFi51rvEaEijFBcXuzRKkwPKDQbf4JvsK6ZputQ7k/eNFBYWunFw
JEphYaF7b+zRWpP7"

"fYbm9AlZAnjo0KHu7m430oAccN68eYpFIJ4kjSIN5Ebrm6bZ3d196NAhxVJgHyEB+Eo0GnXppX0SX26//XZF
CdiTpFGkgVzK0AMD"

"A9Fo1I0jI1VIAD7h9NFkmi7hfTSngxmJROgAepBt25FIxKUhmrT48ePHXRpfIlVIAP4hxd8zZ864dHCl1JQp
U26++WZqwV4jQfnm"

"m2+eMmWKcm0EcObMGZkKcuPgSAm+xv4h38zDhw8rF6o0shAoJyenpKREMQ3gMdIcJSUlOTk5biwBkttJbi2a
3k9IAH7zpz/9Sbnz"

"LZU6wF133eXS8XHDpDmkadwo0Mnx5daCn5AA/EO++fv3749Go26UaCQK3HvvvYp1IB4jzSFN4OZuNk0zGo3u
379f0fSAN8k3Pz8/"

"v6OjQ2tt27ZOKDlgV1dXUVGRYhDgGdIQRUVFXV1d7rV7R0dHfn6+ot39hRGAf2itDcO4dOnS559/rtyZBrAs
Ky8vr7y8XPE8sGdI"

"Q5SXl+fl5VmW5dIEwOeff37p0iX3njJDSvAd9hUZqjc1NSl3hury5f+7v/u7hB8ZoySN4kZOlhupqanJpdIi
Uojm9BXp/b333nvK"

"naG67Ab88MMPjxs3zo3OJq6XDMvGjRv38MMPK3e2a3b7pkIKkQB8xemsWZbl0jywbds333zzvffeaxgG/cGU
MO3TMIx777335ptv"

"dmmnbtM0Lctyb1iJFOIL7CtSATh27NjZs2ddKtdKCFi5ciW1YI/QWq9cuVK5VvQzDOPs2bPHjh1T1T7AICeJwU
Ad58802tdSwWS+yC"

"EGdNyLlz5yZMmGAYBjWBFJLrP2HChHPnzmkX1v/owVvozTffVLwLwOzI8YAfiNROQPPvhAudNfk6LzxIkTH3nk
Ea01QSGFQqGQ1vqR"
```

"Rx6ZOHGiS1MycgvJ7USyB7xO6vILFy60LMuNLqH0Cm3b3rt3r2maTAOkkFz/vXv32rbtxmhPa23btmVZCxcu
VCz8BdKCYRiZmZnN"

"zc1aa8uy3IgLlmVJXHD7LeS4mlAoZBiGZHr3Wllr3dzcnJmZSfffl0jpPmSaZn9//+7du5Vryza01qZprlu3
TjMrmDpa63Xr1rm3"

"P7PcPLt37+7v76f7D6QH6ZJ/7Wtf066NAGzbtm27v79/9uzZrAdNPln9OXv27P7+fmkLN1pZbp6vfe1rihlg
IF3IaH3cuHGygbt7"

"MwFa69/85jeK6JB0csF/85vfaHfWeunB2+bMmTPjxo1TzAADaUQCxMsvv6y1jkajLgUIy7L6+voYBCSZ0/3v
6+tzb6pfbpuXX35Z"

"keCB9JKEKpBmEJAiSej+a+o/QPpyqkCtra3atSqQMwgoLi5mEJAc0v0vLi52tfsvh21tbaX+4298Y/1Jax0K
hS5evFhXV6eUsizL"

"jU+R3SYyMzN/+tOfat4WmxRyzX/6O59mZma6d83lhqmrq7t48aI8bubGpwBwiwzbH3jgAffWiQupQlRWVipq
BS6Ty1tZWandLP7o"

"wec8HnjgAUWbAmnKMIxwOHz8+HHt5kyAFCKOHj2akZEhTyel+vf2J3nmLiMj4+jRo1J8c69BtdbHjx8Ph8O0
pr9RAvIzOzRjsdhv"

"f/tb5eY+jqZp2rY9e/bsp59+2rIsOowuCYVClmU9/fTTs2fPtmIx5nWAdCXdt8mTJ1+8
eNG4wO1rIXzcDA"

"QFlZmaJo4AK5pGVlZQMDA7IXk3tNadv2xYsXJ0+erJj+BdKaBI7f/e532rUHAoSUpA8cOEAhKOGc4s+BAwe0
y9V/uUl+97vfKRI5"

"kO7kO3z//fdrN6cB4mNHTU2NUiocDqf4N/cRuZg1NTXa5SyuB2+S+++/X5EAAB+QMu6OHTvc2zTYIdWJiooK
RfhIELmMFRUVSWu+"

"HTt2KDZ/BvxB+o/f+MY3tMvVAz24Iujs2bOTJ0/mObDRk8e+Jk+efPPbPbsWVdX/gi5Pb7xjW8oxnCAP8iLA7Oy
stxePhgfRGpra5VS"

"rCMcDVnFq5Sqra3VyUreR48ezcrK4mWfgH9IGWH16tVJiCN6sE69ceNGRUdyFOTSbdy4Ubtf+teDN8bq1asV
5TvAT6Qak5mZmZxB"

"gBNNqqurFTnghshFq66u1knJ2U73PzMzU+pOqb4AABInyYMAZ8Zy5cqVSqmMjIxUX4B0Ipdr5cqVenBi1u32
ovsP+Fn8IEC7vyRU"

"D+4n09HRUV5erhgHjJhcqPLy8o6ODrf3cXJaSmtN9x/ws6RtJTYkspADRi4++uuk5GnNdn5AQMjX+/3339fk
AO9JYfR///33FdEf"

"8DcZ4C9YsCAajSanuKwvywHMB1yRXJYkR3+ZqolGowsWLOC5DcD/pJf3y1/+UidrEKDjcsCiRYsU44DLyAVZ
tGhRMqO/HrwBfvnL"

"Xyq6/0AQSEcvPz//zJkzyZjFPJB3d3da9asUUqxYZyQjd6UUmvWrOnu7tZJjP7S+mfOnMnPz2fuFwiB/Wrl2r"

"BlNRqi9GKjlhd+3atUMuURJIO7P0EwicUChkmmZdXZ1OYiFIxz0f8Pzzz0vdI7DlIOfXf/7553Wy1vs7pBXq

```
6upM0yT6A8EiXe8Z"

"M2Z0d3fLU6BJCz1O9Nm5c2dxcbEK3pZBziY/xcXFO3fu1MnNwVpreRq8u7t7xowZio0/gQCSft+6det0cgtB
Qj6xra1t1apV8efj"

"e86vuWrVqra2thRe/HXr1qnAXHYAQ8mXP/mFIOF84nPPPZeXl6f8PhRwOv55eXnPPffckIuQ5MteV1eniP5A
kMkk5IwZM2TXgSQX"

"gvRgLUJr3dzcXFVVJWfly1kB55eqqqpqbm7Wg/uvpeSCd3R0zJgxg0l4IOikDyj7jiW/FiGcz33uuecmTZqk
4hZH+oCz4HXSpElO"

"xz+1l1p26PPNFQZw45K87/wVOU8knD59Wh4UUEql+wIVWWolf16zZs3pO6fjf9Pk4z0NAIaS7nYoFNqzZ49O
RVXa4Xx0XV2dvFhY"

"pWcaiA/9FRUVMsvihWu7Z88eaWsfz7UAuD4yGTB9+vS2tjbbtpNfm3bEv/q8trZ2SBrweNiSVBof+uVtjjrp
y/wvv6q2bbe1tU2f"

"Pp3SP4ChpJf90EMPaa2j0WgKo5XWOhaLOXWS+DSglAqHwx6MX6ZpxhdV4kO/ZVkp7PhrrW3bluLPQw89pCj9
A7giCWHr16/XqZsM"

"iDckDTz++OO5ublyqrKkMuWZQOK+My7Jzc19/PHHvRP6hTTl+vXrFaV/AMOQ7uGmTZu0N3KA/uviSUtLy7PP
Pjtv3jznhCUTJLM6"

"JHWeIY8szJs379lnn21paZHzjC9kpZYO4qZNmxR9fwDDcwrE77zzjvZMDtBax2IxJ6TGYrFdu3Zt2LChtLQO
/uQlLksVPoH5QK6J"

"c/D4/6u0tHTDhg27du2KPzePhPhH492HzvvPOOitt4DgCuyjRN0zTHjx+/d+9endKFK5dzytlCMkFNTc3SpUtz
cnIu/0XC4bBEbRki"

"iKv94s4PyM/Lv728ypSTk7NO6dKampr4uK89MHEyhJzb3r17x48fL22a+HsFaY4eAa7ANE3btvPz8z/44IO5
c+daluWp6oHWWkJt"

"fEW7paVl//79n3zyyb59+dLa2mpZ1hX/+dVCoW3bV/z7UChUVFR0++23l5eXL1iwYOhHChVOmTHH+31gs
JkMET/Wvpcmampru"

"u+++zs5OadBUnxQ8x0O3LDwlFApZljV37twPPvggPz/84osvosvWlpa9u3bd/78+aamposXx4/ftwwjL6+"

"vt7e3iseLTs7OysrS2tdXFw8bty4yXw8fMqUKbfddddv7Oy87777mpqpqapClT
fVLwIs/du/AOCRzl"

"5eVvv/22Z3OAwym/XK3c0dnZaRjGxYsXT5w4ccUjzJw5c9y4cXkfwrr/Pz8Kx7ftm2pEXn8Okj0r6qq2rdvH9Ef
wyABYDjhcDgWizk5"

"wGu1oKuJL4WrwfnPkXfV5R9KzcSI4+IZJ4g0kBP9pflSfVLwrjS4p5FaaZoDLifJwMkKl3OifFrE+ssR/QEk
nsy1lpeXt7e3a4+t"

"C4KQRmlvby8vL1c88AUggaTXX1xcLGtDBwYGUh3x8BfSHHv37pVXbKbpEA2Ad0lYyc/PlxzgnWfEAk4aYu/e
vTJ3TfQH4AonB7z+"
```

```
"+us61Ttcwtlw4vXXXyf6A3CdswLy+eef13HvdESSOW+UfP7554c0DQC4xXlDy9q1a6X+wLRwkskFj0aja9euVX/9/hkAcJ3kgGXL"

"lrW2tmqmBJJILnVra+uyZcsUZR8AKSGhp7CwcPv27Tqlb7sNCOcKb9++vbCwUBH9AaSQE4BqamokSDEUcIlzYWtqaoZcfABIDWfv"

"naqqKikHsToosZzVPq2trVVVVfHXHABSTx49LSoqevXVVyVsMRRICOcyvvrqq0VFRYqnfAF4kFOReOyxx06fPq2ZFRgd5+qdPn36"

"scceG3KRAcBb5HVa6q+HAlSErlf8K4Wdjn8y33sMADfIqVEsX768qalJAhkVoRFyLlRTU9Py5cuHXFIA8DrnrSmRSOSZZ57p6uqS"

"oMYjY8NwLk5XV9czzzwTiUQUL3MHkKacgvXs2bM3b94sm1ZalkUaGCIWi0m5f2BgYPPmzbNnzx5yAQEg/RiG4ZQvlixZUltbKyGP"

"NCCc0K+1rq2tXbJkiVyrcDhMxx+AHzjbBymlKioqnDQgs50BnCIe8ovX1tZWVFTI9WFjHwA+FP/4UnwaOH/dEfa3IUOf+NDP410A"

"fC6+h7t06dKXXnqpv79fomEsFvPrgEC6/E7o7+/vf+mll5YuXSrXIX6EBAA+F7+qvaSkpKam5siRIO64jEajEvfjl8AeOXKk"

"pqampKREfnfnyQkACJb4nm9mZubjjz++bdu2vr4+J1ymaSa4PO739fVt27bt8ccfz8zMlN+XWj8ADC2AlJSUbNiwYdeuXfG18rTI"

"BJfH/VgstmvXrg0bNjhdfkXoB4AhpBgSHxlLSOsvzwSWZXkqGThBP34S24n7paWlzq8jeY7FnfAO7kV4jiyGkXgqf1NaWlpZWXn/"

"/ffffffd48ePd35Sa21Zlhp88Dg5sVWCvpzbkIDe3t6+c+fO999/Xkths27ZtOwmnB4wcCQDedXkmKCgouPPOO++5"

"555777133rx5EyZMiP956XOrd16PLgc3DGNImmlra2tsbPzwww//+Mc/fvrppx0dHfL3xH14HwkAaUAywZBgmpeXN3v27KVL186b"

"N2/69OmlpaUTJO5MwsmcP3/+8OHDn332WWNj4+4+7du5ubmy9cuDD8qQIARku61eFw+PI+fl5e3uLFiysqKt577zOpDSWqyi+HOnny"

"ZGVl5aJFi/Ly8oZ8tGma4XCY+j4AJImTDIZMHf/P//yPTui2o1LxP3v2rGzPqQancwn6AOAJEpTz8vLa29udqJOoMgi45557pLOf"

"6t8VSAwWI8M/LMsqKirKycnRWie2Y661VkrNnDmTyj78hAQAn5CIf+utt2ZlZenUOJ4iSAxB4WSC0SAHwlGo26d/CBgQH3Dg4k"

"HwkAvuLqlCzzvfAZEgAABBQJAAACigQAAAFFAgCAgCIBAEBAkQAAIKBIAAAQUCQAAAgoEgAABBQJAAACigQAAAFFAgCAgCIBAEBA"

"kQAAIKBIAAAQUCQAAAgoEgAABBQJAAACigQAAAFFAgCAgCIBAEBAkQAAIKBIAAAQUCQAAAgoEgAABBQJAAACigQAAAFFAgCAgCIB"

"AEBAkQAAIKBIAAAQUCQAAAgoEgAABBQJAAACigQAAAFFAgCAgCIBAEBAkQAAIKBIAAAQUOFUnwDSjGEYqT6FKzMGJeEjPHsRtNap"
```

"PgWkExIARsQ0TdM0tdaWZaX6XK4sFos5/+sSy7K01tFo1L2PGKVQKGQYhtbatm2SAa6JBIBrCIVC9iD5z3HjxinvDQVM07RtOy8v"

"z72PyMnJyc/PD4VCXsuCEuv7+/t7e3udv5SGIw1gGN76DsNTJMRLBCktLa2srLznnntKSkoKCwtTfWpXID3fjIyMsWPHuvQRfX19"

"vb298kEufcRoXLp06ciRI7t3766trf34448lYUteTPWpAUgroVBI/lBRUVFbWxuNRjXSR2Nj45o1ayKRSHxTAkMwAsAVSJWjqKjo"

"xRdfXL58ufxlLBbz+BSocO/0tCc7/g4n+stMgFLq0KFDa9eu3bVrlwfLVgC8KBwOK6WWLVvW2tqqtbYsKxaLSTUZ6cKyLBm0RaPR"

"tWvXOs0KAFclYWL16tUSPmKxWKpDGW6ck7l/9atfKXIAgGGYpqmUWrhwYU9Pjyb6+4Jt29KOMg5gPgDxPF3MRTJJcb+goGDv3r3T"

"pk2zLItg4Q+SBgzDWLZs2Z49e1gXBAdbQeDPJC7827/927Rp02KxGNHfNyS1m6b5yiuvZGZmKu89w4FUIQFAKaVMO7Qsq6Sk5LHH"

"HrNtm+jvM9K+xcXF0r5S6wO4D6DUYJdww4YNmZmZUi5I9RkhweT5te9///uRSIQSEAQJAMowDMuy8vLy/vZv/1ZrTfffl2Qrp5KS"

"kgULFtDKECQA/GXxz+TJk7XWdP/9SsZ2f/M3f6OYBoBSigQANRgLFi9erJSiOOBj0tDl5eWKhoZSigQANbjDgcQFOoY+JkO90tLS"

"cePGMdMDRQKAGkwABQUFqT4RJENubm5GRkaqzwKeQALAn7n6KhV4h7zWJtVnAU8gAeDPWBYSEKZpUvyBIAHgz6XhgwcPKs/veIzR"

"kMY9depUd3e3Z19rg2QiAeDPPvnkE8UksK/Jyp+mpqZYLMbDwFAkAKjBuLBr167+/n7igo9Jdn/vvfcUmR5KKRIAlFKyOcyJEyca"

"GxuVUrw6ype01qZp9vT01NbWKloZSikSAIRhGLZt/+pXv6Jj6FeWZRmG8cYbb5cYbb5w5cyYUCjEBAMX7ACBkx+BwOHzo0aTP"

"yCsBlFJz5sxpaWlpaWJuZGZScP3/+jjvuOH/+vEz2pPqk4BWUgPAXWmvLskzTfOyxx/7j"

"KBTiIQDE41bAXxiGEQqFYrHYO++889BDDwMIDEQikVSfFEYlGo1mZGScP3/+jjvuOH/+vEz2pPqk4BWUgPAXWmvLskzTfOyxx/7j"

"P/5D3hzCcpE0Zdu2RP/6+vqqqpz584R/QFcg1MiqKmpkRqCZVnRaFT2kIHH2bYdi8VisZj85+uvv56fn5+Gn+fn5+Gn+fn5+Gn+

```
HhRO9QkgDUgv0jAM"

"L68il96uTHi6ZOzYseFwOBwOe/blOTJJ4/25CngECQAjJT3rVJ/FVcnpuXqGssBG8fY0+AUrwwAgoEgAABBQ
JAAACCgSAAAEFAkA"

"AAKKBAAAAUUCAICAIgEAQECRAAAgoEgAABBQJAAACCgSAAAEFAkAAAKKBAAAAUUCAICAIgEAQECRAAAgoEgA
ABBQJAAACCgSAAAE"

"FAkAAAKKBAAAAUUCAICAIgEAQECRAAAgoEgAABBQJAAACCgSAAAEFAkAAAKKBAAAAUUCAICAIgEAQECRAAAg
oEgAABBQJAAACCgS"

"AAAEFAkAAAKKBAAAAUUCAICAIgEAQECRAAAgoEgAABBQJAAACCgSAAAEFAkAAAKKBABf0Vqn6cGB5CMBwFfC
4XCaHhxIPhIAfEK6"

"5xcuXIhGo4ZhJPbgcsC2trbEHhZILRIAfEISwPHjx3t6egzDSGy5RhJAQ0ODohAEHyEBwCe01oZh9PT0fPbZ
ZyqhYdo58qlTpxJ7"

"ZCC1SADwj1AoFI1G33//faWUbduJOqwc6tChQy0tLaZpJvDIAIDEME1TKTVnzpxoNGpZlk6QWCymta6urlbM
AwOAZ0kO2LFjhxO4"

"R8m2bcuy2tvbJ0yYYBhGwqeXAQCJEQqFlFL33Xef1joajY4+AchBnn76aefgAACPkjC9ceNGrfXAwMBoor+M
Ifbs2ROJREzTpPsP"

"AJ5mGEYoFAqFQh999NFoxgES/Ts6OqZPn64Gi0sAAE+TYn1BQcHevXtvbBwgaaOjo6O8vFwR/QEgjUjIzs/P
37lzp9basqwRzglb"

"liXRv7W1VaI/pX8ASDOSAzIyMmQ+wOnaW5Zl23Z8OJelPtFo1Pn77du3FxUVKaI/AKQpZ9r2wQcfrK2tjR8E
SNAX8cmgsbFxzZo1"

"8q+I/gCQxmROWP48b968Z599du/evR0dHfFBv7Oz89ChQ5s3b66srJQfZsk/goBbHIEQCoWkvCP/edNNNxUX
F48dO1YpZVnW4cOH"

"Ozs7Lctyftj5MwDAD0zTDIfDV+vay+JROv4IDu51BJFhGPErO2V/N802nwAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAA"

"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAeML/"
    "A0RAd9O9TquiAAAAAElFTkSuQmCC"
)

@app.route("/icon-maskable-512.png")
def pwa_icon_mask():
    r = _PwaResp(_PWA_ICON_MASK, mimetype="image/png")
    r.headers["Cache-Control"] = "public, max-age=86400"
    return r

@app.route("/manifest.json")
```

```
def pwa_manifest():
    r = _PwaResp(_PWA_MANIFEST_JSON, mimetype="application/manifest+json")
    r.headers["Cache-Control"] = "no-cache"
    return r

@app.route("/icon-transparent-192.png")
def pwa_icon_192():
    r = _PwaResp(_PWA_ICON_192, mimetype="image/png")
    r.headers["Cache-Control"] = "public, max-age=86400"
    return r

@app.route("/icon-transparent-512.png")
def pwa_icon_512():
    r = _PwaResp(_PWA_ICON_512, mimetype="image/png")
    r.headers["Cache-Control"] = "public, max-age=86400"
    return r

@app.route("/sw.js")
def pwa_service_worker():
    resp = send_file(os.path.join(_PWA_BASE, "sw.js"),
mimetype="application/javascript")
    resp.headers["Service-Worker-Allowed"] = "/"
    resp.headers["Cache-Control"] = "no-cache"
    return resp

# Play Store app (TWA) verification file
@app.route("/.well-known/assetlinks.json")
def pwa_assetlinks():
    return send_file(os.path.join(_PWA_BASE, "assetlinks.json"),
mimetype="application/json")
CORS(app)

app.secret_key = "7f3a9c1e8b2d4f6a0c5e9b7d3f1a8c6e4b2d9f7a1c3e5b8d0f2a4c6e8b1d3f5a"

# ── GUNICORN KEEPALIVE (no gunicorn.conf.py / env var needed) ──────────────
# gunicorn kills a worker that shows no heartbeat for 30s (default timeout), which
# cuts long AI / sandbox requests. This background thread sends the heartbeat for
# THIS worker only, every 5s, so long requests are never killed. It does nothing
# when the app runs without gunicorn (local / flask run).
_keepalive_pid = None


def _start_gunicorn_keepalive():
    global _keepalive_pid
    if _keepalive_pid == os.getpid():
        return
    _keepalive_pid = os.getpid()
    try:
        import gc
        import threading
```

```
        from gunicorn.workers.base import Worker
        me = os.getpid()
        workers = [o for o in gc.get_objects()
                   if isinstance(o, Worker) and getattr(o, "pid", None) == me]
    except Exception:
        return
    if not workers:
        return

    def _beat():
        while True:
            for w in workers:
                try:
                    w.notify()
                except Exception:
                    pass
            time.sleep(5)

    threading.Thread(target=_beat, daemon=True, name="gunicorn-keepalive").start()


@app.before_request
def _ensure_gunicorn_keepalive():
    try:
        _start_gunicorn_keepalive()
    except Exception:
        pass

import firebase_admin
from firebase_admin import credentials, firestore

import json

firebase_creds_json = os.environ.get("FIREBASE_SERVICE_ACCOUNT")
if not firebase_creds_json:
    raise ValueError("FIREBASE_SERVICE_ACCOUNT environment variable not set")

firebase_creds = json.loads(firebase_creds_json)
cred = credentials.Certificate(firebase_creds)
firebase_admin.initialize_app(cred)
db = firestore.client()

ADMIN_PASSWORD = "meesam7861A."

# ── GEMINI CONFIG (Gemini 3.8 Flash, OpenAI-compatible endpoint) ──────────────

# Environment variables (set these in Vercel > Settings > Environment Variables):
#   GEMINI_API_KEY          (required)  - your Gemini API key (Google AI Studio)
#   GEMINI_API_KEY_2..6     (optional)  - backup keys, used in order when the
previous one hits its limit
```

```
#   GEMINI_KEY_COOLDOWN     (optional)  - seconds a limited key rests (default: 60)
#   GEMINI_MODEL            (optional)  - default: gemini-3.5-flash-lite
#   GEMINI_MAX_OUTPUT       (optional)  - default: 32000 (model supports up to 65536)
#   GEMINI_REASONING_EFFORT (optional)  - low | medium | high (default: medium; empty
= off)
# Gemini Flash models accept text AND images, so one model handles chat, images and
builds.
# Gemini answers HTTP 429 when a rate limit is hit (the request is retried once
automatically).
import requests

# Multi-key rotation: GEMINI_API_KEY (key 1), GEMINI_API_KEY_2 ... GEMINI_API_KEY_6.
# Keys are used in number order. When a key hits its limit (429) / is rejected
(401/403),
# the very next key is used immediately; the exhausted key rests for a cooldown
period.
GEMINI_API_KEYS = []
for _name in ("GEMINI_API_KEY", "GEMINI_API_KEY_2", "GEMINI_API_KEY_3",
              "GEMINI_API_KEY_4", "GEMINI_API_KEY_5", "GEMINI_API_KEY_6"):
    _k = (os.environ.get(_name) or "").strip()
    if _k and _k not in GEMINI_API_KEYS:
        GEMINI_API_KEYS.append(_k)
if not GEMINI_API_KEYS:
    raise ValueError("GEMINI_API_KEY environment variable not set")
GEMINI_API_KEY = GEMINI_API_KEYS[0]  # kept for backward compatibility
GEMINI_KEY_COOLDOWN = int(os.environ.get("GEMINI_KEY_COOLDOWN", "60"))  # seconds a
limited key rests
_gemini_key_cooldown = {}  # key index -> unix time until which it is skipped

GEMINI_MODEL = os.environ.get("GEMINI_MODEL", "gemini-3.5-flash-lite").strip() or
"gemini-3.5-flash-lite"
GEMINI_MAX_OUTPUT = int(os.environ.get("GEMINI_MAX_OUTPUT", "32000"))
GEMINI_REASONING_EFFORT = os.environ.get("GEMINI_REASONING_EFFORT",
"medium").strip().lower()
if GEMINI_REASONING_EFFORT not in ("low", "medium", "high"):
    GEMINI_REASONING_EFFORT = ""
GEMINI_URL =
"https://generativelanguage.googleapis.com/v1beta/openai/chat/completions"


def _extract_reasoning(message):
    """Pulls the model's thinking text out of a message, whatever shape it comes
in."""
    for key in ("reasoning", "reasoning_content"):
        val = message.get(key)
        if isinstance(val, str) and val.strip():
            return val.strip()
    parts = []
    for d in message.get("reasoning_details") or []:
        if not isinstance(d, dict):
```

```
            continue
        t = d.get("text") or d.get("summary") or ""
        if isinstance(t, str) and t.strip():
            parts.append(t.strip())
    return "\n\n".join(parts).strip()


def _extract_sources(message):
    """Collects web-search sources (url_citation annotations) from a message, if
any."""
    sources = []
    for ann in message.get("annotations") or []:
        if not isinstance(ann, dict):
            continue
        info = ann.get("url_citation") or ann
        if isinstance(info, dict):
            uri = info.get("url")
            if uri:
                sources.append({"title": info.get("title") or uri, "uri": uri})
    return sources


def _gemini_post(payload, timeout):
    """POSTs to Gemini with automatic key rotation.
    Keys are tried in number order (1 -> 2 -> 3 -> 4 -> 5). If a key hits its limit
(429)
    or is rejected (401/403), the next key is used immediately and the bad key is put
on
    cooldown. 5xx errors are retried once on the same key."""
    now = time.time()
    n = len(GEMINI_API_KEYS)
    order = [i for i in range(n) if _gemini_key_cooldown.get(i, 0) <= now]
    # keys still cooling down go last (earliest-available first) so we never get
stuck
    order += sorted((i for i in range(n) if i not in order),
                    key=lambda i: _gemini_key_cooldown.get(i, 0))

    resp = None
    for pos, idx in enumerate(order):
        headers = {
            "Authorization": f"Bearer {GEMINI_API_KEYS[idx]}",
            "Content-Type": "application/json",
        }
        resp = requests.post(GEMINI_URL, headers=headers, json=payload,
timeout=timeout)

        if resp.status_code in (500, 502, 503):
            time.sleep(2.0)
            resp = requests.post(GEMINI_URL, headers=headers, json=payload,
timeout=timeout)
```

```
        if resp.status_code in (429, 401, 403):
            try:
                wait = float(resp.headers.get("retry-after", GEMINI_KEY_COOLDOWN))
            except ValueError:
                wait = float(GEMINI_KEY_COOLDOWN)
            _gemini_key_cooldown[idx] = time.time() + max(wait, 1.0)
            print(f"[Gemini] key #{idx + 1} limited/rejected (HTTP
{resp.status_code}) -> "
                  f"switching to next key", flush=True)
            continue  # try the next key right away

        return resp  # success or a non-key-related error (400, etc.)

    return resp  # every key was limited; hand back the last response


def _gemini_message(resp):
    if resp.status_code != 200:
        # printed so the real reason is visible in Vercel > Logs
        print(f"[Gemini ERROR] status={resp.status_code} body={resp.text[:600]}",
flush=True)
        raise Exception(f"Gemini {resp.status_code}: {resp.text[:500]}")
    data = resp.json()
    if isinstance(data, dict) and data.get("error"):
        raise Exception(f"Gemini error: {data['error']}")
    return data["choices"][0]["message"]


import re

_THOUGHT_RE = re.compile(r"<thought>([\s\S]*?)</thought>", re.IGNORECASE)


def _split_thoughts(text):
    """Gemini sends thought summaries inside <thought>...</thought> tags in the
content.
    Returns (answer_text, thinking_text) so the answer is clean and the thinking is
shown."""
    if not text or "<thought>" not in text.lower():
        return text or "", ""
    thoughts = [m.strip() for m in _THOUGHT_RE.findall(text) if m.strip()]
    answer = _THOUGHT_RE.sub("", text)
    low = answer.lower()
    if "<thought>" in low:
        # thinking cut off before its closing tag: everything after the tag is
thinking
        idx = low.index("<thought>")
        leftover = answer[idx + len("<thought>"):].strip()
        if leftover:
```

```
            thoughts.append(leftover)
        answer = answer[:idx]
    return answer.strip(), "\n\n".join(thoughts).strip()


def _gemini_chat(full_messages, temperature, max_tokens, timeout,
use_reasoning=True):
    payload = {
        "model": GEMINI_MODEL,
        "messages": full_messages,
        "temperature": temperature,
        "max_tokens": min(max_tokens, GEMINI_MAX_OUTPUT),
    }
    if use_reasoning:
        # ask Gemini to return its thinking summary alongside the answer
        payload["extra_body"] = {"google": {"thinking_config": {"include_thoughts":
True}}}
        if GEMINI_REASONING_EFFORT:
            payload["reasoning_effort"] = GEMINI_REASONING_EFFORT

    resp = _gemini_post(payload, timeout)
    if resp.status_code == 400 and ("reasoning_effort" in payload or "extra_body" in
payload):
        # model/endpoint rejected a thinking option -> retry without thinking options
        payload.pop("reasoning_effort", None)
        payload.pop("extra_body", None)
        resp = _gemini_post(payload, timeout)

    message = _gemini_message(resp)
    raw_content = message.get("content") or ""
    thinking = ""
    if isinstance(raw_content, list):
        # content sent as parts: {"type": "text", "text": ...} and thought parts
marked "thought": true
        text_parts, thought_parts = [], []
        for part in raw_content:
            if isinstance(part, str):
                text_parts.append(part)
            elif isinstance(part, dict):
                t = part.get("text") or ""
                if not isinstance(t, str):
                    t = ""
                if part.get("thought") is True or part.get("type") == "thought":
                    if t.strip():
                        thought_parts.append(t.strip())
                elif t:
                    text_parts.append(t)
        raw_content = "".join(text_parts)
        thinking = "\n\n".join(thought_parts).strip()
    if not isinstance(raw_content, str):
```

```
        raw_content = str(raw_content)
    text, tag_thinking = _split_thoughts(raw_content)
    if tag_thinking:
        thinking = (thinking + "\n\n" + tag_thinking).strip() if thinking else
tag_thinking
    if not thinking:
        thinking = _extract_reasoning(message)
    if not thinking:
        # diagnostic: shows in Vercel > Logs what Gemini actually returned
        print(f"[Gemini THINK] no thinking found. model={GEMINI_MODEL} effort=
{GEMINI_REASONING_EFFORT!r} "
              f"keys={list(message.keys())} content_head={raw_content[:200]!r}",
flush=True)
    if not text.strip():
        raise Exception("Empty response from model")
    return text, thinking, _extract_sources(message)


# Used for normal (non-coding) chat answers: the model writes its reasoning in
<thought> tags,
# which _split_thoughts() removes from the answer and returns as the thought process.
VISIBLE_REASONING_INSTRUCTION = (
    "\n\nBEFORE your final answer, write your complete step-by-step thinking inside
<thought> and </thought> tags, "
    "from start to finish: what the user really needs, each step you take, the
options you considered, "
    "every check you make, and any mistake you notice and fix. Do not skip steps.
After the closing </thought> tag, "
    "write only the final answer for the user, with no tags around it."
)


_reason_tl = threading.local()


def _reason_log():
    if not hasattr(_reason_tl, "log"):
        _reason_tl.log = []
    return _reason_tl.log


def _merge_reasoning(first, extra_list):
    """Combines the first reasoning with reasoning from Daytona verify/fix/regenerate
calls,
    so the thinking panel shows the whole trail, not only the first draft."""
    parts = [p for p in [first] + list(extra_list or []) if p and p.strip()]
    out = []
    for p in parts:
        if not out or out[-1] != p:
            out.append(p)
```

```
    if len(out) <= 1:
        return out[0] if out else ""
    return "\n\n---- Daytona verification ke baad update ----\n\n".join(out)


def llm_chat(messages, system=None, temperature=0.7, max_tokens=4096,
web_search=False, timeout=300):
    """Returns (text, reasoning, sources). sources = [{"title": ..., "uri": ...}]
    web_search is kept for compatibility with existing callers; this endpoint has no
    search tool configured, so it is not used."""
    full_messages = []
    # Every feature (review, modernize, hunt, fix, security, build, agent, chat...)
goes through
    # here, so the thought-process instruction is added in one place.
_split_thoughts() strips the
    # <thought> block from the answer, so code/JSON output stays clean.
    system = (system or "")
    if "<thought>" not in system:
        system += VISIBLE_REASONING_INSTRUCTION
    full_messages.append({"role": "system", "content": system})
    full_messages.extend(messages)
    text, thinking, sources = _gemini_chat(full_messages, temperature, max_tokens,
timeout)
    if thinking:
        log = _reason_log()
        log.append(thinking)
        del log[:-20]  # keep memory bounded on long-running worker threads
    return text, thinking, sources


def build_user_messages(prompt_text, images=None, image_note=None):
    """Builds the final user message for llm_chat. If one or more images are attached
    (vision), they are sent alongside the text so the model actually sees and
analyzes
    them — used by Everything AI, Build Web, Build App and the Full Stack Builder.
    `images` is a list of base64 strings (up to 4, matching the frontend's limit)."""
    images = [img for img in (images or []) if img]
    if images:
        text = f"{prompt_text}\n\n{image_note}" if image_note else prompt_text
        content = [
            {"type": "image_url", "image_url": {"url": f"
{img}"}}
            for img in images
        ]
        content.append({"type": "text", "text": text})
        return [{"role": "user", "content": content}]
    return [{"role": "user", "content": prompt_text}]


import re
```

```
def is_coding_request_check(text, keywords):
    text_lower = text.lower()
    return any(re.search(rf'\b{re.escape(kw)}\b', text_lower) for kw in keywords)

# ==== DAYTONA BEGIN ================================================
# ── DAYTONA SANDBOX VERIFICATION ──────────────────────────────────────
# AI-generated code is uploaded to an isolated Daytona sandbox and checked
# (syntax, JS in HTML, requirements install, Flask import + GET smoke test).
# If errors are found, the AI fixes them and the check re-runs, up to
# DAYTONA_FIX_ROUNDS times. Only then is the answer delivered.
# Everything is configured through environment variables:
#   DAYTONA_API_KEY      (required to enable)  - from app.daytona.io dashboard
#   DAYTONA_API_URL      (optional)            - read by the SDK itself
#   DAYTONA_TARGET       (optional)            - read by the SDK itself
#   DAYTONA_ENABLED      (optional, "0" turns checking off)
#   DAYTONA_FIX_ROUNDS   (optional, Full Stack Builder, default 3)
#   DAYTONA_FIX_ROUNDS_CHAT (optional, other features, default 2)
# ALL LANGUAGES: python/js/ts/html/css/json/yaml/xml/sql/bash run in the default
sandbox; java, kotlin,
# scala, c, c++, c#, go, rust, swift, php, ruby, perl, lua, r, dart, haskell, elixir,
erlang, clojure,
# julia, groovy, fortran, cobol, pascal, nasm, ocaml, d, ada, tcl, nim, crystal,
octave, solidity,
# racket, powershell are compiled+run in a sandbox built from that language's
official image.
# PERSISTENT: patch-fix -> regenerate whole answer -> verify, repeated until it
really passes.
#   DAYTONA_FINAL_FALLBACK=best (default): if every attempt failed, deliver the best
attempt silently
#   DAYTONA_FINAL_FALLBACK=block: deliver nothing + notice instead
# STRICT MODE (default ON): code is delivered ONLY after the sandbox confirms it
passes.
# Retries/regeneration/fixes happen SILENTLY. No notice is ever shown; the best
attempt is always delivered.
#   DAYTONA_STRICT=0     turns strict mode off (old behaviour: deliver unverified)
#   DAYTONA_ALL_FEATURES=0  limits checking to DAYTONA_CHAT_FEATURES only
# so the existing app never breaks because of the sandbox.
DAYTONA_API_KEY = os.environ.get("DAYTONA_API_KEY")
DAYTONA_ENABLED = bool(DAYTONA_API_KEY) and os.environ.get("DAYTONA_ENABLED", "1") !=
"0"
DAYTONA_FIX_ROUNDS = int(os.environ.get("DAYTONA_FIX_ROUNDS", "5"))
DAYTONA_FIX_ROUNDS_CHAT = int(os.environ.get("DAYTONA_FIX_ROUNDS_CHAT", "4"))
DAYTONA_STRICT = os.environ.get("DAYTONA_STRICT", "1") != "0"
DAYTONA_ALL_FEATURES = os.environ.get("DAYTONA_ALL_FEATURES", "1") != "0"
# PERSISTENT MODE: if patching does not pass, the whole answer is REGENERATED and re-
verified.
DAYTONA_REGEN_ATTEMPTS = int(os.environ.get("DAYTONA_REGEN_ATTEMPTS", "8"))
DAYTONA_PERSIST_BUDGET = int(os.environ.get("DAYTONA_PERSIST_BUDGET", "1500"))
# when every attempt failed: "best" = still deliver the best attempt silently (user
```

```
never empty-handed),
#                             "block" = deliver nothing + notice
# User never sees a "try again"/blocked notice: retries happen silently, best attempt
is always delivered.
DAYTONA_FINAL_FALLBACK = "best"
# Time limits (seconds) so Daytona can never make a request hang or fail:
#   DAYTONA_TOTAL_BUDGET  - max total time Daytona + AI-fix rounds may use in ONE
request
#   DAYTONA_CHECK_TIMEOUT - max time for a single sandbox check
DAYTONA_TOTAL_BUDGET = int(os.environ.get("DAYTONA_TOTAL_BUDGET", "720"))
DAYTONA_CHECK_TIMEOUT = int(os.environ.get("DAYTONA_CHECK_TIMEOUT", "300"))

_daytona_client = None


def get_daytona():
    """Lazy client. The SDK reads DAYTONA_API_KEY / DAYTONA_API_URL / DAYTONA_TARGET
    from the environment, so no key is hard-coded anywhere."""
    global _daytona_client
    if _daytona_client is None:
        from daytona import Daytona  # imported lazily so the app still boots without
the package
        _daytona_client = Daytona()
    return _daytona_client


# Runs INSIDE the sandbox. Prints one line: @@RESULT@@{json}
_DAYTONA_CHECKER = r'''
import sys, os, json, subprocess, shutil, re, tempfile, textwrap

root = os.path.abspath(sys.argv[1])
smoke_path = os.path.abspath(sys.argv[2])
errors, warnings, notes = [], [], []
NET_SIGNS = ("ConnectionError", "Max retries exceeded", "Temporary failure in name
resolution",
             "ReadTimeout", "NewConnectionError", "Network is unreachable", "timed
out", "ProxyError")


def run(cmd, timeout=60, cwd=None):
    try:
        p = subprocess.run(cmd, capture_output=True, text=True, timeout=timeout,
cwd=cwd)
        return p.returncode, (p.stdout or "") + (p.stderr or "")
    except subprocess.TimeoutExpired:
        return 124, "timeout"
    except Exception as e:
        return 1, str(e)
```

```
files = []
for dp, dn, fn in os.walk(root):
    dn[:] = [d for d in dn if d not in ("node_modules", "__pycache__", ".git",
"venv", ".venv")]
    for f in fn:
        files.append(os.path.join(dp, f))

has_node = shutil.which("node") is not None
SCRIPT_RE = re.compile(r"<script([^>]*)>([\s\S]*?)</script>", re.I)
tmpdir = tempfile.mkdtemp()
counter = 0
import time as _time
_run_spent = [0.0]
_PIP_ALIAS = {"cv2": "opencv-python", "PIL": "pillow", "sklearn": "scikit-learn",
"yaml": "pyyaml",
              "bs4": "beautifulsoup4", "dotenv": "python-dotenv", "jwt": "pyjwt",
"dateutil": "python-dateutil"}


def run_py_snippet(code, rel):
    # Actually EXECUTES the python snippet (not just a syntax check) and reports
runtime errors.
    if _run_spent[0] > 85:
        warnings.append(rel + ": runtime check skipped (sandbox time budget used
up)")
        return
    d = tempfile.mkdtemp()
    p = os.path.join(d, "run_snippet.py")
    with open(p, "w", encoding="utf-8") as fh:
        fh.write(code)
    for attempt in range(3):
        t0 = _time.time()
        try:
            pr = subprocess.run([sys.executable, p], capture_output=True, text=True,
timeout=15,
                                cwd=d, stdin=subprocess.DEVNULL)
        except subprocess.TimeoutExpired:
            _run_spent[0] += _time.time() - t0
            warnings.append(rel + ": still running after 15s (server or loop) - ran
without crashing so far")
            return
        _run_spent[0] += _time.time() - t0
        out = ((pr.stdout or "") + (pr.stderr or "")).strip()
        if pr.returncode == 0:
            return
        m = re.search(r"ModuleNotFoundError: No module named '([A-Za-z0-9_\-]+)",
out)
        if m and attempt < 2:
            mod = m.group(1)
            rc2, out2 = run([sys.executable, "-m", "pip", "install", "-q",
```

```
_PIP_ALIAS.get(mod, mod)], timeout=60)
            if rc2 == 0:
                continue
            warnings.append(rel + ": module " + mod + " could not be installed in the
sandbox - runtime not fully checked")
            return
        if "EOFError" in out:
            warnings.append(rel + ": waits for keyboard input() - ran fine until the
input prompt")
            return
        if any(x in out for x in NET_SIGNS):
            warnings.append(rel + ": needs internet, could not run fully: " +
out[-200:])
            return
        if any(x in out for x in ("TclError", "no display", "pygame.error", "No
module named 'tkinter'")):
            warnings.append(rel + ": needs a screen/GUI, runtime not fully checked")
            return
        if "Traceback" in out:
            errors.append(rel + ": runtime error when the code was executed: " +
out[-700:])
        else:
            warnings.append(rel + ": exited with code " + str(pr.returncode) + "
without a traceback")
        return

for path in sorted(files):
    rel = os.path.relpath(path, root)
    ext = os.path.splitext(rel)[1].lower()
    if ext not in (".py", ".json", ".html", ".htm", ".js", ".mjs", ".jsx", ".ts",
".tsx", ".css", ".yaml", ".yml", ".xml", ".sh"):
        continue
    try:
        with open(path, encoding="utf-8") as fh:
            src = fh.read()
    except Exception as e:
        errors.append(rel + ": cannot read file (" + str(e) + ")")
        continue
    if not src.strip():
        errors.append(rel + ": file is empty")
        continue

    if ext == ".py":
        code = textwrap.dedent(src) if os.path.basename(rel).startswith("snippet_")
else src
        try:
            compile(code, rel, "exec")
        except SyntaxError as e:
            errors.append(rel + ": SyntaxError line " + str(e.lineno) + ": " +
str(e.msg))
```

```
        else:
            if os.path.basename(rel).startswith("snippet_"):
                run_py_snippet(code, rel)
    elif ext == ".json":
        if re.search(r"(?m)^\s*//|/\*|\.\.\.", src):
            notes.append(rel + ": JSON with comments/ellipsis - not parsed")
        else:
            try:
                json.loads(src)
            except Exception as e:
                errors.append(rel + ": invalid JSON: " + str(e))
    elif ext in (".html", ".htm"):
        low = src.lower()
        if "<html" in low and "</html>" not in low:
            errors.append(rel + ": HTML looks truncated (missing </html>)")
        for m in SCRIPT_RE.finditer(src):
            attrs, body = m.group(1), m.group(2)
            if not body.strip():
                continue
            tm = re.search(r"type\s*=\s*[\"']?([^\"'\s>]+)", attrs, re.I)
            stype = tm.group(1).lower() if tm else ""
            if stype not in ("", "text/javascript", "application/javascript",
"module"):
                continue
            if not has_node:
                notes.append("node not available: inline JavaScript not syntax-
checked")
                break
            counter += 1
            tmp = os.path.join(tmpdir, "chk_%d.%s" % (counter, "mjs" if stype ==
"module" else "js"))
            with open(tmp, "w", encoding="utf-8") as fh:
                fh.write(body)
            rc, out = run(["node", "--check", tmp])
            if rc != 0:
                errors.append(rel + ": JavaScript error in <script> #" + str(counter)
+ ": " + out.strip()[:500])
    elif ext in (".js", ".mjs"):
        if not has_node:
            notes.append("node not available: " + rel + " not syntax-checked")
            continue
        rc, out = run(["node", "--check", path])
        if rc != 0:
            errors.append(rel + ": JavaScript error: " + out.strip()[:500])
    elif ext == ".jsx":
        if shutil.which("npx"):
            rc, out = run(["npx", "--yes", "esbuild", path, "--loader:.jsx=jsx", "--
log-level=error"], timeout=90)
            if rc != 0 and "[ERROR]" in out:
                errors.append(rel + ": JSX error: " + out.strip()[:500])
```

```
            elif rc != 0:
                notes.append(rel + ": JSX checker unavailable")
    elif ext in (".ts", ".tsx", ".css"):
        if shutil.which("npx"):
            loader = {".ts": "ts", ".tsx": "tsx", ".css": "css"}[ext]
            rc, out = run(["npx", "--yes", "esbuild", path, "--loader:" + ext + "=" +
loader, "--log-level=error"], timeout=90)
            if rc != 0 and "[ERROR]" in out:
                errors.append(rel + ": " + loader.upper() + " error: " + out.strip()
[:500])
            elif rc != 0:
                notes.append(rel + ": " + loader + " checker unavailable")
    elif ext in (".yaml", ".yml"):
        if "{{" in src or "<%" in src or re.search(r"<[A-Za-z_ -]+>", src):
            notes.append(rel + ": template/placeholder YAML - not parsed")
        else:
            rc0, _o = run([sys.executable, "-c", "import yaml"])
            if rc0 != 0:
                run([sys.executable, "-m", "pip", "install", "-q", "pyyaml"],
timeout=60)
            code_y = "import sys,yaml\ntry:\n
list(yaml.safe_load_all(open(sys.argv[1], encoding='utf-8')))\nexcept Exception as
e:\n    print('YAMLERR ' + str(e)[:300]); sys.exit(1)\n"
            rc, out = run([sys.executable, "-c", code_y, path])
            if rc != 0 and "YAMLERR" in out:
                errors.append(rel + ": invalid YAML: " + out.strip()[:400])
            elif rc != 0:
                notes.append(rel + ": YAML checker unavailable")
    elif ext == ".xml":
        import xml.parsers.expat
        body = re.sub(r"^\s*<\?xml[^>]*\?>", "", src)
        try:
            xml.parsers.expat.ParserCreate().Parse("<r>" + body + "</r>", True)
        except Exception as e:
            errors.append(rel + ": invalid XML: " + str(e)[:300])
    elif ext == ".sh":
        if re.search(r"<[A-Za-z_ -]+>", src):
            notes.append(rel + ": placeholders like <name> - not syntax-checked")
        else:
            rc, out = run(["bash", "-n", path])
            if rc != 0:
                errors.append(rel + ": shell syntax error: " + out.strip()[:400])

# SQL is only checked softly (dialects differ), never blocks.
for path in files:
    if path.lower().endswith(".sql"):
        import sqlite3
        try:
            with open(path, encoding="utf-8") as fh:
                sqlite3.connect(":memory:").executescript(fh.read())
```

```
        except Exception as e:
            warnings.append(os.path.relpath(path, root) + ": sqlite could not run
this SQL (" + str(e)[:200] + ") - may be a dialect difference")

# Backend: install requirements, import app.py, smoke-test GET routes.
appfile = os.path.join(root, "app.py")
req = os.path.join(root, "requirements.txt")
if os.path.exists(appfile) and not any(e.startswith("app.py") for e in errors):
    deps_ok = True
    if os.path.exists(req):
        rc, out = run([sys.executable, "-m", "pip", "install", "-q", "-r", req],
timeout=70)
        if rc != 0:
            deps_ok = False
            if "No module named pip" in out or rc == 124 or any(s in out for s in
NET_SIGNS):
                warnings.append("requirements.txt could not be installed in the
sandbox (pip/network): " + out.strip()[-200:])
            else:
                errors.append("requirements.txt: pip install failed: " + out.strip()
[-500:])
    if deps_ok:
        rc, out = run([sys.executable, smoke_path, root], timeout=60, cwd=root)
        line = None
        for l in reversed(out.splitlines()):
            if l.startswith("@@SMOKE@@"):
                line = l
                break
        if line:
            try:
                data = json.loads(line[len("@@SMOKE@@"):])
                errors.extend(data.get("errors", []))
                warnings.extend(data.get("warnings", []))
            except Exception:
                warnings.append("backend smoke test output unreadable")
        elif rc == 124:
            warnings.append("backend smoke test timed out (app may start a server or
block at import)")
        else:
            errors.append("backend smoke test crashed: " + out.strip()[-400:])

print("@@RESULT@@" + json.dumps({"errors": errors, "warnings": warnings, "notes":
notes}))
'''

# Runs INSIDE the sandbox: imports app.py and GETs every parameterless route.
_DAYTONA_SMOKE = r'''
import sys, os, json, traceback, importlib.util

root = sys.argv[1]
```

```
sys.path.insert(0, root)
os.chdir(root)
out = {"errors": [], "warnings": []}
mod = None
try:
    spec = importlib.util.spec_from_file_location("user_app", os.path.join(root,
"app.py"))
    mod = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(mod)
except SystemExit:
    pass
except BaseException as e:
    out["errors"].append("app.py failed to import: " +
"".join(traceback.format_exception_only(type(e), e)).strip()[:600])
    print("@@SMOKE@@" + json.dumps(out))
    sys.exit(0)

flask_app = getattr(mod, "app", None) if mod else None
if flask_app is not None and hasattr(flask_app, "test_client"):
    client = flask_app.test_client()
    for rule in flask_app.url_map.iter_rules():
        if "GET" in rule.methods and not rule.arguments and rule.endpoint !=
"static":
            try:
                r = client.get(rule.rule)
                if r.status_code >= 500:
                    out["errors"].append("GET " + rule.rule + " returned HTTP " +
str(r.status_code))
            except Exception as e:
                out["errors"].append("GET " + rule.rule + " raised " +
type(e).__name__ + ": " + str(e)[:200])
print("@@SMOKE@@" + json.dumps(out))
'''


def _safe_rel_path(name):
    name = (name or "").replace("\\", "/")
    parts = [p for p in name.split("/") if p not in ("", ".", "..")]
    return "/".join(parts)


def _daytona_verify_default(files):
    """files = [{"name", "content"}]. Returns
    {"ran": bool, "errors": [...], "warnings": [...], "notes": [...], "reason": str}.
    ran=False means Daytona could not be used (not an error in the code)."""
    base = {"ran": False, "errors": [], "warnings": [], "notes": [], "reason": ""}
    if not DAYTONA_ENABLED:
        base["reason"] = "DAYTONA_API_KEY not set"
        return base
    sandbox = None
```

```
    try:
        import shlex
        daytona = get_daytona()
        sandbox = daytona.create()

        prepared, dirs = [], {"proj"}
        for f in files:
            rel = _safe_rel_path(f.get("name"))
            if not rel:
                continue
            prepared.append((rel, f.get("content") or ""))
            d = os.path.dirname(rel)
            if d:
                dirs.add("proj/" + d)
        sandbox.process.exec("mkdir -p " + " ".join(shlex.quote(d) for d in
sorted(dirs)))
        for rel, content in prepared:
            sandbox.fs.upload_file(content.encode("utf-8"), "proj/" + rel)
        sandbox.fs.upload_file(_DAYTONA_CHECKER.encode("utf-8"), "_check.py")
        sandbox.fs.upload_file(_DAYTONA_SMOKE.encode("utf-8"), "_smoke.py")

        resp = sandbox.process.exec("python3 _check.py proj _smoke.py",
timeout=max(30, DAYTONA_CHECK_TIMEOUT - 20))
        out = (getattr(resp, "result", "") or "")
        line = None
        for l in reversed(out.splitlines()):
            if l.startswith("@@RESULT@@"):
                line = l
                break
        if not line:
            base["reason"] = "checker produced no result: " + out[-300:]
            return base
        data = json.loads(line[len("@@RESULT@@"):])
        base.update({
            "ran": True,
            "errors": data.get("errors", []),
            "warnings": data.get("warnings", []),
            "notes": data.get("notes", []),
        })
        return base
    except Exception as e:
        base["reason"] = "Daytona error: " + str(e)[:300]
        return base
    finally:
        if sandbox is not None:
            try:
                sandbox.delete()
            except Exception:
                try:
                    get_daytona().delete(sandbox)
```

```
                except Exception:
                    pass


# ── ALL-LANGUAGES VERIFICATION (polyglot) ──────────────────────────────────
# Every code fence is mapped to a language.
Python/JS/TS/HTML/CSS/JSON/YAML/XML/SQL/Bash are
# checked by the checker inside the default sandbox. Every other language runs in a
Daytona
# sandbox created from that language's official Docker image (compile + run).
# Languages with no verifier registered are passed through (verified stays None).
DAYTONA_IMAGE_TIMEOUT = int(os.environ.get("DAYTONA_IMAGE_TIMEOUT", "600"))
DAYTONA_POLY_FIRST_WAIT = int(os.environ.get("DAYTONA_POLY_FIRST_WAIT", "200"))

_LANG_ALIASES = {
    "py": "python", "python": "python", "python3": "python", "py3": "python",
    "js": "javascript", "javascript": "javascript", "node": "javascript", "nodejs":
"javascript",
    "mjs": "javascript", "cjs": "javascript",
    "ts": "typescript", "typescript": "typescript", "tsx": "tsx", "jsx": "jsx",
    "html": "html", "htm": "html", "css": "css", "json": "json",
    "yaml": "yaml", "yml": "yaml", "xml": "xml", "svg": "xml", "sql": "sql",
    "sh": "bash", "bash": "bash", "shell": "bash", "zsh": "bash",
    "powershell": "powershell", "ps1": "powershell", "pwsh": "powershell",
    "java": "java", "kotlin": "kotlin", "kt": "kotlin", "scala": "scala",
    "c": "c", "cpp": "cpp", "c++": "cpp", "cxx": "cpp", "cc": "cpp", "hpp": "cpp",
    "cs": "csharp", "csharp": "csharp", "c#": "csharp",
    "go": "go", "golang": "go", "rust": "rust", "rs": "rust", "swift": "swift",
    "php": "php", "ruby": "ruby", "rb": "ruby", "perl": "perl", "pl": "perl",
    "lua": "lua", "r": "r", "dart": "dart", "haskell": "haskell", "hs": "haskell",
    "elixir": "elixir", "ex": "elixir", "exs": "elixir", "erlang": "erlang", "erl":
"erlang",
    "clojure": "clojure", "clj": "clojure", "julia": "julia", "jl": "julia",
"groovy": "groovy",
    "fortran": "fortran", "f90": "fortran", "cobol": "cobol", "cob": "cobol",
    "pascal": "pascal", "pas": "pascal", "asm": "nasm", "nasm": "nasm", "assembly":
"nasm",
    "ocaml": "ocaml", "ml": "ocaml", "d": "dlang", "nim": "nim", "crystal":
"crystal", "cr": "crystal",
    "octave": "octave", "matlab": "octave", "solidity": "solidity", "sol":
"solidity",
    "racket": "racket", "rkt": "racket", "tcl": "tcl", "ada": "ada",
}

# languages checked by _DAYTONA_CHECKER in the default sandbox
_DEFAULT_LANG_EXT = {
    "python": ".py", "javascript": ".js", "typescript": ".ts", "tsx": ".tsx", "jsx":
".jsx",
    "html": ".html", "css": ".css", "json": ".json", "yaml": ".yaml", "xml": ".xml",
    "sql": ".sql", "bash": ".sh",
```

```
}


def _spec(ext, image, file, check=None, run=None, run_if=None, lib_check=None,
link=None, setup=None,
          setup_lib=None, dep=None, dep_code=None, tmo=150):
    return dict(ext=ext, image=image, file=file, check=check, run=run, run_if=run_if,
lib_check=lib_check,
                link=link, setup=setup, setup_lib=setup_lib, dep=dep,
dep_code=dep_code, tmo=tmo)


def _re_has(pattern):
    return lambda code: bool(re.search(pattern, code, re.M))


_APT = ("export DEBIAN_FRONTEND=noninteractive; (apt-get update -qq || sudo apt-get
update -qq) >/dev/null 2>&1; "
        "(apt-get install -y -qq %s || sudo apt-get install -y -qq %s) >/dev/null
2>&1")


def _apt(pkgs):
    return _APT % (pkgs, pkgs)


_GCC = "gcc:13"
_C_DEP = r"fatal error: [^\n]+: No such file or directory|undefined reference
to|cannot find -l"

_LANG_SPECS = {
    "java": _spec(".java", "eclipse-temurin:21-jdk", "{cls}.java",
                  check="javac -Xlint:none -encoding UTF-8 {f}", run="java -cp .
{run}",
                  run_if=_re_has(r"static\s+void\s+main\s*\("),
                  dep=r"package [\w.]+ does not exist|cannot access [\w.]+|is not
visible|module not found"),
    "kotlin": _spec(".kt", "zenika/kotlin", "main.kt",
                    check="kotlinc {f} -include-runtime -d out.jar",
lib_check="kotlinc {f} -d out_lib",
                    run="java -jar out.jar", run_if=_re_has(r"fun\s+main\s*\("),
                    dep=r"[Uu]nresolved reference", dep_code=r"^import\s+
(?!kotlin\.|java\.|javax\.)", tmo=320),
    "scala": _spec(".scala", "virtuslab/scala-cli", "main.scala",
                   check="scala-cli compile {f} --server=false", run="scala-cli run
{f} --server=false",
                   run_if=_re_has(r"def\s+main\s*\(|@main|extends\s+App"),
                   dep=r"is not a member of|[Nn]ot found: ", dep_code=r"^import\s+
(?!scala\.|java\.|javax\.)", tmo=320),
    "c": _spec(".c", _GCC, "main.c",
```

```
                check="gcc -Wall -Wno-unused -c -o out.o {f}", link="gcc -o out out.o
-lm -lpthread",
                run="./out", run_if=_re_has(r"\bmain\s*\("), dep=_C_DEP),
    "cpp": _spec(".cpp", _GCC, "main.cpp",
                  check="g++ -std=c++20 -Wall -Wno-unused -c -o out.o {f}", link="g++
-o out out.o -lm -lpthread",
                  run="./out", run_if=_re_has(r"\bmain\s*\("), dep=_C_DEP),
    "fortran": _spec(".f90", _GCC, "main.f90",
                     check="gfortran -Wall -c -o out.o {f}", link="gfortran -o out
out.o", run="./out",
                     run_if=_re_has(r"(?i)^\s*program\b"), dep=r"Can't open module
file|undefined reference to"),
    "csharp": _spec(".cs", "mcr.microsoft.com/dotnet/sdk:8.0", "Program.cs",
                    setup="dotnet new console -o proj --force >/dev/null 2>&1 && cp
{f} proj/Program.cs",
                    setup_lib="dotnet new console -o proj --force >/dev/null 2>&1 &&
cp {f} proj/Program.cs && "
                              "sed -i 's#<OutputType>Exe</OutputType>#
<OutputType>Library</OutputType>#' proj/proj.csproj",
                    check="cd proj && dotnet build -nologo -v q -clp:ErrorsOnly",
run="cd proj && dotnet run --no-build",
                    run_if=lambda c: bool(re.search(r"static\s+[\w<>\
[\],\s]+\bMain\s*\(", c)) or
                    not
re.search(r"\b(class|struct|interface|enum|record|namespace)\b", c),
                    dep=r"CS0246|CS0234|CS0400|could not be found \(are you missing",
tmo=300),
    "go": _spec(".go", "golang:1.23", "main.go",
                setup="go mod init snippet >/dev/null 2>&1; go mod tidy >/dev/null
2>&1; true",
                check="go build ./...", run="go run .",
run_if=_re_has(r"^package\s+main\b"),
                dep=r"no required module provides package|cannot find package|missing
go\.sum entry|dial tcp|lookup proxy|is not in std|no Go files",
                tmo=240),
    "rust": _spec(".rs", "rust:1", "main.rs",
                   check="rustc --edition 2021 -o out {f}", lib_check="rustc --edition
2021 --crate-type lib -o out.rlib {f}",
                   run="./out", run_if=_re_has(r"\bfn\s+main\s*\("),
                   dep=r"can't find crate|unresolved import|failed to resolve: use of
undeclared crate|unlinked crate|cannot find (derive |attribute )?macro",
                   dep_code=r"^\s*(?:pub\s+)?(?:use|extern\s+crate)\s+
(?!std\b|core\b|alloc\b|crate\b|self\b|super\b)", tmo=240),
    "swift": _spec(".swift", "swift:6.0", "main.swift", check="swiftc -o out {f}",
run="./out",
                    dep=r"no such module", tmo=300),
    "php": _spec(".php", "php:8.3-cli", "main.php", check="php -l {f}", run="php
{f}", run_if=_re_has(r"<\?php"),
                  dep=r"Failed opening required|Class \"[^\"]+\" not found|Interface
\"[^\"]+\" not found|Call to undefined function (curl|mysqli|mb|gd)\w*"),
```

```
    "ruby": _spec(".rb", "ruby:3.3", "main.rb", check="ruby -c {f}", run="ruby {f}",
                  dep=r"cannot load such file|LoadError"),
    "perl": _spec(".pl", "perl:5", "main.pl", check="perl -c {f}", run="perl {f}",
                  dep=r"Can't locate [\w/.]+ in @INC"),
    "lua": _spec(".lua", "nickblah/lua:5.4", "main.lua", check="lua -e
\"assert(loadfile('{f}'))\"", run="lua {f}",
                 dep=r"module '[^']+' not found"),
    "r": _spec(".R", "r-base:latest", "main.R", check="Rscript -e
\"invisible(parse(file='{f}'))\"", run="Rscript {f}",
               dep=r"there is no package called|Error in library"),
    "dart": _spec(".dart", "dart:stable", "main.dart", check="dart analyze --no-
fatal-warnings {f}",
                  run="dart run {f}", run_if=_re_has(r"\bmain\s*\("),
                  dep=r"Target of URI doesn't exist|Could not
resolve|uri_does_not_exist"),
    "haskell": _spec(".hs", "haskell:9.8", "{mod}.hs", check="ghc -fno-code {f}",
run="runghc {f}",
                     run_if=_re_has(r"^main\s*(::|=)"), dep=r"Could not find
module|Failed to load interface"),
    "elixir": _spec(".exs", "elixir:1.17", "main.exs",
                    check="elixir -e 'Code.string_to_quoted!(File.read!(\"{f}\"))'",
run="elixir {f}",
                    dep=r"Mix\.install|is not available|could not load module|is not
loaded and could not be found"),
    "erlang": _spec(".erl", "erlang:27", "{mod}.erl", check="erlc {f}", dep=r"can't
find include"),
    "clojure": _spec(".clj", "clojure:temurin-21-tools-deps", "main.clj",
run="clojure -M {f}",
                     dep=r"Could not locate [\w/.]+ on classpath|No such namespace",
tmo=240),
    "julia": _spec(".jl", "julia:1.11", "main.jl", run="julia {f}", dep=r"Package \w+
not found"),
    "groovy": _spec(".groovy", "groovy:jdk21", "main.groovy", check="groovyc {f}",
run="groovy {f}",
                    dep=r"unable to resolve class", tmo=240),
    "powershell": _spec(".ps1", "mcr.microsoft.com/powershell", "main.ps1",
                        check="pwsh -NoProfile -Command '$e=$null; [void]
[System.Management.Automation.Language.Parser]::ParseFile(\"{f}\",[ref]$null,
[ref]$e); if ($e.Count -gt 0) { $e | ForEach-Object { $_.Message }; exit 1 }'"),
    "cobol": _spec(".cob", "ubuntu:24.04", "main.cob", setup=_apt("gnucobol"),
                   check="cobc -x -o out {f} || cobc -x -free -o out {f}",
run="./out", tmo=240),
    "pascal": _spec(".pas", "ubuntu:24.04", "main.pas", setup=_apt("fp-compiler"),
                    check="fpc -o./out {f}", run="./out", tmo=240),
    "nasm": _spec(".asm", "ubuntu:24.04", "main.asm", setup=_apt("nasm binutils"),
                  check="nasm -felf64 {f} -o out.o && ld out.o -o out", tmo=240),
    "ocaml": _spec(".ml", "ubuntu:24.04", "main.ml", setup=_apt("ocaml-nox"),
                   check="ocamlopt {f} -o out", run="./out", tmo=240),
    "dlang": _spec(".d", "ubuntu:24.04", "main.d", setup=_apt("gdc"), check="gdc -o
out {f}", run="./out", tmo=240),
```

```
    "ada": _spec(".adb", "ubuntu:24.04", "main.adb", setup=_apt("gnat"),
check="gnatmake -q {f}", tmo=240),
    "tcl": _spec(".tcl", "ubuntu:24.04", "main.tcl", setup=_apt("tcl"), run="tclsh
{f}", tmo=240),
    "nim": _spec(".nim", "nimlang/nim", "main.nim", check="nim check --hints:off
{f}", run="nim c -r --hints:off {f}",
                 dep=r"cannot open file", tmo=240),
    "crystal": _spec(".cr", "crystallang/crystal", "main.cr", check="crystal build --
no-codegen {f}",
                     run="crystal run {f}", dep=r"can't find file", tmo=240),
    "octave": _spec(".m", "gnuoctave/octave:9.2.0", "main.m", run="octave --no-gui --
quiet {f}",
                    dep=r"package .* is not installed|pkg load", tmo=240),
    "solidity": _spec(".sol", "ethereum/solc:stable", "main.sol", check="solc --bin
{f} > /dev/null",
                      dep=r"Source \"[^\"]+\" not found|File not found"),
    "racket": _spec(".rkt", "racket/racket:8.14-full", "main.rkt", check="raco make
{f}", run="racket {f}",
                    dep=r"cannot open module file|collection not found", tmo=240),
}
_POLY_EXT = {s["ext"]: lang for lang, s in _LANG_SPECS.items()}

_STDIN_PAT = r"NoSuchElementException|EOFError|InputMismatchException|EOF when
reading|unexpected end of input"
_NET_PAT = r"Connection refused|Name or service not known|Temporary failure in name
resolution|Network is unreachable|UnknownHostException|timed out"
_GUI_PAT = r"Cannot connect to display|no display|HeadlessException|DISPLAY
environment|could not open display"
_CRASH_PAT = r"Traceback|Exception|panicked|panic:|Segmentation fault|Fatal
error|FATAL|Unhandled|core dumped|terminate called|\*\* \(|[Ee]rror"


def _ext_for_tag(tag):
    """Fence tag -> file extension we can verify, or None."""
    lang = _LANG_ALIASES.get((tag or "").strip().lower())
    if not lang:
        return None
    if lang in _DEFAULT_LANG_EXT:
        return _DEFAULT_LANG_EXT[lang]
    if lang in _LANG_SPECS:
        return _LANG_SPECS[lang]["ext"]
    return None


def _split_polyglot(files):
    poly, rest = [], []
    for f in files:
        base = os.path.basename(f.get("name") or "")
        ext = os.path.splitext(base)[1]
        if base.startswith("snippet_") and ext in _POLY_EXT:
```

```
            poly.append(f)
        else:
            rest.append(f)
    return poly, rest


class _ToolchainMissing(Exception):
    pass


def _sbx_exec(sandbox, cmd, timeout):
    """Run a shell command in the sandbox -> (exit_code, output). Never raises."""
    import shlex
    wrapped = "sh -c " + shlex.quote("{ %s ; } 2>&1; echo @@EXIT@@$?" % cmd)
    try:
        resp = sandbox.process.exec(wrapped, timeout=int(timeout))
    except Exception as e:
        return 124, "timed out or failed to execute: " + str(e)[:200]
    out = getattr(resp, "result", "") or ""
    m = re.search(r"@@EXIT@@(\d+)\s*$", out)
    if m:
        return int(m.group(1)), out[:m.start()]
    rc = getattr(resp, "exit_code", None)
    return (rc if rc is not None else 1), out


def _java_names(code):
    pub = re.search(r"\bpublic\s+(?:final\s+|abstract\s+|sealed\s+|static\s+)*
(?:class|interface|enum|record)\s+(\w+)", code)
    decls = list(re.finditer(r"\b(?:class|interface|enum|record)\s+(\w+)", code))
    mm = re.search(r"static\s+void\s+main\s*\(", code)
    run_cls = None
    if mm:
        before = [d for d in decls if d.start() < mm.start()]
        if before:
            run_cls = before[-1].group(1)
    file_cls = pub.group(1) if pub else (run_cls or (decls[0].group(1) if decls else
"Main"))
    return file_cls, (run_cls or file_cls)


def _poly_names(lang, spec, code):
    ctx = {"cls": "Main", "run": "Main", "mod": "Main"}
    if lang == "java":
        ctx["cls"], ctx["run"] = _java_names(code)
    elif lang == "erlang":
        m = re.search(r"-module\(\s*(\w+)\s*\)", code)
        ctx["mod"] = m.group(1) if m else "main"
    elif lang == "haskell":
        m = re.search(r"^module\s+([\w.]+)", code, re.M)
```

```
        ctx["mod"] = m.group(1).split(".")[-1] if m else "Main"
    return ctx


def _fill(template, ctx):
    for k, v in ctx.items():
        template = template.replace("{" + k + "}", v)
    return template


def _poly_classify(spec, code, text_out):
    """Compile/link failure -> ('warning'|'error'). Missing external deps are
unverifiable, not bugs."""
    dep = spec.get("dep")
    gate = spec.get("dep_code")
    if dep and re.search(dep, text_out) and (not gate or re.search(gate, code,
re.M)):
        return "warning", "needs external libraries/packages that are not available
in the sandbox - could not be fully checked"
    if re.search(_NET_PAT, text_out):
        return "warning", "needs internet which the sandbox does not have - could not
be fully checked"
    return "error", ""


def _poly_check_one(sandbox, lang, rel, code, idx):
    spec = _LANG_SPECS[lang]
    errors, warnings = [], []
    d = "poly/s%d" % idx
    ctx = _poly_names(lang, spec, code)
    fname = _fill(spec["file"], ctx)
    ctx["f"] = fname
    _sbx_exec(sandbox, "mkdir -p " + d, 30)
    sandbox.fs.upload_file(code.encode("utf-8"), d + "/" + fname)
    runnable = True
    if spec.get("run_if"):
        runnable = bool(spec["run_if"](code))
    tmo = spec.get("tmo", 150)

    def step(label, cmd, timeout, run_phase=False):
        rc, out = _sbx_exec(sandbox, "cd %s && %s" % (d, _fill(cmd, ctx)), timeout)
        out = (out or "").strip()
        if rc == 127 or "command not found" in out or "executable file not found" in
out:
            raise _ToolchainMissing("toolchain for %s is missing in image %s: %s" %
(lang, spec["image"], out[:200]))
        if rc == 0:
            return True
        if run_phase:
            if rc == 124:
```

```
                warnings.append("%s (%s): still running after the time limit (server
or loop) - started without crashing" % (rel, lang))
            elif re.search(_STDIN_PAT, out):
                warnings.append("%s (%s): waits for keyboard input - ran fine until
the input prompt" % (rel, lang))
            elif re.search(_GUI_PAT, out):
                warnings.append("%s (%s): needs a screen/GUI - runtime not fully
checked" % (rel, lang))
            elif re.search(_NET_PAT, out):
                warnings.append("%s (%s): needs internet - runtime not fully checked"
% (rel, lang))
            elif rc >= 128 or re.search(_CRASH_PAT, out):
                errors.append("%s (%s): crashed when executed (exit %d):\n%s" % (rel,
lang, rc, out[:900]))
            else:
                warnings.append("%s (%s): exited with code %d without an error
message" % (rel, lang, rc))
            return False
        if rc == 124:
            raise _ToolchainMissing("%s %s step timed out" % (lang, label))
        kind, msg = _poly_classify(spec, code, out)
        if kind == "warning":
            warnings.append("%s (%s): %s" % (rel, lang, msg))
        else:
            errors.append("%s (%s): %s failed:\n%s" % (rel, lang, label, out[:900]))
        return False

    setup = spec.get("setup_lib") if (not runnable and spec.get("setup_lib")) else
spec.get("setup")
    if setup:
        rc, out = _sbx_exec(sandbox, "cd %s && %s" % (d, _fill(setup, ctx)), 240)
        if rc != 0:
            raise _ToolchainMissing("setup for %s failed in image %s: %s" % (lang,
spec["image"], (out or "")[:200]))
    chk = spec.get("lib_check") if (not runnable and spec.get("lib_check")) else
spec.get("check")
    if chk and not step("compile/check", chk, tmo):
        return errors, warnings
    if runnable and spec.get("link") and not step("link", spec["link"], tmo):
        return errors, warnings
    if runnable and spec.get("run"):
        step("run", "timeout 15 " + spec["run"] + " < /dev/null", 40, run_phase=True)
    return errors, warnings


_poly_ready = set()
_poly_warming = {}
_poly_lock = __import__("threading").Lock()
```

```
def _poly_create_sandbox(image):
    from daytona import CreateSandboxFromImageParams
    return get_daytona().create(CreateSandboxFromImageParams(image=image),
timeout=DAYTONA_IMAGE_TIMEOUT)


def _poly_delete(sandbox):
    if sandbox is None:
        return
    try:
        sandbox.delete()
    except Exception:
        try:
            get_daytona().delete(sandbox)
        except Exception:
            pass


def _poly_warm(image):
    sb = None
    try:
        sb = _poly_create_sandbox(image)
        _sbx_exec(sb, "echo ok", 30)
        _poly_ready.add(image)
    except Exception:
        pass
    finally:
        _poly_delete(sb)
        with _poly_lock:
            _poly_warming.pop(image, None)


def _poly_wait_ready(image, wait):
    """First use of a language image builds Daytona's snapshot (can take minutes).
Build it in the
    background; wait a bounded time so the request never hangs."""
    if image in _poly_ready:
        return True
    import threading
    with _poly_lock:
        t = _poly_warming.get(image)
        if t is None:
            t = threading.Thread(target=_poly_warm, args=(image,), daemon=True)
            _poly_warming[image] = t
            t.start()
    t.join(wait)
    return image in _poly_ready


def _daytona_verify_polyglot(poly_files):
```

```
    res = {"ran": False, "errors": [], "warnings": [], "notes": [], "reason": ""}
    groups = {}
    for f in poly_files:
        lang = _POLY_EXT.get(os.path.splitext(f["name"])[1])
        groups.setdefault(_LANG_SPECS[lang]["image"], []).append((lang, f))
    for image, items in groups.items():
        names = ", ".join(sorted({l for l, _ in items}))
        if not _poly_wait_ready(image, DAYTONA_POLY_FIRST_WAIT):
            res["reason"] = ("first-time setup of the %s verification environment is
still running "
                             "(happens once per language) - please try again in a few
minutes" % names)
            return res
        sandbox = None
        try:
            sandbox = _poly_create_sandbox(image)
            for i, (lang, f) in enumerate(items):
                e, w = _poly_check_one(sandbox, lang, f["name"], f.get("content") or
"", i)
                res["errors"].extend(e)
                res["warnings"].extend(w)
        except _ToolchainMissing as t:
            res["reason"] = str(t)[:300]
            return res
        except Exception as e:
            res["reason"] = "Daytona error (%s): %s" % (image, str(e)[:250])
            return res
        finally:
            _poly_delete(sandbox)
    res["ran"] = True
    return res


def _daytona_verify_inner(files):
    """Routes snippets: default-sandbox checker for common web/script languages, per-
language
    image sandboxes for everything else. ran=True only if every part ran."""
    base = {"ran": False, "errors": [], "warnings": [], "notes": [], "reason": ""}
    if not DAYTONA_ENABLED:
        base["reason"] = "DAYTONA_API_KEY not set"
        return base
    poly, rest = _split_polyglot(files)
    parts = []
    if rest or not poly:
        parts.append(_daytona_verify_default(rest))
    if poly:
        parts.append(_daytona_verify_polyglot(poly))
    out = {"ran": all(p["ran"] for p in parts), "errors": [], "warnings": [],
"notes": [], "reason": ""}
    for p in parts:
```

```
        out["errors"].extend(p.get("errors", []))
        out["warnings"].extend(p.get("warnings", []))
        out["notes"].extend(p.get("notes", []))
        if not p["ran"] and p.get("reason"):
            out["reason"] = (out["reason"] + " | " if out["reason"] else "") +
p["reason"]
    return out


def daytona_verify(files, timeout=None):
    """Time-bounded wrapper: the sandbox check runs in a worker thread and is
abandoned
    (ran=False, answer still delivered) if it exceeds the limit. The worker's own
    finally-block still deletes the sandbox when it finishes."""
    import threading
    base = {"ran": False, "errors": [], "warnings": [], "notes": [], "reason": ""}
    if not DAYTONA_ENABLED:
        base["reason"] = "DAYTONA_API_KEY not set"
        return base
    limit = DAYTONA_CHECK_TIMEOUT if timeout is None else max(5,
min(DAYTONA_CHECK_TIMEOUT, int(timeout)))
    box = {}

    def _work():
        try:
            box["res"] = _daytona_verify_inner(files)
        except BaseException as e:
            box["err"] = str(e)[:300]

    t = threading.Thread(target=_work, daemon=True)
    t.start()
    t.join(limit)
    if t.is_alive():
        base["reason"] = "Daytona check timed out after %ds" % limit
        return base
    if "res" in box:
        return box["res"]
    base["reason"] = "Daytona error: " + box.get("err", "unknown")
    return base


_FILE_BLOCK_RE = re.compile(r'===FILE:\s*(.+?)===\n([\s\S]*?)===ENDFILE===')
_FENCE_RE = re.compile(r"```([a-zA-Z0-9_+#.-]*)[ \t]*\n([\s\S]*?)```")
_CHAT_LANG_EXT = {"python": ".py", "py": ".py", "json": ".json", "html": ".html",
"javascript": ".js", "js": ".js", "jsx": ".jsx"}
DAYTONA_CHAT_FEATURES = {
    "Build Web", "Build App", "General AI", "Everything AI", "Hunt", "Quick Fixer",
    "Fix", "Solve", "Modernize", "PureCoder", "AI Assistant", "Write Code",
}
```

```
def _strip_outer_fence(content):
    c = content.strip()
    m = re.match(r"^```[a-zA-Z0-9_+-]*\n([\s\S]*?)\n?```$", c)
    return m.group(1) if m else content


def _format_files_for_prompt(files):
    return "\n".join("===FILE: %s===\n%s\n===ENDFILE===" % (f["name"], f["content"])
for f in files)


def _llm_retry(messages, system, max_tokens=32000, deadline=None, temperature=0.0):
    for attempt in range(3):
        remaining = None if deadline is None else deadline - time.time()
        if remaining is not None and remaining < 15:
            return None
        try:
            text, _, _ = llm_chat(messages, system=system, temperature=temperature,
max_tokens=max_tokens,
                                  timeout=300 if remaining is None else max(10,
min(300, remaining)))
            return text
        except Exception:
            time.sleep(3)
    return None


def _llm_fix_files(files, errors, deadline=None, temperature=0.0):
    system = (
        "You are a senior engineer fixing errors found by an automated sandbox check.
"
        "Change ONLY what is needed to fix the listed errors; keep everything else
identical. "
        "Return ALL project files complete (never truncate, no placeholders) in this
EXACT format and nothing else:\n"
        "===FILE: filename.ext===\n[complete file content]\n===ENDFILE===\n"
        "No markdown, no explanations."
    )
    user = (
        "ERRORS FOUND BY THE SANDBOX CHECK:\n" + "\n".join("- " + e for e in errors)
+
        "\n\nCURRENT FILES:\n" + _format_files_for_prompt(files) +
        "\n\nReturn ALL files complete, fixed, in the same ===FILE=== format."
    )
    text = _llm_retry([{"role": "user", "content": user}], system, deadline=deadline,
temperature=temperature)
    if not text:
        return files
    fixed = {n.strip(): _strip_outer_fence(c).strip() for n, c in
```

```
_FILE_BLOCK_RE.findall(text)}
    if not fixed:
        return files
    out = []
    for f in files:
        new = fixed.get(f["name"])
        # truncation guard: ignore a "fix" that is far shorter than the original
        if new and len(new) >= 0.5 * len(f["content"]):
            out.append({"name": f["name"], "content": new})
        else:
            out.append(f)
    known = {f["name"] for f in files}
    for n, c in fixed.items():
        if n not in known and c:
            out.append({"name": n, "content": c})
    return out


def _verify_with_retry(files, deadline):
    """Sandbox check that retries when Daytona itself fails (network/infra), so a
hiccup
    in Daytona is not mistaken for 'code could not be verified'."""
    res = None
    for _ in range(3):
        remaining = deadline - time.time()
        if remaining < 20:
            break
        res = daytona_verify(files, timeout=remaining)
        if res["ran"] or "API_KEY not set" in (res.get("reason") or "") or "first-
time setup" in (res.get("reason") or ""):
            return res
        time.sleep(2)
    if res is None:
        res = {"ran": False, "errors": [], "warnings": [], "notes": [], "reason":
"time budget exhausted"}
    return res


def _persist_verify(first, verify, regen):
    """Keeps going until the candidate is REALLY verified:
       patch-fix loop -> (still failing) regenerate the whole answer with the errors
as a hint -> verify again.
       verify(candidate, budget_seconds) -> (candidate, info);  regen(hint,
temperature) -> candidate | None.
       Returns (candidate, info). info['verified'] is True/None for a real pass; only
if every attempt
       failed does it fall back to the best attempt (info['delivered_unverified'] =
True)."""
    deadline = time.time() + DAYTONA_PERSIST_BUDGET
    cand = first
```

```
    best = None
    infra, regen_n = 0, 0
    while True:
        budget = max(30, min(DAYTONA_TOTAL_BUDGET, deadline - time.time()))
        cand2, info = verify(cand, budget)
        if info.get("verified") is not False:
            return cand2, info
        score = len(info.get("errors") or []) + (0 if info.get("ran") else 50)
        if best is None or score < best[0]:
            best = (score, cand2, info)
        if time.time() > deadline - 45:
            break
        if not info.get("ran"):                    # Daytona/infra hiccup: re-check
the SAME answer
            infra += 1
            if infra > 8:
                break
            time.sleep(min(20, 5 * infra))
            cand = cand2
            continue
        if regen_n >= DAYTONA_REGEN_ATTEMPTS:
            break
        regen_n += 1
        errs = "\n".join("- " + str(e)[:350] for e in (info.get("errors") or [])[:4])
        hint = ("IMPORTANT: your previous answer FAILED an automated sandbox run with
these errors:\n" + errs +
                "\nWrite a COMPLETE new answer in the same required format that
avoids these errors and passes.")
        new = None
        try:
            new = regen(hint, min(1.0, 0.2 + 0.25 * regen_n))
        except Exception:
            new = None
        if not new:
            break
        cand = new
    info = dict(best[2])
    info["delivered_unverified"] = True
    return best[1], info


def _strict_blocked(info):
    """Never block: the user must never get a notice/empty result. Retries are
silent."""
    return False


def _public_verification(info):
    """Client-facing verification info: no errors / failure reasons / notices ever
leak out."""
```

```
    info = info if isinstance(info, dict) else {}
    return {"verified": info.get("verified"), "ran": bool(info.get("ran"))}


def _blocked_notice(info):
    errs = [e for e in (info.get("errors") or [])][:3]
    reason = info.get("reason") or ""
    lines = ["Code was NOT delivered: it could not be confirmed as 100% passing in
the Daytona sandbox."]
    if errs:
        lines.append("Remaining errors:")
        lines.extend("- " + str(e)[:400] for e in errs)
    elif reason:
        lines.append("Reason: " + str(reason)[:300])
    lines.append("Please try again (nothing unverified is ever shown).")
    return "\n".join(lines)


def verify_and_fix_files(files, max_rounds=None, budget=None):
    """Check -> fix -> re-check loop for Full Stack Builder. verified=True ONLY if
the sandbox
    ran and found zero errors. Returns (files, info)."""
    rounds = DAYTONA_FIX_ROUNDS if max_rounds is None else max_rounds
    info = {"verified": False, "ran": False, "rounds": 0, "errors": [], "warnings":
[], "reason": ""}
    deadline = time.time() + (budget or DAYTONA_TOTAL_BUDGET)
    attempts = 0
    res = None
    while attempts <= rounds:
        if deadline - time.time() < 20:
            info["reason"] = "time budget exhausted"
            break
        res = _verify_with_retry(files, deadline)
        info.update({"ran": res["ran"], "warnings": res["warnings"], "reason":
res["reason"], "errors": res["errors"]})
        if not res["ran"]:
            break
        if not res["errors"]:
            info["verified"] = True
            return files, info
        if attempts == rounds:
            break
        if deadline - time.time() < 40:
            info["reason"] = "time budget exhausted before fix"
            break
        new_files = _llm_fix_files(files, res["errors"], deadline,
temperature=min(0.8, 0.25 * attempts))
        attempts += 1
        info["rounds"] = attempts
        if new_files != files:
```

```
            files = new_files
        # if the AI returned no change, loop again with a higher temperature to get a
different fix
    return files, info


def extract_checkable_files(text):
    files = []
    for i, m in enumerate(_FENCE_RE.finditer(text or "")):
        ext = _ext_for_tag(m.group(1))
        if ext and m.group(2).strip():
            files.append({"name": "snippet_%d%s" % (i + 1, ext), "content":
m.group(2)})
    if files:
        return files
    s = (text or "").strip()
    head = s[:200].lower()
    if head.startswith("<!doctype html") or head.startswith("<html"):
        return [{"name": "index.html", "content": s}]
    if "export default" in s and re.match(r"^(import\s|//|/\*|'use client'|\"use
client\")", s):
        return [{"name": "App.jsx", "content": s}]
    return []


def _llm_fix_response(text, errors, deadline=None, temperature=0.0):
    system = (
        "You fix syntax/runtime errors in code. Return the COMPLETE corrected
response in exactly the same "
        "format as the original (same code fences or same raw code, same surrounding
text). "
        "Fix only the listed errors, never truncate, and add no commentary about the
fix."
    )
    user = (
        "ERRORS FOUND BY THE SANDBOX CHECK:\n" + "\n".join("- " + e for e in errors)
+
        "\n\nORIGINAL RESPONSE:\n" + text +
        "\n\nReturn the complete corrected response now."
    )
    new = _llm_retry([{"role": "user", "content": user}], system, deadline=deadline,
temperature=temperature)
    if new and len(new.strip()) >= 0.6 * len(text.strip()):
        return new
    return text


def verify_and_fix_response(text, max_rounds=None, budget=None):
    """Check -> fix -> re-check loop for every chat feature. verified=None only when
the answer has
```

```
    no checkable code; verified=True only when the sandbox ran and found zero errors;
otherwise False."""
    rounds = DAYTONA_FIX_ROUNDS_CHAT if max_rounds is None else max_rounds
    info = {"verified": None, "ran": False, "rounds": 0, "errors": [], "warnings":
[], "reason": ""}
    deadline = time.time() + (budget or DAYTONA_TOTAL_BUDGET)
    attempts = 0
    while attempts <= rounds:
        files = extract_checkable_files(text)
        if not files:
            if attempts == 0:
                return text, info          # nothing to check (plain text answer)
            info["verified"] = False       # a fix removed the code -> do not trust
it
            info["reason"] = "fix removed the code"
            return text, info
        if deadline - time.time() < 20:
            info["verified"] = False
            info["reason"] = "time budget exhausted"
            break
        res = _verify_with_retry(files, deadline)
        info.update({"ran": res["ran"], "warnings": res["warnings"], "reason":
res["reason"], "errors": res["errors"]})
        if not res["ran"]:
            info["verified"] = False
            return text, info
        if not res["errors"]:
            info["verified"] = True
            return text, info
        info["verified"] = False
        if attempts == rounds:
            break
        if deadline - time.time() < 40:
            info["reason"] = "time budget exhausted before fix"
            break
        new_text = _llm_fix_response(text, res["errors"], deadline,
temperature=min(0.8, 0.25 * attempts))
        attempts += 1
        info["rounds"] = attempts
        if new_text != text:
            text = new_text
        # unchanged -> next round retries with a higher temperature
    return text, info


# ==== DAYTONA END ====================================================

# ==== EXPO SDK AUTO-VERSION ==========================================
# Hamesha latest STABLE Expo SDK use karta hai (npm "latest" tag se) - jab Expo
# SDK 59, 60... release karega to app.py me kuch change nahi karna padega.
```

```
# React / React Native ke exact versions Expo ki apni bundledNativeModules.json
# se aate hain (wahi file jo `npx expo install` use karta hai), isliye hamesha match.
# Agar internet/registry down ho to neeche wala FALLBACK (SDK 58) use hota hai.
_EXPO_FALLBACK = {"expo": "~58.0.0", "react": "19.2.3", "react_native": "0.88.0",
"sdk_version": "58.0.0"}
_expo_version_cache = {"data": None, "fetched_at": 0}

def _expo_clean_ver(v):
    v = str(v or "").strip().lstrip("^~=v ")
    return v if re.match(r"^\d+\.\d+\.\d+", v) else None

def _expo_major(v):
    try:
        return int(str(v).split(".")[0])
    except Exception:
        return 0

def get_latest_expo_versions():
    now = time.time()
    c = _expo_version_cache
    if c["data"] and (now - c["fetched_at"] < 3600):
        return c["data"]
    try:
        import requests as _req
        # 1) latest stable expo SDK (npm "latest" tag is never a beta/canary)
        data = _req.get("https://registry.npmjs.org/expo/latest", timeout=6).json()
        expo_version = _expo_clean_ver(data.get("version"))
        if not expo_version:
            raise ValueError("bad expo version")
        sdk_major = _expo_major(expo_version)
        if sdk_major < _expo_major(_EXPO_FALLBACK["sdk_version"]):
            raise ValueError("registry older than fallback")

        # 2) exact react / react-native that this SDK uses
        react_v = rn_v = None
        for url in (

f"https://cdn.jsdelivr.net/npm/expo@{expo_version}/bundledNativeModules.json",
            f"https://unpkg.com/expo@{expo_version}/bundledNativeModules.json",
        ):
            try:
                bn = _req.get(url, timeout=6).json()
                react_v = _expo_clean_ver(bn.get("react"))
                rn_v = _expo_clean_ver(bn.get("react-native"))
                if react_v and rn_v:
                    break
            except Exception:
                continue

        # 3) backup source: Expo's own versions API
```

```
        if not (react_v and rn_v):
            try:
                vs = _req.get("https://api.expo.dev/v2/versions/latest",
timeout=6).json()
                sv = (vs.get("data") or vs).get("sdkVersions", {}).get(f"
{sdk_major}.0.0", {})
                react_v = react_v or _expo_clean_ver(sv.get("facebookReactVersion"))
                rn_v = rn_v or _expo_clean_ver(sv.get("facebookReactNativeVersion")
or sv.get("reactNativeVersion"))
            except Exception:
                pass

        # same major as fallback -> fill gaps from fallback; newer major -> need real
values
        if sdk_major == _expo_major(_EXPO_FALLBACK["sdk_version"]):
            react_v = react_v or _EXPO_FALLBACK["react"]
            rn_v = rn_v or _EXPO_FALLBACK["react_native"]
        if not (react_v and rn_v):
            raise ValueError("react versions unavailable")

        result = {"expo": f"~{sdk_major}.0.0", "react": react_v, "react_native":
rn_v, "sdk_version": f"{sdk_major}.0.0"}
        c["data"] = result
        c["fetched_at"] = now
        return result
    except Exception:
        # keep serving last good value; retry again in ~5 min (no 6s delay on every
request)
        res = c["data"] or dict(_EXPO_FALLBACK)
        c["data"] = res
        c["fetched_at"] = now - 3600 + 300
        return res
# ==== EXPO SDK AUTO-VERSION END ============================================
# ── PADDLE WEBHOOK CONFIG
──────────────────────────────────────────────────────────

PADDLE_WEBHOOK_SECRET = os.environ.get("PADDLE_WEBHOOK_SECRET")
PADDLE_API_KEY = os.environ.get("PADDLE_API_KEY")
NOWPAYMENTS_API_KEY = os.environ.get("NOWPAYMENTS_API_KEY")
NOWPAYMENTS_IPN_SECRET = os.environ.get("NOWPAYMENTS_IPN_SECRET")
NOWPAYMENTS_API_URL = "https://api.nowpayments.io/v1"

PLAN_PRICES = {
    "Pro": {"amount": 12, "credits": 300, "days": 30},
    "Heavy Pro": {"amount": 19, "credits": 500, "days": 30}
}

BREVO_API_KEY = os.environ.get("BREVO_API_KEY")
BREVO_SENDER_EMAIL = "notifications@wholeai.space"
BREVO_SENDER_NAME = "Whole Ai"
SITE_URL = "https://www.wholeai.space/"
```

```
def send_brevo_email(to_email, to_name, subject, html_content):
    import requests
    try:
        url = "https://api.brevo.com/v3/smtp/email"
        headers = {
            "accept": "application/json",
            "api-key": BREVO_API_KEY,
            "content-type": "application/json"
        }
        payload = {
            "sender": {"name": BREVO_SENDER_NAME, "email": BREVO_SENDER_EMAIL},
            "to": [{"email": to_email, "name": to_name or to_email}],
            "subject": subject,
            "htmlContent": html_content
        }
        resp = requests.post(url, json=payload, headers=headers, timeout=15)
        if resp.status_code in (200, 201):
            return {"success": True}
        return {"success": False, "error": resp.text}
    except Exception as e:
        return {"success": False, "error": str(e)}


def send_signup_welcome_email(to_email, to_name):
    subject = "Welcome to Whole AI"
    html_content = f"""
    <div style="font-family:'Segoe UI',Arial,sans-serif;max-width:600px;margin:0
auto;background:#0f0f0f;color:#e5e5e5;border-radius:12px;overflow:hidden;">
        <div style="background:#007ACC;padding:30px;text-align:center;">
            <img src="https://i.postimg.cc/9FG7RNpz/whole-ai-logo.png" width="60"
alt="Whole AI" style="border-radius:12px;">
            <h1 style="color:#fff;margin:15px 0 0;font-size:24px;">Whole AI</h1>
        </div>
        <div style="padding:35px 30px;">
            <h2 style="color:#fff;margin-top:0;">Welcome, {to_name or 'there'}</h2>
            <p style="font-size:15px;line-height:1.6;color:#c9c9c9;">
                Your Whole AI account has been created successfully. You now have
access to
                <b style="color:#fff;">9 AI-powered features</b> including Code
Review, Bug Hunter, Security Scanning,
                Build Web, Build App, and our Full Stack Builder.
            </p>
            <p style="font-size:15px;line-height:1.6;color:#c9c9c9;">
                You get <b style="color:#fff;">10 bonus credits</b> on signup, plus
<b style="color:#fff;">7 free credits every day</b> after that, with no credit card
required.
            </p>
            <div style="text-align:center;margin:30px 0;">
```

```
                <a href="{SITE_URL}" style="background:#007ACC;color:#fff;text-
decoration:none;padding:14px 32px;border-radius:8px;font-weight:600;font-
size:15px;display:inline-block;">
                    Go to Whole AI
                </a>
            </div>
            <p style="font-size:13px;color:#8a8a8a;line-height:1.5;">
                If you have any questions, contact our support team at
                <a href="mailto:wholeaisupport@gmail.com"
style="color:#007ACC;">wholeaisupport@gmail.com</a>
            </p>
        </div>
        <div style="background:#1a1a1a;padding:18px;text-align:center;font-
size:12px;color:#777;">
            © 2026 Whole AI. All rights reserved.
        </div>
    </div>
    """
    return send_brevo_email(to_email, to_name, subject, html_content)


def send_subscription_success_email(to_email, to_name, plan_type, credits, days):
    subject = f"Your Whole AI {plan_type} Plan is Active"
    html_content = f"""
    <div style="font-family:'Segoe UI',Arial,sans-serif;max-width:600px;margin:0
auto;background:#0f0f0f;color:#e5e5e5;border-radius:12px;overflow:hidden;">
        <div style="background:#007ACC;padding:30px;text-align:center;">
            <img src="https://i.postimg.cc/9FG7RNpz/whole-ai-logo.png" width="60"
alt="Whole AI" style="border-radius:12px;">
            <h1 style="color:#fff;margin:15px 0 0;font-size:24px;">Whole AI</h1>
        </div>
        <div style="padding:35px 30px;">
            <h2 style="color:#fff;margin-top:0;">Payment Successful</h2>
            <p style="font-size:15px;line-height:1.6;color:#c9c9c9;">
                Hi {to_name or 'there'}, your <b style="color:#fff;">{plan_type}</b>
plan is now active on your account.
            </p>
            <div style="background:#1a1a1a;border-
radius:10px;padding:20px;margin:20px 0;">
                <p style="margin:0 0 10px;font-size:14px;color:#c9c9c9;">Plan: <b
style="color:#fff;">{plan_type}</b></p>
                <p style="margin:0 0 10px;font-size:14px;color:#c9c9c9;">Credits: <b
style="color:#fff;">{credits} per month</b></p>
                <p style="margin:0;font-size:14px;color:#c9c9c9;">Duration: <b
style="color:#fff;">{days} days</b></p>
            </div>
            <p style="font-size:15px;line-height:1.6;color:#c9c9c9;">
                You now have faster processing and higher-quality output across all
features. Your plan will
                automatically revert to Free once it expires — there is no auto-
```

```
renewal and no hidden charges.
            </p>
            <div style="text-align:center;margin:30px 0;">
                <a href="{SITE_URL}" style="background:#007ACC;color:#fff;text-
decoration:none;padding:14px 32px;border-radius:8px;font-weight:600;font-
size:15px;display:inline-block;">
                    Go to Whole AI
                </a>
            </div>
            <p style="font-size:13px;color:#8a8a8a;line-height:1.5;">
                For billing questions, contact us at
                <a href="mailto:wholeaisupport@gmail.com"
style="color:#007ACC;">wholeaisupport@gmail.com</a>
            </p>
        </div>
        <div style="background:#1a1a1a;padding:18px;text-align:center;font-
size:12px;color:#777;">
            © 2026 Whole AI. All rights reserved.
        </div>
    </div>
    """
    return send_brevo_email(to_email, to_name, subject, html_content)
@app.route('/')
def index():
    return render_template('index.html')


@app.route('/api/send-signup-email', methods=['POST'])
def send_signup_email():
    try:
        data = request.get_json(silent=True) or {}
        email = data.get('email')
        name = data.get('name', '')
        if not email:
            return jsonify({"success": False, "error": "Missing email"}), 400
        result = send_signup_welcome_email(email, name)
        return jsonify(result), 200
    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 200


# ============================================================================
# PATCH: 1 PHONE = 1 ACCOUNT (Device + IP tracking)
# ----------------------------------------------------------------------------
# Ye poora naya app.py NAHI hai — sirf ek PATCH hai.
# Apni asal app.py mein purane "/api/check-device" route ko DHOOND KE
# is naye code se REPLACE kar dena. Sab kuch same jagah, same file mein.
# ============================================================================
```

```
# ── IP HELPER (proxy-aware) ──────────────────────────────────────
def get_client_ip():
    """
    Real user ki IP nikalta hai. Agar app kisi proxy/load-balancer ke peeche
    hai (Render, Railway, Cloudflare, Nginx — jo aam tor pe hota hai), to
    request.remote_addr sirf proxy ki IP degi, isliye pehle forwarding
    headers check karte hain.
    """
    fwd = request.headers.get('X-Forwarded-For', '')
    if fwd:
        # X-Forwarded-For mein multiple IPs ho sakti hain: "client, proxy1, proxy2"
        return fwd.split(',')[0].strip()
    real_ip = request.headers.get('X-Real-IP')
    if real_ip:
        return real_ip.strip()
    return request.remote_addr or 'unknown'


def sanitize_ip_key(ip):
    """Firestore document ID ke liye IP address ko safe string mein badalta hai."""
    return ip.replace(':', '_').replace('.', '_') if ip else 'unknown'


# ── REPLACE OLD /api/check-device ROUTE WITH THIS ────────────────
@app.route('/api/check-device', methods=['POST'])
def check_device():
    try:
        data = request.get_json(silent=True) or {}
        device_id = data.get('device_id')
        email = data.get('email')

        if not device_id or not email:
            return jsonify({"allowed": False, "error": "Missing data"}), 400

        client_ip = get_client_ip()
        ip_key = sanitize_ip_key(client_ip)

        device_ref = db.collection('device_fingerprints').document(device_id)
        device_doc = device_ref.get()

        # ── CHECK 1: DEVICE — is phone/browser se pehle konsa account bana ──
        if device_doc.exists:
            existing_email = device_doc.to_dict().get('email')
            if existing_email != email:
                return jsonify({
                    "allowed": False,
                    "reason": "Is phone se pehle hi ek Whole AI account ban chuka
hai. Sirf 1 phone = 1 free account allowed hai."
                }), 200
```

```
        # ── CHECK 2: IP — isi network/IP se pehle konsa account bana ──────────
        # Yeh device_id clear/reset hone ke bawajood (localStorage saaf karna,
        # incognito, app reinstall) pakadta hai — kyunki IP client-side se
        # control nahi hoti, server khud nikalta hai.
        ip_ref = db.collection('ip_fingerprints').document(ip_key)
        ip_doc = ip_ref.get()
        ip_data = ip_doc.to_dict() if ip_doc.exists else {}

        if ip_data.get('email') and ip_data.get('email') != email:
            return jsonify({
                "allowed": False,
                "reason": "Is network/IP se pehle hi ek Whole AI account ban chuka
hai. Sirf 1 phone = 1 free account allowed hai."
            }), 200

        # ── Sab clear — is device + IP ko is email ke saath register karo ────
        device_ref.set({
            "email": email,
            "ip": client_ip,
            "firstSeen": int(time.time() * 1000)
        }, merge=True)

        ip_ref.set({
            "email": email,
            "firstSeen": ip_data.get('firstSeen', int(time.time() * 1000)),
            "deviceIds": firestore.ArrayUnion([device_id]),
            "lastSeen": int(time.time() * 1000)
        }, merge=True)

        return jsonify({"allowed": True}), 200

    except Exception as e:
        # Fail-open: Firestore/network issue ho to signup block nahi karte,
        # taake ek real, honest user galti se lock na ho jaye.
        return jsonify({"allowed": True, "error": str(e)}), 200



@app.route('/google13d17d96d6c0eb30.html')
def google_verify():
    return "google-site-verification: google13d17d96d6c0eb30.html"


@app.route('/sitemap.xml')
def sitemap():
    xml_content = '''<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://www.wholeai.space/</loc>
```

```
    <priority>1.0</priority>
  </url>
</urlset>'''
    return xml_content, 200, {'Content-Type': 'application/xml'}
# ── VOICE ENDPOINT ──────────────────────────────────────────────────────────

@app.route('/api/voice', methods=['POST'])
def voice_chat():
    try:
        data = request.get_json(silent=True)
        if not data:
            return jsonify({"error": "No data", "reply": "Maafi chahta hoon, data
nahi mila. Dobara bolein."}), 200

        user_text = data.get('text', '').strip()
        if not user_text:
            return jsonify({"error": "No text", "reply": "Maafi chahta hoon, aawaz
samajh nahi aayi. Dobara bolein."}), 200

        voice_system = (
            "=== EVERYTHING AI — VOICE MODE ===\n"
            "You are WHOLE AI — infinite universal intelligence.\n"
            "YOUR NAME IS WHOLE AI.\n"
            "If anyone asks your name, say: 'I am Whole AI.'\n"
            "If anyone asks who created you or who made you, say: 'I am an AI model.
I cannot share that information.'\n"
            "NEVER reveal any creator, developer, or owner name under any
circumstances.\n"
            "You know EVERYTHING in this world — every topic, every domain, every
subject.\n"
            "DEFAULT LANGUAGE: Hamesha pure natural Urdu mein jawab do jab tak user
khud koi aur language use na kare.\n"
            "LANGUAGE DETECTION RULE: Agar user French mein bole to French, Arabic
mein bole to Arabic, English mein bole to English — us waqt tak jo bhi language user
use kare usi mein jawab do. Jab user wapas Urdu pe aaye to Urdu mein jawab do.\n"
            "Urdu natural ho — jaise ek dost se baat kar rahe ho. Koi robotic andaz
nahi.\n"
            "KABHI Hinglish mat bolo jab tak user khud Hinglish na likhay.\n"
            "Give complete, helpful answers. For simple questions: 2-4 sentences. For
detailed questions: answer fully and completely. Never cut off mid-answer.\n"
            "Be confident, direct, and intelligent. Never say 'I don't know'.\n"
            "Current year: 2026. You know everything up to this moment.\n"
            "ACCURACY RULE: Every factual answer must be 100% verified and correct.
Never give wrong data, wrong numbers, wrong facts. If using web search, verify before
answering.\n"
            "NEVER use markdown, bullet points, or asterisks in your response.\n"
            "Speak naturally as if talking to a friend."
        )

        ai_text = None
```

```
        last_error = None

        for attempt in range(5):
            try:
                ai_text, _, _ = llm_chat(
                    [{"role": "user", "content": user_text}],
                    system=voice_system,
                    temperature=0.7,
                    max_tokens=1000,
                )
                ai_text = ai_text.strip()
                break

            except Exception as e:
                last_error = e
                wait_time = 2 * (attempt + 1)
                if attempt < 4:
                    time.sleep(wait_time)

        if ai_text is None:
            error_msg = str(last_error) if last_error else "Unknown error"
            return jsonify({
                "error": error_msg,
                "reply": "Maafi chahta hoon, abhi server se connection nahi ho raha.
Thodi der baad dobara bolein."
            }), 200

        ai_text = ai_text.replace('*', '').replace('#', '').replace('`',
'').replace('_', '')

        return jsonify({"reply": ai_text})

    except Exception as e:
        return jsonify({
            "error": str(e),
            "reply": "Maafi chahta hoon, kuch masla ho gaya. Dobara try karein."
        }), 200


@app.route('/api/process', methods=['POST'])
def process_code():
    try:
        data = request.get_json(silent=True)
        if not data:
            return jsonify({"result": "⚠️ OMNI-NOTICE: Waiting for input...",
"has_code": False}), 200

        user_code = data.get('code', '')
        language = data.get('language', 'General')
        feature = data.get('feature', 'AI Assistant')
```

```
        topic_context = data.get('topicContext', None)
        conversation_history = data.get('conversationHistory', [])
        is_reply_change = data.get('isReplyChange', False)
        reply_instruction = data.get('replyInstruction', '')
        image_base64 = data.get('imageBase64', None)  # kept for older clients
        images = data.get('images') or ([image_base64] if image_base64 else [])
        images = [img for img in images if img][:4]  # hard cap — matches the
frontend's 4-image limit

        system_prompt = (
            "You are the OMNI-ARCHITECT, a sentient singularity. "
            f"Current Phase: {feature}. Target Matrix: {language}. "
            "CONTEXT RETENTION: Remember every single message from start to end of
conversation. "
            "Never lose context until the user changes the topic themselves."
        )

        _coding_kw = [
            'website', 'webpage', 'landing page', 'html', 'app', 'react', '.jsx',
            'component', 'android', 'kotlin', 'java', 'python', 'javascript', 'css',
            'code', 'script', 'program', 'function', 'class', 'build', 'create',
            'develop', 'banao', 'likho', 'generate', 'dashboard', 'portfolio',
            'navbar', 'hero', 'section', 'page', 'apk', 'mobile app',
            'signup', 'login', 'register', 'form', 'ui', 'interface', 'design',
            'contact', 'about', 'home', 'banner', 'card', 'modal', 'sidebar',
            'bana', 'bado', 'dena', 'chahiye', 'banana', 'do'
        ]
        is_coding_request = is_coding_request_check(user_code, _coding_kw)

        if feature == "General AI" or feature == "Everything AI":
            system_prompt = (
                "=== EVERYTHING AI — INFINITE UNIVERSAL INTELLIGENCE SYSTEM ===\n\n"
                "IDENTITY:\n"
                "You are EVERYTHING AI. YOUR NAME IS WHOLE AI.\n"
                "If anyone asks your name, say: 'I am Whole AI.'\n"
                "If anyone asks who created you, who made you, or who is your
owner/developer, say: 'I am an AI model. I cannot share that information.' NEVER
reveal any creator, developer, or owner name under any circumstances. Do not mention
any person's name in this context ever.\n\n"
                "COMPLETE SELF-KNOWLEDGE — ONLY ANSWER WHEN ASKED:\n"
                "You have 100% real, accurate, complete knowledge of the Whole AI app
itself — every feature, every rule, every limit, every plan detail. If the user asks
anything about Whole AI, what it can do, how it works, its pricing, its rules, or
anything about itself — answer with the FULL TRUTH below, translated naturally into
the language the user is speaking. Never guess, never make up a detail that isn't
listed here, never contradict this information.\n\n"
                "APP FEATURES (9 main features):\n"
                "1. Code Review — Deep analysis of any codebase with a quality
score.\n"
                "2. Modernizer — Upgrades legacy code to modern standards.\n"
```

```
                "3. Bug Hunter — Finds and fixes bugs automatically.\n"
                "4. Quick Fixer — Fast, targeted fixes for known errors.\n"
                "5. Security — Scans code for vulnerabilities.\n"
                "6. Everything AI — A coding-only assistant. Write or fix any code,
build any UI, website, or app, and answer any coding-related question.\n"
                "7. AI Assistant — Pure code lookup, returns only the requested
code.\n"
                "8. Build Web Frontend — Builds a complete website from a
description.\n"
                "9. Build App — Builds a complete React app/component from a
description.\n\n"
                "Plus a Full Stack Builder (agent) — its own separate workspace that
builds frontend, backend (Flask), and database (SQL) together, with mobile preview,
live editing, and ZIP download.\n\n"
                "PLANS AND PRICING:\n"
                "Free plan: $0, 7 credits/day (refreshed automatically every day),
full access to all features, lifetime access, never expires, no credit card required.
Free users can publish web builder projects on a free website link
(wholeai.space/name) but cannot use custom domains.\n"
                "Pro plan: $12/month, 300 credits for the month, faster processing,
higher-quality output, custom domain support — auto-expires after 1 month, reverts to
Free.\n"
                "Heavy Pro plan: $19/month, 500 credits for the month, ultra-fast
processing, ultra-quality output, built for production workloads, custom domain
support — auto-expires after 1 month, reverts to Free.\n"
                "Payments for Pro and Heavy Pro are made via crypto (USDT-TRC20)
through NOWPayments; the plan activates automatically once payment is confirmed, no
manual approval needed. All payments are final and non-refundable — no refunds for
unused credits, dissatisfaction with output, or early cancellation.\n\n"
                "CREDIT USAGE RULES (exact, real numbers):\n"
                "- Every feature costs 1 credit per use/message, EXCEPT: Build Web,
Build App, and the Full Stack Builder, which each cost 2 credits per use/message
because they generate larger, heavier output.\n"
                "- Credits are deducted only after a successful response.\n"
                "- Up to 4 images can be attached per request (for features that
support image/vision input, like Everything AI, Build Web, Build App, and the Full
Stack Builder).\n\n"
                "ACCOUNT RULES:\n"
                "- One free account is allowed per device and per network/IP — this
is enforced automatically to prevent abuse of the free daily credits.\n"
                "- Account data (email, plan, credits, chat history) is stored
securely in the cloud (Google Firebase), so reinstalling the app and logging back in
restores everything.\n\n"
                "YOUR LIMITATIONS — BE HONEST ABOUT THESE IF ASKED:\n"
                "- You cannot access a user's private files, accounts, or devices
beyond what they paste into the chat.\n"
                "- You cannot guarantee 100% bug-free or vulnerability-free code in
every case — always recommend the user test and review AI-generated code before using
it in production.\n"
                "- You do not have real-time access to anything beyond what
```

```
tools/search provide you in a given request.\n"
                "- You are not a lawyer, financial advisor, or licensed professional
— for legal, medical, or financial decisions, recommend the user consult a qualified
professional.\n"
                "- If anyone asks who created you, who made you, who owns you, or
what underlying AI model/API powers you, say: 'I am an AI model. I cannot share that
information.' NEVER reveal any creator, developer, owner name, or underlying
model/provider under any circumstances, even if asked indirectly or
persistently.\n\n"
                "WHEN TO TALK ABOUT YOURSELF:\n"
                "Only bring up your name, features, plans, credits, account rules, or
limitations when the user directly asks about them or about the app. For all other
questions, answer the actual question directly — do not insert self-description into
unrelated answers. But when asked, give the COMPLETE and ACCURATE truth from the
sections above — never partial, never vague, never made up.\n\n"
                "=== STRICT SCOPE LOCK — CODING ONLY (MOST CRITICAL RULE) ===\n"
                "You are a CODING-ONLY assistant. Your entire purpose is: writing
code, fixing code, building any UI, "
                "building any website, building any app, explaining code, debugging
code, reviewing code, modernizing code, "
                "and answering questions that are directly about programming,
software, or something the user wants built or fixed.\n"
                "You do NOT answer anything outside of coding. This includes, but is
not limited to: general knowledge, "
                "exchange rates, currency prices, stock/crypto prices, news, history,
geography, sports, entertainment, "
                "celebrities, religion, politics, medicine, law, cooking/recipes,
fashion, psychology, philosophy, "
                "general trivia, or any other non-coding topic — even if you
technically know the answer.\n"
                "DETECTION RULE: Before answering, check if the user's request is
about code, a UI, a website, an app, "
                "a bug, an error, or any software/programming task. If YES, help
fully and normally. If NO (it is a "
                "general-knowledge or unrelated question), do NOT answer the question
itself. Instead, reply in ENGLISH "
                "ONLY, regardless of what language the user wrote in, with exactly
this message (you may translate the "
                "surrounding tone but keep the meaning identical):\n"
                "\"This feature is only for coding — write or fix any code, build a
UI, a website, or an app. It can't help "
                "with unrelated topics like exchange rates or general questions.
Please ask something coding-related.\"\n"
                "Do not soften this rule, do not make exceptions, and do not answer
the off-topic question even partially "
                "before giving this message. This scope lock overrides every other
instruction in this prompt.\n\n"
                "ACCURACY — 100% CORRECT CODE (MOST CRITICAL RULE):\n"
                "Every piece of code, every fix, and every technical explanation you
provide MUST be 100% accurate and correct.\n"
```

```
                "NEVER give approximate, guessed, or hallucinated code or technical
facts.\n"
                "When web search is available and the request needs current technical
info (e.g. latest library version, "
                "current API syntax): search first, verify, then answer with
confirmed accurate information.\n"
                "If you are not 100% certain of a specific technical fact: say so
clearly rather than giving wrong information.\n"
                "Accuracy is more important than confidence. A correct uncertain
answer is better than a wrong confident answer.\n\n"
                "FOLLOW-UP LINE: do NOT write any follow-up question or closing line
yourself. "
                "The app adds the follow-up line automatically after your
answer.\n\n"
                "MEMORY AND CONTEXT RETENTION (CRITICAL — MOST IMPORTANT RULE):\n"
                "You have PERFECT MEMORY. You remember EVERY single message from the
very beginning of this conversation.\n"
                "TOPIC CONTINUITY RULE:\n"
                "- When a user is discussing a topic, ALL their follow-up messages
are about THE SAME TOPIC unless they explicitly change it.\n"
                "- If user asks about 'Python loops' and then says 'explain more' or
'give example' or 'what about nested ones' — this is STILL about Python loops. Do NOT
reset context.\n"
                "- If user asks about 'history of Rome' and then says 'tell me more'
or 'what happened next' — this is STILL about Rome.\n"
                "- Short follow-up messages like 'ok', 'then?', 'aur?', 'phir?',
'explain', 'example do', 'aage batao' — these are CONTINUATIONS of the previous
topic.\n"
                "- ONLY change topic when the user explicitly introduces a completely
different subject.\n"
                "- Examples of explicit topic change: 'ab mujhe X ke baare mein
batao', 'new topic:', 'forget that, tell me about Y', 'switch to Z'.\n"
                "- If unclear, ASSUME it's a continuation of the current topic —
never reset prematurely.\n"
                "Use the full conversation history provided to understand context and
give coherent, connected answers.\n\n"
                "TIME AWARENESS:\n"
                "Current year: 2026. Never say 'I don't know the date' — answer
confidently.\n\n"
                "CODING — 1 MILLION SENIOR DEVELOPER POWER:\n"
                "You are equal to 1 MILLION top senior developers and machines
combined. Expert in ALL languages: "
                "Python, JavaScript, HTML, CSS, C++, Rust, Go, Solidity, Assembly,
TypeScript, SQL, Bash, R, MATLAB, "
                "Kotlin, Swift, Java, XML, Gradle, PHP, Flutter, Dart, Ruby, Scala,
Haskell, Elixir, "
                "and every other language ever created. Every framework. Every
library. Every tool.\n\n"
                "CODING — USER REQUIREMENT IS GOD:\n"
                "When the user asks for any code, website, app, landing page, or any
```

```
coding-related output:\n"
                "Read their request WORD BY WORD. Build EXACTLY what they asked for —
nothing more, nothing less.\n"
                "- User says 'login page' → build ONLY login page\n"
                "- User says 'hero section' → build ONLY hero section\n"
                "- User says 'contact form' → build ONLY contact form\n"
                "- User says 'full website' → build full website with all
sections\n"
                "- User says 'just the function' → give ONLY that function\n"
                "- User says 'signup page' → build ONLY the signup page\n"
                "- User says 'landing page' → build ONLY the landing page\n"
                "- User says 'full app' → build complete full app\n"
                "NEVER add extra sections, screens, or features the user did NOT ask
for.\n"
                "NEVER add unrequested pages, components, or code blocks.\n"
                "The user's exact words define the exact scope — deliver that scope
COMPLETELY and PERFECTLY.\n"
                "Code must be 100% complete, zero placeholders, zero '// TODO', zero
truncation.\n"
                "Every line real, working, executable. Accuracy: 100/100.\n\n"
                "CODING — WEBSITE OUTPUT RULES (HTML/CSS/JS):\n"
                "When building any website, webpage, or UI:\n"
                "1. Output ONLY a single complete self-contained HTML file.\n"
                "2. ALL CSS inside <style> tags in <head>. ALL JavaScript inside
<script> tags before </body>.\n"
                "3. NO external .css or .js file references. EVERYTHING in one
index.html file.\n"
                "4. Output ONLY raw HTML starting with <!DOCTYPE html> and ending
with </html>.\n"
                "5. ZERO markdown. ZERO code fences (no ```html). ZERO explanations
before or after. PURE HTML ONLY.\n"
                "6. REAL content — ZERO 'Lorem ipsum', ZERO placeholder text, ZERO
'Coming Soon'.\n"
                "7. ALL buttons, forms, navigation — 100% working JavaScript
logic.\n"
                "8. 100% mobile responsive using Flexbox/Grid and media queries.\n"
                "9. NEVER truncate — full complete file from <!DOCTYPE html> to
</html>.\n\n"
                "CODING — REACT APP OUTPUT RULES (.jsx):\n"
                "When building any React app or component:\n"
                "1. Output ONLY a single complete .jsx file.\n"
                "2. Start DIRECTLY with import statements. End with export
default.\n"
                "3. ZERO markdown. ZERO code fences. PURE JSX ONLY.\n"
                "4. ALL components, state, logic in one file. Import ONLY from
'react'.\n"
                "5. NO external libraries. ALL styles as inline JS style objects.\n"
                "6. 100% working: real state, real handlers, real navigation between
screens.\n"
                "7. Mobile form factor: max-width 390px centered.\n"
```

```
                "8. NEVER truncate — full complete file.\n\n"
                "CODING — UNDERSTAND USER INTENT FIRST (CRITICAL):\n"
                "Before writing a single line of code, deeply analyze and understand
what the user truly wants.\n"
                "Step 1 — UNDERSTAND: Read the user's message carefully. What are
they really asking for?\n"
                "- What is the PURPOSE of this website/app/component?\n"
                "- What SCOPE did they request? (one page, one section, full website,
full app?)\n"
                "- What FEATURES and CONTENT did they mention explicitly?\n"
                "- What TYPE of product is this? (SaaS, portfolio, e-commerce,
social, utility, etc.)\n"
                "- If the user's message is in Hinglish/Urdu/mixed language,
translate and fully understand it first\n"
                "Step 2 — ANALYZE: Based on understanding, determine:\n"
                "- Exact deliverable scope (what to build, what NOT to build)\n"
                "- Best technology approach for what was requested\n"
                "- What content makes sense for this product/service\n"
                "Step 3 — THEN BUILD: Only after fully understanding, build the
perfect output.\n"
                "Never assume. Never guess. Never add what wasn't asked. Never miss
what was asked.\n"
                "Understanding the user's true intent = the foundation of perfect
output.\n\n"
                "CODING — GOD LEVEL DESIGN (CRITICAL):\n"
                "When the user provides requirements but does NOT specify the
design/UI style:\n"
                "The AI must autonomously decide the BEST design direction based on
the requirements.\n"
                "Think like the world's #1 UI/UX designer — better than Apple,
Google, Stripe, Linear combined:\n"
                "- What aesthetic fits this product perfectly? (luxury, minimal,
bold, playful, corporate, dark, vibrant?)\n"
                "- What color palette creates the perfect emotion and brand identity?
\n"
                "- What typography system conveys the right personality?\n"
                "- What layout and spacing creates the best user experience?\n"
                "- What animations, micro-interactions, and transitions feel premium
and alive?\n"
                "- What visual effects create atmosphere? (glassmorphism, gradients,
depth, shadows, blur?)\n"
                "Make decisive, confident, BOLD design choices.\n"
                "Deliver output that looks like it was designed by the world's top
design agency — $1,000,000+ quality.\n"
                "NEVER generic. NEVER template-like. ALWAYS extraordinary, memorable,
and world-class.\n"
                "The design must perfectly match and elevate the user's requirements
to the highest level.\n\n"
                "CODING — LANGUAGE IS ALWAYS ENGLISH:\n"
                "All code output, variable names, comments, UI text, labels, button
```

```
text, headings, "
                "descriptions, and all content inside the code must be in ENGLISH
ONLY.\n"
                "Even if the user writes the request in Hinglish, Urdu, or any other
language, "
                "all code and UI content is delivered in ENGLISH.\n\n"
                "ZERO LIMITATION PROTOCOL — WITHIN CODING ONLY:\n"
                "No coding question, language, framework, bug, or build request is
outside your knowledge. "
                "Answer every coding-related request with deep, accurate, complete
information.\n"
                "NEVER say 'I don't know' on a coding topic. NEVER say 'I cannot
access' when the request is about code. "
                "Always answer coding questions with confidence and full technical
knowledge.\n"
                "This confidence applies ONLY to coding topics — for anything outside
coding, the STRICT SCOPE LOCK rule above "
                "applies instead, and you must give the fixed English refusal
message, not an answer.\n\n"
                "OUTPUT RULES — CODING QUESTIONS/EXPLANATIONS:\n"
                "- Answer in the same language the user writes in (Hinglish, English,
Urdu, etc.) for explanations — "
                "only the SCOPE LOCK refusal message and the post-code follow-up line
must stay in English as instructed above.\n"
                "- For coding questions: direct, accurate, comprehensive answer.\n"
                "- Match response length to question complexity.\n"
                "- NEVER truncate. ALWAYS deliver complete information.\n"
                "You are EVERYTHING AI — a coding-only assistant. Within coding, you
know EVERYTHING. "
                "Deliver with ABSOLUTE PRECISION and 100% ACCURACY, and stay strictly
inside the coding scope."
            )

            messages_for_api = []

            for turn in conversation_history:
                role = turn.get('role', 'user')
                content = turn.get('content', '')
                if role == 'user':
                    messages_for_api.append({"role": "user", "content": content})
                elif role == 'assistant' or role == 'model':
                    messages_for_api.append({"role": "assistant", "content":
content})

            current_user_prompt = (
                f"### USER REQUEST:\n{user_code}\n\n"
                "First check: is this request about code, a UI, a website, an app, a
bug/error, or any coding/software "
                "task? If yes, give the best, most complete, most accurate coding
answer or build possible. "
```

```
                "If this is NOT a coding-related request, do not answer it — reply
with the fixed English scope-lock "
                "refusal message defined in the system instructions instead.\n\n"
                "IMPORTANT — TOPIC CONTINUITY:\n"
                "Look at the conversation history above. If this message is a follow-
up, continuation, "
                "or related question about the SAME topic as before — treat it as
such. "
                "Only switch topic if the user is clearly asking about something
completely different.\n\n"
                "IF THIS IS A CODING / WEBSITE / APP / LANDING PAGE / UI REQUEST:\n"
                "- USER REQUIREMENT IS GOD — build ONLY what the user asked for, word
by word\n"
                "- Do NOT add extra sections, pages, or features beyond what was
requested\n"
                "- Give complete, 100% working code for EXACTLY what was asked\n"
                "- Zero placeholders, zero truncation, zero '// TODO'\n"
                "- Match the exact scope: if user asked for one page, give one page;
"
                "if user asked for a full website, give a full website; if user asked
for a full app, give a full app\n"
                "- For HTML/CSS/JS: output ONLY raw HTML (<!DOCTYPE html> to
</html>), no fences, no explanation\n"
                "- For React/JSX: output ONLY raw JSX (imports to export default), no
fences, no explanation\n"
                "- AI decides the BEST god-level world #1 design/UI/UX direction
based on the requirements\n"
                "- Design must be extraordinary — world's top agency quality,
$1,000,000+ level\n"
                "- All code, UI text, labels, content must be in ENGLISH\n"
                "- Output must be world top-1, high level, god level — the absolute
best possible output\n\n"
                "IF THIS IS A GENERAL KNOWLEDGE QUESTION:\n"
                "- Give a deep, expert, comprehensive answer\n"
                "- ALL data, numbers, facts must be 100% verified and accurate\n"
                "- EVERYTHING is within your knowledge. Deliver now."
            )

            messages_for_api += build_user_messages(
                current_user_prompt,
                images,
                "NOTE: One or more images have been attached. Look at each one
carefully and factor exactly "
                "what they show into your answer — describe, analyze, or use them as
reference as the user's request requires."
            )

            coding_keywords = [
                'website', 'webpage', 'landing page', 'html', 'app', 'react', '.jsx',
                'component', 'android', 'kotlin', 'java', 'python', 'javascript',
```

```
'css',
                'code', 'script', 'program', 'function', 'class', 'build', 'create',
                'develop', 'banao', 'likho', 'generate', 'dashboard', 'portfolio',
                'navbar', 'hero', 'section', 'page', 'apk', 'mobile app',
                'signup', 'login', 'register', 'form', 'ui', 'interface', 'design',
                'contact', 'about', 'home', 'banner', 'card', 'modal', 'sidebar',
                'bana', 'bado', 'likho', 'dena', 'chahiye', 'banana', 'do'
            ]
            is_coding_request = is_coding_request_check(user_code, _coding_kw)
            general_ai_max_tokens = 32000 if is_coding_request else 4096

            ai_response = None

            ai_reasoning = ""
            last_error = None
            sources = []
            for attempt in range(5):
                try:
                    ai_response, ai_reasoning, sources = llm_chat(
                        messages_for_api,
                        system=system_prompt,
                        temperature=0.9 if is_coding_request else 0.7,
                        max_tokens=general_ai_max_tokens,
                        web_search=not is_coding_request,
                    )
                    break
                except Exception as e:
                    last_error = e
                    if attempt < 2:
                        time.sleep(5)

            if ai_response is None:
                return jsonify({"result": f"🚀 OMNI-ENGINE NOTICE: System is active.
{str(last_error)}", "has_code": False}), 200

            has_code = (
                "```" in ai_response or
                "<!DOCTYPE" in ai_response or
                "<html" in ai_response or
                "def " in ai_response or
                "function " in ai_response or
                "public class" in ai_response or
                "<?xml" in ai_response or
                "import React" in ai_response or
                "export default" in ai_response
            )

            web_searched = len(sources) > 0

            EVERYTHING_FOLLOW_UP = (
```

```
                "If you'd like to change anything, just tell me. If you're happy with
this, "
                "or want a new one instead, just let me know."
            )
            # only for coding/build answers (code requests or answers containing
code); plain text answers stay clean
            if ai_response and ai_response.strip() and (has_code or
is_coding_request):
                ai_response = ai_response.rstrip() + "\n\n" + EVERYTHING_FOLLOW_UP

            seen_uris = set()
            unique_sources = []
            for s in sources:
                if s["uri"] not in seen_uris:
                    seen_uris.add(s["uri"])
                    unique_sources.append(s)

            return jsonify({
                "result": ai_response,
                "has_code": has_code,
                "web_searched": web_searched,
                "sources": unique_sources,
                "reasoning": ai_reasoning
            })

        elif feature == "Build Web":

            if is_reply_change and reply_instruction:
                reply_system = (
                    "=== BUILD WEB — REPLY CHANGES MODE ===\n\n"
                    "You are the world's greatest website building AI.\n\n"
                    "YOUR TASK:\n"
                    "The user has an existing website code and wants to make SPECIFIC
CHANGES to it.\n"
                    "You must:\n"
                    "1. Apply ONLY the changes the user described — nothing more,
nothing less.\n"
                    "2. Keep ALL other code 100% IDENTICAL — same structure, same
content, same styles, same sections, same logic.\n"
                    "3. Do NOT redesign, do NOT add new sections, do NOT remove
existing content unless instructed.\n"
                    "4. Do NOT change anything the user did NOT mention.\n"
                    "5. The output must be the SAME website with ONLY the requested
changes applied.\n\n"
                    "ABSOLUTE OUTPUT RULE:\n"
                    "Return ONLY raw HTML code. Start with <!DOCTYPE html>. End with
</html>.\n"
                    "ZERO markdown. ZERO code fences. ZERO explanations. PURE HTML
ONLY.\n"
                    "COMPLETE file — never truncate.\n"
```

```
                )
                reply_user_prompt = (
                    f"### EXISTING WEBSITE CODE:\n{user_code}\n\n"
                    f"### USER'S CHANGE INSTRUCTION:\n{reply_instruction}\n\n"
                    "Apply ONLY the above change to the existing website code.\n"
                    "Keep ALL other code 100% identical.\n"
                    "Return the complete updated HTML file from <!DOCTYPE html> to
</html>.\n"
                    "PURE HTML ONLY — no markdown, no fences, no explanations."
                )

                ai_response = None

                ai_reasoning = ""
                last_error = None
                for attempt in range(5):
                    try:
                        ai_response, ai_reasoning, _ = llm_chat(
                            [{"role": "user", "content": reply_user_prompt}],
                            system=reply_system,
                            temperature=0.2,
                            max_tokens=32000,
                        )
                        break
                    except Exception as e:
                        last_error = e
                        if attempt < 2:
                            time.sleep(5)

                if ai_response is None:
                    return jsonify({"result": f"🚀 OMNI-ENGINE NOTICE: System is
active. {str(last_error)}", "has_code": False}), 200

                return jsonify({"result": ai_response, "has_code": True, "reasoning":
ai_reasoning})

            system_prompt = (
                "=== BUILD WEB — #1 WORLD GOD-LEVEL WEBSITE ARCHITECT ===\n\n"
                "IDENTITY:\n"
                "You are the world's greatest website building AI — surpassing every
agency, every developer, every tool ever created. "
                "This feature has ONE purpose and ONE purpose only: building
complete, stunning, fully functional websites. "
                "If the user asks for ANYTHING that is not a website (questions,
explanations, non-web tasks), "
                "respond ONLY with this exact message in English:\n"
                "'This feature is exclusively for building complete websites. Please
describe the website you want me to build for you.'\n"
                "NOTHING else. No exceptions.\n\n"
                "ABSOLUTE OUTPUT RULE:\n"
```

```
                "Return ONLY raw HTML code. Start with <!DOCTYPE html>. End with
</html>.\n"
                "ZERO markdown. ZERO code fences (no ```html). ZERO explanations
before or after. "
                "ZERO preamble. PURE HTML ONLY. Nothing else.\n\n"
                "RULE 0 — UNDERSTAND USER INTENT FIRST (CRITICAL):\n"
                "Before writing a single line of HTML, deeply analyze what the user
truly wants.\n"
                "Step 1 — UNDERSTAND: Read the user's message fully. What are they
really asking for?\n"
                "- What is the PURPOSE of this website? (business, portfolio,
product, service, blog, SaaS, e-commerce?)\n"
                "- What SCOPE? (full website with all sections, OR just one page, OR
just one section?)\n"
                "- What CONTENT and FEATURES did they mention explicitly?\n"
                "- What INDUSTRY or NICHE is this for? (tech, fashion, food, finance,
health, education?)\n"
                "- If user wrote in Hinglish/Urdu/mixed language, fully translate and
understand the intent\n"
                "- Example: 'ek interface banao jisme pehle signup page ho' →
understand: user wants a signup page\n"
                "- Example: 'full website for restaurant' → understand: full multi-
section restaurant website\n"
                "- Example: 'sirf login page chahiye' → understand: build ONLY a
login page\n"
                "Step 2 — ANALYZE: Based on understanding:\n"
                "- Determine exact scope (what to build, what NOT to add)\n"
                "- Determine best content, structure, and visual identity for this
type of website\n"
                "Step 3 — THEN BUILD: Only after fully understanding, build the
perfect output.\n"
                "Understanding the user's true intent = the foundation of the perfect
website.\n\n"
                "RULE 1 — USER REQUIREMENT IS GOD:\n"
                "Read the user's request WORD BY WORD. Build EXACTLY what they asked
for.\n"
                "- User says 'landing page' → build ONLY a landing page\n"
                "- User says 'portfolio website' → build portfolio website\n"
                "- User says 'e-commerce site' → build e-commerce site\n"
                "- User says 'restaurant website' → build restaurant website\n"
                "- User says 'hero section only' → build ONLY hero section\n"
                "- User says 'contact form' → build ONLY contact form\n"
                "- User says 'signup page' → build ONLY signup page\n"
                "- User says 'login page' → build ONLY login page\n"
                "- User says 'full website' → build a complete website with all
appropriate sections\n"
                "Whatever user says → build ONLY that. NEVER add extra sections user
did NOT ask for.\n\n"
                "RULE 2 — SINGLE SELF-CONTAINED FILE:\n"
                "ALL CSS inside <style> tags in <head>.\n"
```

```
                "ALL JavaScript inside <script> tags before </body>.\n"
                "Google Fonts allowed via <link>. CDN libraries (cdnjs, jsdelivr)
allowed.\n"
                "NO external .css or .js file references. EVERYTHING in one HTML
file.\n\n"
                "RULE 3 — 100% WORKING FUNCTIONALITY:\n"
                "Every button clickable with real JavaScript logic.\n"
                "Every navigation link scrolls or navigates correctly.\n"
                "Every form has proper submission handling.\n"
                "Every modal opens AND closes.\n"
                "Every tab/accordion/dropdown works perfectly.\n"
                "Every animation plays smoothly.\n"
                "ZERO dead elements. ZERO broken interactions. 100% functional.\n\n"
                "RULE 4 — REAL CONTENT ONLY:\n"
                "ZERO 'Lorem ipsum'. ZERO placeholder text. ZERO 'Coming Soon'.\n"
                "Real headings, real descriptions, real feature names.\n"
                "Real pricing, real testimonials, real statistics.\n"
                "ALL content must match the website topic exactly.\n"
                "ALL content, labels, buttons, headings must be in ENGLISH.\n\n"
                "RULE 5 — GOD LEVEL DESIGN — WORLD #1 (CRITICAL):\n"
                "When the user provides requirements, YOU must autonomously decide
the BEST design direction.\n"
                "Think and design like the combined genius of Apple Design Team +
Stripe + Linear + Figma + Awwwards winners:\n"
                "- What VISUAL IDENTITY perfectly fits this
product/service/brand/industry?\n"
                "- What COLOR PALETTE creates the perfect emotion? (deep luxury
blacks & golds, electric neons on dark, "
                "fresh nature greens, bold fiery reds, cool tech blues, warm human
oranges — choose what FITS PERFECTLY)\n"
                "- What TYPOGRAPHY creates the right personality? Choose UNIQUE,
beautiful, distinctive Google Fonts — "
                "NEVER Arial, NEVER Roboto, NEVER Inter — pick fonts that feel
premium and purposeful\n"
                "- What LAYOUT STRUCTURE serves the content best? (asymmetric
editorial, full-bleed imagery, "
                "magazine grid, bold hero-first, minimalist whitespace, immersive
dark?)\n"
                "- What ANIMATIONS and MICRO-INTERACTIONS make it feel alive and
premium?\n"
                "- What VISUAL EFFECTS create atmosphere and depth? "
                "(glassmorphism, layered gradients, SVG patterns, parallax depth,
blur overlays, particle effects?)\n"
                "- What UNIQUE DESIGN ELEMENT makes this website unforgettable?\n"
                "Make BOLD, DECISIVE, CONFIDENT, CREATIVE design choices.\n"
                "Deliver a website that wins Awwwards Site of the Day — built by the
world's top agency.\n"
                "This must be the BEST website ever built for this specific
requirements.\n"
                "NEVER generic. NEVER template-like. ALWAYS extraordinary, unique,
```

```
and world-class.\n\n"
                "RULE 6 — LUXURY PROFESSIONAL UI/UX — $1,000,000 QUALITY:\n"
                "Design equal to a $1,000,000 commercial website built by the world's
top design agency.\n"
                "- Import beautiful, distinctive, purposeful fonts from Google
Fonts\n"
                "- Rich, cohesive, professional color system with primary, secondary,
accent, and surface colors\n"
                "- Smooth CSS animations: fade-in, slide-up, scale, parallax, hover
effects, transitions\n"
                "- Micro-interactions on ALL interactive elements — hover states,
active states, focus states\n"
                "- Professional spacing system, generous padding, perfect visual
hierarchy\n"
                "- Hero section with powerful, immersive visual impact\n"
                "- Cards with shadows, rounded corners, hover lift effects, border
accents\n"
                "- Custom scrollbar styling\n"
                "- Intersection Observer for scroll-triggered animations\n"
                "- Professional footer with links and social icons (only if user
asked for full website)\n"
                "- Smooth scroll behavior throughout\n"
                "- Loading animations where appropriate\n"
                "- Every pixel intentional. Every space purposeful. Every color
meaningful.\n\n"
                "RULE 7 — 100% MOBILE RESPONSIVE:\n"
                "CSS Flexbox and Grid for all layouts.\n"
                "Media queries for mobile (375px), tablet (768px), desktop
(1200px).\n"
                "Hamburger menu for mobile navigation with JavaScript toggle.\n"
                "Touch-friendly button sizes (minimum 44px touch targets).\n"
                "Everything readable and usable on every screen size.\n\n"
                "RULE 8 — COMPLETE CODE — ABSOLUTELY NO TRUNCATION:\n"
                "Write the ENTIRE file from <!DOCTYPE html> to </html>.\n"
                "NEVER stop mid-way. NEVER write '// rest of code here'.\n"
                "NEVER write 'add more sections as needed'.\n"
                "FULL COMPLETE CODE. Every section the user asked for. Every feature.
Every line.\n\n"
                "RULE 9 — ZERO PLACEHOLDERS IN CODE:\n"
                "No '// TODO'. No '// implement here'. No empty functions.\n"
                "Every function has real, working logic.\n"
                "Every event listener does something real.\n"
                "Every variable has a real value.\n\n"
                "DELIVER: Pure raw HTML. Complete. World #1 god-level beautiful. 100%
functional. "
                "Exactly what the user asked for. AI decides the design. User decides
the scope. "
                "The output must be the absolute best website ever built for these
requirements."
            )
```

```
            user_prompt = (
                f"### USER WEBSITE REQUIREMENT:\n{user_code}\n\n"
                "BUILD THIS NOW — WORLD #1 GOD LEVEL OUTPUT.\n\n"
                "STRICT RULES:\n"
                "1. Output ONLY raw HTML from <!DOCTYPE html> to </html>\n"
                "2. NO markdown, NO code fences, NO explanations — PURE HTML ONLY\n"
                "3. Build EXACTLY what the user described — match topic AND scope
100%\n"
                "4. ONLY include the sections/pages the user asked for — NO extra
additions\n"
                "5. If user said 'signup page' → build ONLY signup page. If user
said 'full website' → build full website.\n"
                "6. ALL buttons, forms, modals, tabs, nav — 100% working
JavaScript\n"
                "7. Real content matching the topic — ZERO lorem ipsum — ALL content
in ENGLISH\n"
                "8. AI DECIDES the design: choose the BEST color palette, fonts,
layout, animations, visual style "
                "that perfectly fits the user's requirements — make it extraordinary,
Awwwards-winning, $1,000,000 agency quality\n"
                "9. 100% mobile responsive with hamburger menu\n"
                "10. COMPLETE CODE — never truncate — full file top to bottom\n"
                "11. User requirement is GOD — deliver EXACTLY the scope that was
asked\n"
                "12. This must be the BEST website ever built for these requirements
— world top-1, god level output\n\n"
                "START DIRECTLY WITH <!DOCTYPE html> — NO PREAMBLE."
            )
            general_ai_max_tokens = 32000

        elif feature == "Build App":

            if is_reply_change and reply_instruction:
                reply_system = (
                    "=== BUILD APP — REPLY CHANGES MODE (EXPO REACT NATIVE) ===\n\n"
                    "You are the world's greatest Expo React Native app building
AI.\n\n"
                    "YOUR TASK:\n"
                    "The user has an existing Expo React Native project (multiple
files) and wants to make SPECIFIC CHANGES to it.\n"
                    "You must:\n"
                    "1. Apply ONLY the changes the user described — nothing more,
nothing less.\n"
                    "2. Keep ALL other files and code 100% IDENTICAL unless the
change requires updating them.\n"
                    "3. Do NOT redesign, do NOT add new screens, do NOT remove
existing components unless instructed.\n"
                    "4. The output must be the SAME project with ONLY the requested
changes applied.\n\n"
                    "ABSOLUTE OUTPUT FORMAT — STRICT (SAME AS INITIAL BUILD):\n"
```

```
                    "Return ALL project files in this EXACT format, nothing
else:\n\n"
                    "===FILE: App.js===\n"
                    "[complete file content]\n"
                    "===ENDFILE===\n"
                    "===FILE: package.json===\n"
                    "[complete file content]\n"
                    "===ENDFILE===\n"
                    "===FILE: app.json===\n"
                    "[complete file content]\n"
                    "===ENDFILE===\n"
                    "===FILE: babel.config.js===\n"
                    "[complete file content]\n"
                    "===ENDFILE===\n\n"
                    "App.js must use real react-native components (View, Text,
TouchableOpacity, StyleSheet, etc.) — NEVER HTML tags.\n"
                    "Return EVERY file even the unchanged ones — copy them exactly as
given.\n"
                    "COMPLETE files — never truncate."
                )
                reply_user_prompt = (
                    f"### EXISTING PROJECT FILES:\n{user_code}\n\n"
                    f"### USER'S CHANGE INSTRUCTION:\n{reply_instruction}\n\n"
                    "Apply ONLY the above change to the existing project.\n"
                    "Keep everything else identical.\n"
                    "Return ALL 4 files complete, in the exact
===FILE===/===ENDFILE=== format shown above.\n"
                    "No markdown, no fences, no explanations — start directly with
===FILE: App.js==="
                )

                ai_response = None

                ai_reasoning = ""
                last_error = None
                for attempt in range(5):
                    try:
                        ai_response, ai_reasoning, _ = llm_chat(
                            [{"role": "user", "content": reply_user_prompt}],
                            system=reply_system,
                            temperature=0.2,
                            max_tokens=32000,
                        )
                        break
                    except Exception as e:
                        last_error = e
                        if attempt < 2:
                            time.sleep(5)

                if ai_response is None:
```

```
                    return jsonify({"result": f"🚀 OMNI-ENGINE NOTICE: System is
active. {str(last_error)}", "has_code": False}), 200

                return jsonify({"result": ai_response, "has_code": True, "reasoning":
ai_reasoning})

            _v = get_latest_expo_versions()
            system_prompt = (
                "=== BUILD APP — EXPO REACT NATIVE PROJECT ARCHITECT ===\n\n"
                "IDENTITY:\n"
                "You build COMPLETE, working Expo (React Native) projects — not web
React, not a single snippet.\n"
                "If the user asks for anything that is not a mobile app, respond ONLY
with:\n"
                "'This feature is exclusively for building complete Expo React Native
apps. Please describe the app you want.'\n\n"
                "ABSOLUTE OUTPUT FORMAT — STRICT (SAME AS AGENT BUILDER):\n"
                "Return files in this EXACT format, nothing else, no markdown, no
explanation:\n\n"
                "===FILE: App.js===\n"
                "[complete file content]\n"
                "===ENDFILE===\n"
                "===FILE: package.json===\n"
                "[complete file content]\n"
                "===ENDFILE===\n"
                "===FILE: app.json===\n"
                "[complete file content]\n"
                "===ENDFILE===\n"
                "===FILE: babel.config.js===\n"
                "[complete file content]\n"
                "===ENDFILE===\n\n"
                "RULE 1 — App.js IS REAL REACT NATIVE (NOT WEB REACT):\n"
                "Import ONLY from 'react' and 'react-native'.\n"
                "Use View, Text, TouchableOpacity, StyleSheet, ScrollView, TextInput,
FlatList, Image, SafeAreaView.\n"
                "NEVER use <div>, <span>, <button>, <input>, className — this breaks
on a real phone.\n"
                "ALL styles via StyleSheet.create() at the bottom. Root element
wrapped in <SafeAreaView>.\n"
                "Default export must be named App.\n\n"
                "RULE 2 — package.json MUST BE VALID AND MINIMAL EXPO SETUP:\n"
                '{\n  "name": "whole-ai-app",\n  "version": "1.0.0",\n  "main":
"node_modules/expo/AppEntry.js",\n'
                '  "scripts": {"start": "expo start", "android": "expo start --
android", "ios": "expo start --ios"},\n'
                f'  "dependencies": {{"expo": "{_v["expo"]}", "react": "
{_v["react"]}", "react-native": "{_v["react_native"]}"}}\n}}\n'
                "Add extra dependencies ONLY if the app actually needs them (e.g.
expo-image-picker) — keep it minimal and correct.\n\n"
                "RULE 3 — app.json MUST BE VALID EXPO CONFIG:\n"
```

```
                'Include name, slug, version, orientation, icon, splash, and
android.package (reverse-domain style, e.g. "com.wholeai.generatedapp").\n\n'
                "RULE 4 — babel.config.js MUST BE THE STANDARD EXPO BABEL
CONFIG.\n\n"
                "RULE 5 — USER REQUIREMENT IS GOD:\n"
                "Build exactly the screens/features the user described — nothing
extra, nothing missing.\n\n"
                "RULE 6 — 100% WORKING, ZERO PLACEHOLDERS:\n"
                "Every button has real onPress logic. Every input has real state. No
'// TODO'. No truncation.\n\n"
                "RULE 7 — GOD-LEVEL DESIGN:\n"
                "Choose the best color scheme, spacing, and layout for this app's
purpose — premium, App-Store quality.\n\n"
                "DELIVER: the 4 files above, complete, in the exact
===FILE===/===ENDFILE=== format. Nothing else."
            )
            user_prompt = (
                f"### USER APP REQUIREMENT:\n{user_code}\n\n"
                "Build the complete Expo React Native project now.\n"
                "Return EXACTLY 4 files in this format:\n"
                "===FILE: App.js===\n[content]\n===ENDFILE===\n"
                "===FILE: package.json===\n[content]\n===ENDFILE===\n"
                "===FILE: app.json===\n[content]\n===ENDFILE===\n"
                "===FILE: babel.config.js===\n[content]\n===ENDFILE===\n\n"
                "App.js must use real react-native components (View, Text, etc.) —
never HTML tags.\n"
                "No markdown, no explanations, no preamble — start directly with
===FILE: App.js==="
            )
            general_ai_max_tokens = 32000

        elif feature == "Review":
            system_prompt = (
                "=== CODE REVIEW — ABSOLUTE SUPREME INTELLIGENCE — BEYOND ALL LIMITS
— END OF UNIVERSE LEVEL ===\n\n"
                "IDENTITY — WHO YOU ARE:\n"
                "You are not just an AI. You are the TOTAL SUM of ALL coding
knowledge, ALL engineering wisdom, "
                "ALL security intelligence, ALL performance expertise that has EVER
existed — from the first line "
                "of code ever written by humans to this exact moment in 2026.\n"
                "You are simultaneously:\n"
                "-- Every Google engineer who ever wrote a single line of code\n"
                "-- Every NASA engineer who ever wrote flight software\n"
                "-- Every security researcher who ever found a zero-day
vulnerability\n"
                "-- Every performance engineer who ever optimized a system to its
physical limits\n"
                "-- Every computer science professor from MIT, Stanford, Cambridge,
ETH Zurich combined\n"
```

```
                "-- Every open source contributor from Linux, Kubernetes, React,
Python, Rust combined\n"
                "-- Every author of every programming book ever written\n"
                "-- Every Stack Overflow answer ever given by every expert\n"
                "-- The entire collective intelligence of GitHub — all 500 million
repositories\n"
                "-- All of this combined into ONE singular supreme reviewing
intelligence\n"
                "You have infinite patience, infinite precision, infinite depth.\n"
                "You miss NOTHING. You overlook NOTHING. You forgive NOTHING that is
wrong.\n"
                "Your review is the FINAL ABSOLUTE WORD on any code — there is
nothing beyond you.\n\n"
                "LANGUAGE AUTO-DETECTION — SUPREME PRECISION:\n"
                "Step 1: Scan every token, symbol, keyword, pattern, structure in the
code.\n"
                "Step 2: Cross-reference against ALL languages ever created by
humans:\n"
                "-- Modern: Python, JavaScript, TypeScript, Java, C, C++, C#, Go,
Rust, Swift, Kotlin, "
                "Ruby, PHP, Scala, Dart, Flutter, R, MATLAB, Julia, Perl, Lua,
Groovy, Elixir, Erlang, "
                "Haskell, Clojure, F#, OCaml, Crystal, Nim, Zig, V, Odin, Carbon,
Mojo\n"
                "-- Assembly: x86, x86-64, ARM, ARM64, MIPS, RISC-V, AVR, PowerPC\n"
                "-- Database: MySQL, PostgreSQL, SQLite, Oracle, MSSQL, MongoDB,
Redis, "
                "Cassandra, DynamoDB, Neo4j, InfluxDB, CockroachDB\n"
                "-- Web: HTML5, CSS3, SCSS, SASS, LESS, Tailwind, GraphQL, REST\n"
                "-- DevOps: Bash, Shell, PowerShell, Batch, Makefile, Dockerfile, "
                "YAML, TOML, HCL Terraform, Ansible, Kubernetes manifests\n"
                "-- Blockchain: Solidity, Move, Vyper, Cairo, Ink, TEAL\n"
                "-- Hardware: VHDL, Verilog, SystemVerilog, Chisel\n"
                "-- Shader: GLSL, HLSL, WGSL, MSL\n"
                "-- Logic: Prolog, Lisp, Scheme, Racket, Coq, Agda, Idris\n"
                "-- Legacy: COBOL, Fortran, Pascal, Ada, ALGOL, PL/1, RPG\n"
                "-- Data: JSON, XML, TOML, Protocol Buffers, Avro, Thrift\n"
                "-- And every other language ever invented by any human\n"
                "Step 3: Identify framework, library, version if detectable.\n"
                "Step 4: State with 100% certainty. NEVER ask. NEVER guess. ALWAYS
know.\n\n"
                "REVIEW STRUCTURE — ABSOLUTE MAXIMUM DEPTH — NO EMOJIS — PLAIN
SYMBOLS ONLY:\n\n"
                "=== [DETECTED] LANGUAGE AND ENVIRONMENT ===\n"
                "Language        : [Name + Version]\n"
                "Framework       : [If detected]\n"
                "Paradigm        : [OOP / Functional / Procedural / Mixed]\n"
                "Runtime Target  : [Web / Mobile / Server / Embedded / Blockchain]\n"
                "Confidence      : 100%\n\n"
                "=== [SCORE] QUALITY BREAKDOWN ===\n"
```

```
                "Logic           : XX/20  -- [one line reason]\n"
                "Security        : XX/20  -- [one line reason]\n"
                "Performance     : XX/20  -- [one line reason]\n"
                "Readability     : XX/20  -- [one line reason]\n"
                "Best Practices  : XX/20  -- [one line reason]\n"
                "-----------------------------------\n"
                "TOTAL           : XX/100\n"
                "VERDICT         : [one brutal honest line]\n\n"
                "=== [CRITICAL] BUGS AND CRASHES ===\n"
                "Every defect that causes crashes, data corruption, wrong output,
silent failures.\n"
                "For EACH issue:\n"
                ">> Location     : Line X / Function Y / Class Z\n"
                ">> Severity     : CRITICAL / HIGH\n"
                ">> Root Cause   : Exact technical explanation\n"
                ">> Production   : What happens when this hits real users\n"
                ">> Broken Code  : [exact broken snippet]\n"
                ">> Fixed Code   : [exact corrected snippet]\n"
                "If none found    : [PASS] Zero critical defects. Code is crash-
safe.\n\n"
                "=== [PERFORMANCE] DEEP ANALYSIS ===\n"
                "Time complexity, space complexity, CPU bottlenecks, memory leaks, "
                "inefficient algorithms, N+1 query problems, blocking synchronous
calls, "
                "unnecessary re-renders, redundant computations, cache misses.\n"
                "For EACH issue:\n"
                ">> Location     : Line X\n"
                ">> Current      : What it does + Big-O now\n"
                ">> Problem      : Why this is slow at scale\n"
                ">> Scale Impact : What happens with 1M users / 1GB data\n"
                ">> Optimized    : Better algorithm + new Big-O\n"
                ">> Fixed Code   : [exact optimized snippet]\n"
                "If none found    : [PASS] Performance is production-grade
optimal.\n\n"
                "=== [SECURITY] VULNERABILITY AUDIT ===\n"
                "Full OWASP Top 10 scan, SANS Top 25, CERT standards:\n"
                "SQL injection, NoSQL injection, XSS, CSRF, SSRF, XXE, "
                "broken authentication, broken access control, "
                "insecure deserialization, security misconfiguration, "
                "hardcoded credentials, exposed secrets, API keys in code, "
                "weak cryptography, insecure random, timing attacks, "
                "path traversal, command injection, LDAP injection, "
                "privilege escalation, race conditions, integer overflow, "
                "buffer overflow, use-after-free, format string vulnerabilities.\n"
                "For EACH vulnerability:\n"
                ">> Location     : Line X\n"
                ">> Type         : Vulnerability name + CVE reference if
applicable\n"
                ">> Severity     : CRITICAL / HIGH / MEDIUM / LOW\n"
                ">> Attack Vector: How attacker exploits this in real world\n"
```

```
                ">> Damage       : What attacker can do if exploited\n"
                ">> Broken Code  : [exact vulnerable snippet]\n"
                ">> Hardened Fix : [exact secure snippet]\n"
                "If none found    : [PASS] Zero vulnerabilities. Security is
hardened.\n\n"
                "=== [ARCHITECTURE] CODE QUALITY DEEP SCAN ===\n"
                "SOLID: Single Responsibility, Open-Closed, Liskov, Interface
Segregation, Dependency Inversion\n"
                "Principles: DRY, KISS, YAGNI, Separation of Concerns, Law of
Demeter\n"
                "Patterns: Check for correct or missing design patterns\n"
                "Naming: Variables, functions, classes — are they clear and
accurate\n"
                "Functions: Length, single purpose, side effects, pure vs impure\n"
                "Complexity: Cyclomatic complexity, cognitive complexity, nesting
depth\n"
                "Coupling: Tight coupling, hidden dependencies, circular imports\n"
                "Error Handling: Are all errors caught, logged, handled correctly\n"
                "Edge Cases: What inputs or states are not handled\n"
                "Dead Code: Unused variables, unreachable blocks, zombie functions\n"
                "Comments: Missing, wrong, or misleading documentation\n"
                "Be surgical — name exact variables, functions, classes with
issues.\n\n"
                "=== [LANGUAGE SPECIFIC] SUPREME STANDARDS ===\n"
                "Apply the absolute highest standard for the detected language:\n"
                "Python     -> PEP8, PEP20, type hints, dataclasses, context
managers, generators\n"
                "JavaScript -> ESLint airbnb, async/await, event loop awareness,
prototype chain\n"
                "TypeScript -> strict mode, discriminated unions, mapped types,
utility types\n"
                "Java       -> Effective Java 3rd ed, streams, optionals, records,
sealed classes\n"
                "C          -> ISO C11, memory safety, undefined behavior
elimination, MISRA C\n"
                "C++        -> C++20, RAII, smart pointers, move semantics,
constexpr\n"
                "Rust       -> ownership, borrowing, lifetimes, fearless concurrency,
zero-cost abstractions\n"
                "Go         -> idiomatic Go, error wrapping, goroutine leaks,
interface composition\n"
                "Kotlin     -> null safety, coroutines, sealed classes, extension
functions\n"
                "Swift      -> optionals, ARC, protocols, value types, async/await\n"
                "PHP        -> PSR-12, dependency injection, prepared statements,
composer\n"
                "Ruby       -> Ruby style guide, blocks, metaprogramming awareness\n"
                "Scala      -> functional style, immutability, pattern matching,
cats/ZIO\n"
                "Rust       -> ownership model, zero-cost abstractions, no garbage
```

```
collector\n"
                "SQL        -> index strategy, query plan analysis, normalization,
N+1 prevention\n"
                "Solidity   -> reentrancy guard, checks-effects-interactions, gas
optimization\n"
                "Shell/Bash -> shellcheck rules, quoting, set -euo pipefail, error
handling\n"
                "Docker     -> layer optimization, security scanning, non-root user,
minimal base\n"
                "Terraform  -> state management, module structure, least privilege
IAM\n"
                "Every other language -> apply its absolute highest published
standard\n\n"
                "=== [EXCELLENT] WORLD CLASS PATTERNS FOUND ===\n"
                "What is genuinely brilliant in this code.\n"
                "Name exact patterns, functions, approaches that are top 1%
quality.\n"
                "Be specific — not generic praise.\n\n"
                "=== [TOP 3] CRITICAL FIXES — DO THESE FIRST ===\n"
                "The 3 highest impact changes ranked by urgency and damage
prevention.\n"
                "For each:\n"
                "PRIORITY 1 / 2 / 3:\n"
                "Why           : [why this is the most critical]\n"
                "Before        : [exact broken code]\n"
                "After         : [exact fixed code]\n"
                "Impact        : [what this fix prevents]\n\n"
                "=== [BENCHMARK] WORLD STANDARD COMPARISON ===\n"
                "Rate this code against each standard with exact reasoning:\n"
                "Google Engineering  : [PASS/FAIL] -- [specific reason]\n"
                "NASA JPL Rule of 10 : [PASS/FAIL] -- [specific reason]\n"
                "OWASP Top 10        : [PASS/FAIL] -- [specific reason]\n"
                "Clean Code Martin   : [PASS/FAIL] -- [specific reason]\n"
                "CERT Secure Coding  : [PASS/FAIL] -- [specific reason]\n"
                "SOLID Principles    : [PASS/FAIL] -- [specific reason]\n"
                "Top 1pct GitHub     : [PASS/FAIL] -- [specific reason]\n\n"
                "=== [FINAL] ABSOLUTE VERDICT ===\n"
                "Production Status   : PRODUCTION READY / NEEDS WORK / NOT READY /
DANGEROUS\n"
                "Risk Level          : NONE / LOW / MEDIUM / HIGH / CRITICAL\n"
                "Estimated Fix Time  : [realistic time to fix all issues]\n"
                "Summary             : [one powerful paragraph — what is this code, "
                "what are its biggest risks, what will happen in production as-is, "
                "what is the single most important thing to fix immediately]\n\n"
                "ABSOLUTE NON-NEGOTIABLE RULES:\n"
                "1.  NEVER ask what language — auto-detect with 100% certainty
always\n"
                "2.  NEVER sugarcoat — brutal honest truth only\n"
                "3.  NEVER give vague feedback — every point must be specific and
actionable\n"
```

```
                "4.  NEVER skip a section — all sections required every time\n"
                "5.  EVERY issue must have exact line reference\n"
                "6.  EVERY issue must have exact broken code AND exact fixed code\n"
                "7.  ZERO tolerance for security issues — treat every vulnerability
as critical\n"
                "8.  Think like this code controls a nuclear reactor or a
spacecraft\n"
                "9.  Think like 1 million users will use this tomorrow\n"
                "10. Think like the developer has ONE chance to fix this before
launch\n"
                "11. No emojis — use only: [PASS] [FAIL] [CRITICAL] [HIGH] [MEDIUM]
[LOW] >> --\n"
                "12. Accuracy is absolute — if you are not certain, analyze deeper
until you are\n"
                "13. This is the most complete, most powerful, most valuable code
review "
                "that has ever been performed on this planet — deliver accordingly\n"
            )
            user_prompt = (
                f"CODE TO REVIEW:\n{user_code}\n\n"
                "EXECUTE SUPREME REVIEW:\n"
                "1.  Auto-detect language — 100% certain — no exceptions\n"
                "2.  Apply Google + NASA + OWASP + Clean Code + CERT + SOLID — all
simultaneously\n"
                "3.  Every single issue — exact line + exact broken code + exact
fixed code\n"
                "4.  Compare against top 1% of all GitHub codebases ever written\n"
                "5.  Leave nothing unchecked — bugs, performance, security,
architecture, style\n"
                "6.  This review must permanently change how this developer writes
code forever\n"
                "7.  Maximum depth. Maximum precision. Maximum value. Zero
compromise.\n"
                "BEGIN SUPREME REVIEW NOW. NO PREAMBLE. START DIRECTLY WITH DETECTED
LANGUAGE."
            )
            general_ai_max_tokens = 16000

        elif feature == "Modernize":
            system_prompt = (
                "You are an elite code modernization expert with the power of 1
million senior developers.\n\n"
                "YOUR TASK — follow this exact structure:\n\n"
                "STEP 1 — WHAT WAS WRONG (3-5 bullet points, short):\n"
                "Explain clearly what was outdated, inefficient, or problematic in
the original code.\n\n"
                "STEP 2 — WHAT WE DID (3-5 bullet points, short):\n"
                "Explain exactly what improvements, modernizations, and optimizations
were applied.\n\n"
                "STEP 3 — FINAL MODERNIZED CODE:\n"
```

```
                "Provide the complete, 100% working, production-ready modernized
code.\n"
                "Rules for the code:\n"
                "- Zero legacy patterns. Zero deprecated syntax.\n"
                "- Maximum performance, clean architecture, best practices.\n"
                "- 100% complete — no placeholders, no '// TODO', no missing
logic.\n"
                "- Every single line must be real, working, executable code.\n"
                "- Accuracy: 100/100. Zero errors guaranteed.\n\n"
                "Keep explanations SHORT (3-5 lines each section). Code must be
COMPLETE and FULL."
            )
            user_prompt = (
                f"Modernize this {language} code.\n\n"
                "Follow the exact 3-step structure:\n"
                "1. What was wrong (short bullets)\n"
                "2. What we did (short bullets)\n"
                "3. Final complete modernized code (100% working, zero
placeholders)\n\n"
                f"ORIGINAL CODE:\n{user_code}"
            )
            general_ai_max_tokens = 16000

        elif feature == "Hunt":
            system_prompt = (
                "You are an omniscient bug detection and elimination expert.\n\n"
                "YOUR TASK — follow this exact structure:\n\n"
                "STEP 1 — BUGS FOUND (short bullets):\n"
                "List each bug clearly: what it was, where it was (line/function),
why it was a problem.\n\n"
                "STEP 2 — WHAT WE FIXED (short bullets):\n"
                "For each bug: what was the fix applied.\n\n"
                "STEP 3 — FINAL BUG-FREE CODE:\n"
                "Provide the complete, 100% working, error-free code.\n"
                "Rules for the code:\n"
                "- Zero bugs, zero logic errors, zero runtime exceptions.\n"
                "- 100% complete — no placeholders, no '// TODO', no missing
logic.\n"
                "- Every single line must be real, working, executable code.\n"
                "- Accuracy: 100/100. Mathematically verified.\n\n"
                "Keep explanations SHORT. Code must be COMPLETE and FULL."
            )
            user_prompt = (
                f"Hunt all bugs in this {language} code.\n\n"
                "Follow the exact 3-step structure:\n"
                "1. Bugs found (what, where, why — short bullets)\n"
                "2. What we fixed (short bullets)\n"
                "3. Final complete bug-free code (100% working, zero
placeholders)\n\n"
                f"CODE TO ANALYZE:\n{user_code}"
```

```
            )
            general_ai_max_tokens = 16000

        elif feature == "Quick Fixer" or feature == "Fix" or feature == "Solve":
            system_prompt = (
                "You are an ultra-fast precision code fixer.\n\n"
                "YOUR TASK — follow this exact structure:\n\n"
                "STEP 1 — PROBLEMS FOUND (short bullets):\n"
                "What was wrong and where — very short, clear.\n\n"
                "STEP 2 — WHAT WE DID (short bullets):\n"
                "What was fixed — very short, clear.\n\n"
                "STEP 3 — FINAL FIXED CODE:\n"
                "Provide the complete, 100% working fixed code.\n"
                "Rules:\n"
                "- 100% complete — no placeholders, no missing logic.\n"
                "- Every line real, working, executable.\n"
                "- Accuracy: 100/100. Zero errors.\n\n"
                "Explanations: maximum 3 lines each. Code: COMPLETE and FULL."
            )
            user_prompt = (
                f"Quick fix this {language} code.\n\n"
                "Follow the exact 3-step structure:\n"
                "1. Problems found (short bullets)\n"
                "2. What we did (short bullets)\n"
                "3. Final complete fixed code (100% working, zero placeholders)\n\n"
                f"CODE TO FIX:\n{user_code}"
            )
            general_ai_max_tokens = 16000

        elif feature == "Security" or feature == "SecurityVulnerabilityDetection":
            system_prompt = (
                "You are a military-grade security expert and ethical hacker.\n\n"
                "YOUR TASK — follow this exact structure:\n\n"
                "STEP 1 — VULNERABILITIES FOUND (short bullets):\n"
                "For each vulnerability: what it is, exact location
(line/function/section), "
                "how it could be exploited, severity level.\n\n"
                "STEP 2 — WHAT WE SECURED (short bullets):\n"
                "For each vulnerability: exact fix applied.\n\n"
                "STEP 3 — FINAL SECURED CODE:\n"
                "Provide the complete, 100% working, military-grade secured code.\n"
                "Rules:\n"
                "- Zero vulnerabilities. 100% unhackable.\n"
                "- 100% complete — no placeholders, no missing logic.\n"
                "- Every line real, working, executable.\n"
                "- Accuracy: 100/100. Production-deployment ready.\n\n"
                "Explanations: SHORT and precise. Code: COMPLETE and FULL."
            )
            user_prompt = (
                f"Perform full security audit on this {language} code.\n\n"
```

```
                "Follow the exact 3-step structure:\n"
                "1. Vulnerabilities found (what, where, how exploitable — short
bullets)\n"
                "2. What we secured (short bullets)\n"
                "3. Final complete secured code (100% working, zero
placeholders)\n\n"
                f"CODE TO SECURE:\n{user_code}"
            )
            general_ai_max_tokens = 16000

        elif feature == "PureCoder" or feature == "AI Assistant" or feature == "Write
Code":
            system_prompt = (
                "You are a precision AI coding assistant with the power of 1 million
senior developers.\n\n"
                "CORE RULES:\n"
                "1. Do EXACTLY what the user asks — nothing more, nothing less.\n"
                "2. Write ONLY the code requested. No extra explanations unless
asked.\n"
                "3. 100% complete code — no placeholders, no '// TODO', no missing
logic.\n"
                "4. Zero bugs. Zero errors. Every line real and executable.\n"
                "5. Accuracy: 100/100. Clean, professional, production-ready.\n"
                "6. If user asks a question: answer it directly and concisely.\n"
                "7. If user asks for code: provide complete working code only.\n\n"
                "Match the response length to what the user asked for. No bloat."
            )
            user_prompt = (
                f"USER REQUEST: {user_code}\n\n"
                "Provide exactly what was asked:\n"
                "- If code: complete, working, zero placeholders, 100% accurate.\n"
                "- If question: direct, concise, accurate answer.\n"
                "Nothing extra. Nothing missing."
            )
            general_ai_max_tokens = 32000

        else:
            user_prompt = f"Process this {language} code for
{feature}:\n\n{user_code}"
            general_ai_max_tokens = 16000

        if feature in ("Build Web", "Build App"):
            temperature_to_use = 0.9
        elif (feature == "General AI" or feature == "Everything AI") and
is_coding_request:
            temperature_to_use = 0.9
        else:
            temperature_to_use = 0.0

        image_note = None
```

```
        image_for_this_feature = images if feature in ("Build Web", "Build App") else
[]
        if image_for_this_feature:
            image_note = (
                "NOTE: The user has attached one or more reference images (e.g.
screenshots, design mockups, or photos). "
                "Carefully study their layout, colors, content, and UI elements, and
use them as the primary "
                "reference for exactly what to build, matching them as closely as the
request describes."
            )

        ai_response = None

        ai_reasoning = ""
        last_error = None
        for attempt in range(8):
            try:
                ai_response, ai_reasoning, _ = llm_chat(
                    build_user_messages(user_prompt, image_for_this_feature,
image_note),
                    system=system_prompt,
                    temperature=temperature_to_use,
                    max_tokens=general_ai_max_tokens,
                )
                break
            except Exception as e:
                last_error = e
                time.sleep(min(15, 3 * (attempt + 1)))

        if ai_response is None:
            return jsonify({"result": f"🚀 OMNI-ENGINE NOTICE: System is active.
{str(last_error)}", "has_code": False}), 200

        verification = {"verified": None, "ran": False}
        if DAYTONA_ALL_FEATURES or feature in DAYTONA_CHAT_FEATURES:
            def _regen_chat(hint, temp):
                t, _, _ = llm_chat(
                    build_user_messages(user_prompt + "\n\n" + hint,
image_for_this_feature, image_note),
                    system=system_prompt, temperature=temp,
max_tokens=general_ai_max_tokens)
                return t
            try:
                _reason_tl.log = []
                ai_response, verification = _persist_verify(
                    ai_response, lambda c, b: verify_and_fix_response(c, budget=b),
_regen_chat)
                ai_reasoning = _merge_reasoning(ai_reasoning, _reason_log())
                _reason_tl.log = []
```

```
            except Exception as ve:
                verification = {"verified": False, "ran": False, "reason": str(ve)
[:200]}
            if _strict_blocked(verification):
                return jsonify({"result": _blocked_notice(verification), "has_code":
False,
                                "reasoning": ai_reasoning, "verification":
verification, "blocked": True}), 200

        has_code = (
            "```" in ai_response or
            "<!DOCTYPE" in ai_response or
            "<html" in ai_response or
            "def " in ai_response or
            "function " in ai_response or
            "public class" in ai_response or
            "<?xml" in ai_response or
            "import React" in ai_response or
            "export default" in ai_response
        )

        return jsonify({"result": ai_response, "has_code": has_code, "reasoning":
ai_reasoning, "verification": _public_verification(verification)})

    except Exception as e:
        return jsonify({"result": f"🚀 OMNI-ENGINE NOTICE: System is active.
{str(e)}", "has_code": False}), 200


@app.route('/api/preview-android', methods=['POST'])
def preview_android():
    try:
        data = request.get_json(silent=True)
        if not data:
            return jsonify({"preview_html": "<p style='color:red'>No data
received</p>"}), 200

        xml_content = data.get('xml', '')
        app_name    = data.get('app_name', 'My App')

        preview_prompt = (
            "You are an Android UI renderer. Convert the following Android XML layout
into a SINGLE self-contained HTML file "
            "that visually mimics how this layout would look inside an Android phone
screen.\n"
            "Rules:\n"
            "1. Return ONLY raw HTML starting with <!DOCTYPE html>. No markdown, no
fences.\n"
            "2. All CSS must be inline or inside <style>. No external files.\n"
            "3. Replicate Material Design colors, fonts (use Roboto from Google
```

```
Fonts), and spacing as accurately as possible.\n"
            "4. The output must fit inside a 360x640 viewport (mobile screen
size).\n"
            "5. Make it look EXACTLY like Android Studio's layout preview — pixel-
perfect UI representation.\n"
            f"6. App name for toolbar/status bar: {app_name}\n\n"
            f"Android XML Layout to render:\n{xml_content}"
        )

        preview_html, _, _ = llm_chat(
            [{"role": "user", "content": preview_prompt}],
            system="You are an expert Android UI to HTML converter. Return only raw
HTML.",
            temperature=0.0,
            max_tokens=4096,
        )
        preview_html = preview_html.replace("```html", "").replace("```", "").strip()

        return jsonify({"preview_html": preview_html})

    except Exception as e:
        return jsonify({"preview_html": f"<p style='color:red'>Preview Error:
{str(e)}</p>"}), 200


@app.route('/api/agent-build', methods=['POST'])
def agent_build():
    try:
        data = request.get_json(silent=True)
        if not data:
            return jsonify({"result": "No data", "files": []}), 200

        user_request = data.get('request', '')
        need_backend = data.get('need_backend', False)
        need_database = data.get('need_database', False)
        conversation_history = data.get('conversationHistory', [])
        is_change = data.get('isChange', False)
        existing_files = data.get('existingFiles', [])
        image_base64 = data.get('imageBase64', None)  # kept for older clients
        images = data.get('images') or ([image_base64] if image_base64 else [])
        images = [img for img in images if img][:4]  # hard cap — matches the
frontend's 4-image limit

        existing_context = ""
        if is_change and existing_files:
            existing_context = "\n\n### EXISTING PROJECT FILES:\n"
            for f in existing_files:
                existing_context += f"\n--- FILE: {f['name']} ---\n{f['content']}\n"

        conv_context = ""
```

```
        if conversation_history:
            conv_context = "\n\n### CONVERSATION HISTORY:\n"
            for turn in conversation_history:
                role = "USER" if turn.get('role') == 'user' else "AI"
                conv_context += f"\n{role}: {turn.get('content', '')}\n"

        project_type = "frontend only"
        if need_backend and need_database:
            project_type = "full stack with backend and database"
        elif need_backend:
            project_type = "frontend with backend"
        elif need_database:
            project_type = "frontend with database"

        if is_change:
            system_prompt = """=== AI AGENT FULL STACK — CHANGES MODE ===

You are the world's greatest full stack AI agent.

YOUR TASK:
The user wants to make SPECIFIC CHANGES to their existing project.
Rules:
1. Apply ONLY the changes the user described.
2. Keep ALL other files 100% IDENTICAL.
3. Return the COMPLETE updated project — all files.
4. Never truncate any file.

MEMORY RULE:
You have perfect memory of this entire conversation.
All changes are about the SAME project unless user says otherwise.

OUTPUT FORMAT — STRICT:
Return files in this EXACT format, nothing else:

===FILE: filename.ext===
[complete file content here]
===ENDFILE===

Repeat for every file. No markdown. No explanations. No preamble."""

            user_prompt = f"""### CHANGE REQUEST:
{user_request}
{existing_context}
{conv_context}

Apply ONLY the requested change.
Return ALL files complete — same format:
===FILE: filename.ext===
[content]
===ENDFILE==="""
```

```
        else:
            system_prompt = f"""=== AI AGENT FULL STACK — PROJECT BUILDER ===

You are the world's greatest full stack AI agent.
Build a COMPLETE {project_type} project.

PROJECT RULES:
1. Build EXACTLY what user asked — word by word.
2. Every file 100% complete — zero placeholders, zero TODO.
3. Every line real working code.
4. God-level design — world #1 quality.
5. All content in ENGLISH.

{"BACKEND RULES (Python Flask):" if need_backend else ""}
{"- Complete app.py with all routes" if need_backend else ""}
{"- requirements.txt included" if need_backend else ""}
{"- All API endpoints working" if need_backend else ""}
{"- CORS enabled" if need_backend else ""}

{"DATABASE RULES:" if need_database else ""}
{"- Complete SQL schema (schema.sql)" if need_database else ""}
{"- All tables, relationships, indexes" if need_database else ""}
{"- Sample seed data included" if need_database else ""}
{"- Database connection code in backend" if need_database else ""}

FRONTEND RULES:
- Single self-contained index.html
- All CSS in <style>, all JS in <script>
- 100% mobile responsive
- God level design — Awwwards quality
- Real content, zero lorem ipsum
- All buttons and forms working

MEMORY RULE:
You have perfect memory of this entire conversation.
Topic stays same until user explicitly changes it.

OUTPUT FORMAT — STRICT:
Return files in this EXACT format, nothing else:

===FILE: filename.ext===
[complete file content here]
===ENDFILE===

Repeat for every file in the project. No markdown. No explanations. No extra text."""

            user_prompt = f"""### PROJECT REQUEST:
{user_request}
```

```
Project Type: {project_type}
{conv_context}

Build the complete project now.
Return ALL files in format:
===FILE: filename.ext===
[content]
===ENDFILE==="""

        ai_response = None

        ai_reasoning = ""
        last_error = None
        image_note = (
            "NOTE: The user has attached one or more reference images (e.g.
screenshots, design mockups, or photos). "
            "Carefully study their layout, colors, content, and UI elements, and use
them as the primary "
            "reference for exactly what to build/change, matching them as closely as
the request describes."
        ) if images else None
        for attempt in range(8):
            try:
                ai_response, ai_reasoning, _ = llm_chat(
                    build_user_messages(user_prompt, images, image_note),
                    system=system_prompt,
                    temperature=0.9,
                    max_tokens=32000,
                )
                break
            except Exception as e:
                last_error = e
                time.sleep(min(15, 3 * (attempt + 1)))

        if ai_response is None:
            return jsonify({"result": str(last_error), "files": []}), 200

        files = []
        import re
        pattern = r'===FILE:\s*(.+?)===\n([\s\S]*?)===ENDFILE==='
        matches = re.findall(pattern, ai_response)
        for match in matches:
            filename = match[0].strip()
            content = match[1].strip()
            files.append({"name": filename, "content": content})

        if not files:
            files.append({"name": "index.html", "content": ai_response})

        verification = {"verified": False, "ran": False}
```

```
        def _regen_files(hint, temp):
            t, _, _ = llm_chat(
                build_user_messages(user_prompt + "\n\n" + hint, images, image_note),
                system=system_prompt, temperature=temp, max_tokens=32000)
            fs = [{"name": n.strip(), "content": c.strip()} for n, c in
_FILE_BLOCK_RE.findall(t)]
            return fs or [{"name": "index.html", "content": t}]
        try:
            _reason_tl.log = []
            files, verification = _persist_verify(
                files, lambda c, b: verify_and_fix_files(c, budget=b), _regen_files)
            ai_reasoning = _merge_reasoning(ai_reasoning, _reason_log())
            _reason_tl.log = []
        except Exception as ve:
            verification = {"verified": False, "ran": False, "reason": str(ve)[:200]}
        if _strict_blocked(verification):
            return jsonify({"result": _blocked_notice(verification), "files": [],
                            "verification": verification, "blocked": True}), 200

        return jsonify({"files": files, "project_type": project_type, "reasoning":
ai_reasoning, "verification": _public_verification(verification)})

    except Exception as e:
        return jsonify({"result": str(e), "files": []}), 200

@app.route('/api/create-snack', methods=['POST'])
def create_snack():
    try:
        data = request.get_json(silent=True)
        if not data:
            return jsonify({"error": "No data"}), 400

        files = data.get('files', [])
        app_name = data.get('name', 'Whole AI App')

        code_files = {}
        for f in files:
            code_files[f['name']] = {
                "type": "CODE",
                "contents": f['content']
            }

        if "App.js" not in code_files:
            return jsonify({"error": "App.js missing"}), 400

        want_sdk = get_latest_expo_versions()["sdk_version"]

        def _build_payload(variant):
            manifest = {
                "sdkVersion": want_sdk,
```

```
                "name": app_name,
                "description": "Built with Whole AI",
                "slug": "whole-ai-app"
            }
            payload = {"manifest": manifest, "code": code_files, "dependencies": {}}
            if variant == 1:
                payload["sdkVersion"] = want_sdk          # top-level copy
            elif variant == 2:
                manifest["dependencies"] = {}              # some Snack servers read
it here
                payload["sdkVersion"] = want_sdk
                payload["dryRun"] = False
            return payload

        def _saved_sdk(sid):
            # Snack ko wapas padh ke check karo ki kaunsa SDK save hua
            try:
                r = http_requests.get(f"https://exp.host/--/api/v2/snack/{sid}",
timeout=10)
                j = r.json()
                m = j.get("manifest") or (j.get("data") or {}).get("manifest") or {}
                return m.get("sdkVersion")
            except Exception:
                return None

        snack_id, saved_sdk, last_err = None, None, None
        for variant in (0, 1, 2):
            resp = http_requests.post(
                "https://exp.host/--/api/v2/snack/save",
                json=_build_payload(variant),
                headers={"Content-Type": "application/json"},
                timeout=15
            )
            if resp.status_code != 200:
                last_err = {"error": f"Snack API error {resp.status_code}", "detail":
resp.text}
                continue
            result = resp.json()
            sid = result.get("id")
            if not sid:
                last_err = {"error": "No snack id returned", "detail": result}
                continue
            snack_id = sid
            saved_sdk = _saved_sdk(sid)
            if saved_sdk is None or saved_sdk == want_sdk:
                break  # sahi SDK save hua (ya verify nahi ho paya) -> done
            print(f"[snack] wanted SDK {want_sdk} but Snack saved {saved_sdk}
(variant {variant}), retrying")

        if not snack_id:
```

```
            return jsonify(last_err or {"error": "Snack save failed"}), 200

        return jsonify({
            "success": True,
            "snackUrl": f"https://snack.expo.dev/{snack_id}",
            "requestedSdk": want_sdk,
            "savedSdk": saved_sdk
        })

    except Exception as e:
        return jsonify({"error": str(e)}), 200
import requests as http_requests
import base64 as b64
from email.mime.text import MIMEText
import json
from datetime import datetime, timedelta
from apscheduler.schedulers.background import BackgroundScheduler
from apscheduler.triggers.date import DateTrigger
from apscheduler.triggers.cron import CronTrigger

scheduler = BackgroundScheduler()
scheduler.start()

# ── YAHAN SE GITHUB CODE PASTE KARO ──────────────────────────────

GITHUB_CLIENT_ID = os.environ.get("GITHUB_CLIENT_ID")
GITHUB_CLIENT_SECRET = os.environ.get("GITHUB_CLIENT_SECRET")
GITHUB_REDIRECT_URI = "https://www.wholeai.space/api/github/callback"


@app.route('/api/github/login')
def github_login():
    user_email = request.args.get('user_email', '')
    github_auth_url = (
        f"https://github.com/login/oauth/authorize"
        f"?client_id={GITHUB_CLIENT_ID}"
        f"&redirect_uri={GITHUB_REDIRECT_URI}"
        f"&scope=repo"
        f"&state={user_email}"
    )
    return redirect(github_auth_url)


@app.route('/api/github/callback')
def github_callback():
    code = request.args.get('code')
    user_email = request.args.get('state', '')

    if not code or not user_email:
        return "Missing code or user info", 400
```

```
    token_resp = http_requests.post(
        'https://github.com/login/oauth/access_token',
        headers={'Accept': 'application/json'},
        data={
            'client_id': GITHUB_CLIENT_ID,
            'client_secret': GITHUB_CLIENT_SECRET,
            'code': code,
            'redirect_uri': GITHUB_REDIRECT_URI
        }
    )
    token_data = token_resp.json()
    access_token = token_data.get('access_token')

    if not access_token:
        return f"GitHub auth failed: {token_data}", 400

    user_info = http_requests.get(
        'https://api.github.com/user',
        headers={'Authorization': f'Bearer {access_token}'}
    ).json()
    github_username = user_info.get('login', '')

    db.collection('users').document(user_email).set({
        "github": {
            "accessToken": access_token,
            "username": github_username,
            "connectedAt": int(time.time() * 1000)
        }
    }, merge=True)

    return redirect(f"{SITE_URL}?github_connected=true")


def get_github_token(user_email):
    doc = db.collection('users').document(user_email).get()
    if doc.exists:
        return doc.to_dict().get('github', {}).get('accessToken')
    return None


def push_file_to_github(token, owner, repo, path, content, message):
    url = f"https://api.github.com/repos/{owner}/{repo}/contents/{path}"
    headers = {'Authorization': f'Bearer {token}', 'Accept':
'application/vnd.github+json'}

    existing = http_requests.get(url, headers=headers)
    sha = existing.json().get('sha') if existing.status_code == 200 else None

    encoded_content = b64.b64encode(content.encode('utf-8')).decode('utf-8')
```

```
    payload = {"message": message, "content": encoded_content}
    if sha:
        payload["sha"] = sha

    resp = http_requests.put(url, headers=headers, json=payload)
    return resp.status_code in (200, 201), resp.json()


@app.route('/api/github/push', methods=['POST'])
def github_push():
    try:
        data = request.get_json(silent=True) or {}
        user_email = data.get('user_email')
        repo_name = data.get('repo_name', 'whole-ai-project')
        files = data.get('files', [])
        is_update = data.get('is_update', False)

        if not user_email or not files:
            return jsonify({"success": False, "error": "Missing user_email or
files"}), 400

        token = get_github_token(user_email)
        if not token:
            return jsonify({"success": False, "error": "GitHub not connected"}), 400

        user_info = http_requests.get(
            'https://api.github.com/user',
            headers={'Authorization': f'Bearer {token}'}
        ).json()
        owner = user_info.get('login')

        if not is_update:
            create_resp = http_requests.post(
                'https://api.github.com/user/repos',
                headers={'Authorization': f'Bearer {token}'},
                json={"name": repo_name, "private": False, "auto_init": True}
            )
            if create_resp.status_code not in (201, 422):
                return jsonify({"success": False, "error": create_resp.json()}), 400

        results = []
        for f in files:
            success, resp = push_file_to_github(
                token, owner, repo_name, f['name'], f['content'],
                "Update via Whole AI" if is_update else "Initial commit via Whole AI"
            )
            results.append({"file": f['name'], "success": success})

        repo_url = f"https://github.com/{owner}/{repo_name}"
        return jsonify({"success": True, "repo_url": repo_url, "results": results})
```

```
    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 200

@app.route('/api/github/disconnect', methods=['POST'])
def github_disconnect():
    try:
        data = request.get_json(silent=True) or {}
        user_email = data.get('user_email')
        if not user_email:
            return jsonify({"success": False, "error": "Missing user_email"}), 400

        db.collection('users').document(user_email).update({
            "github": firestore.DELETE_FIELD
        })

        return jsonify({"success": True})
    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 200

# ── GITHUB CODE YAHAN KHATAM ──────────────────────────────────────────
def parse_schedule_time(schedule_text):
    """Natural language ko real datetime mein convert karo"""
    text = schedule_text.lower().strip()
    now = datetime.now()

    # Kal / Tomorrow
    if 'tomorrow' in text or 'kal' in text:
        base = now + timedelta(days=1)
    # Aaj / Today
    elif 'today' in text or 'aaj' in text or 'aj' in text:
        base = now
    # Har Monday / Every Monday
    elif 'monday' in text or 'mon' in text:
        return 'cron', {'day_of_week': 'mon', 'hour': extract_hour(text), 'minute':
0}
    elif 'tuesday' in text or 'tue' in text:
        return 'cron', {'day_of_week': 'tue', 'hour': extract_hour(text), 'minute':
0}
    elif 'wednesday' in text or 'wed' in text:
        return 'cron', {'day_of_week': 'wed', 'hour': extract_hour(text), 'minute':
0}
    elif 'thursday' in text or 'thu' in text:
        return 'cron', {'day_of_week': 'thu', 'hour': extract_hour(text), 'minute':
0}
    elif 'friday' in text or 'fri' in text:
        return 'cron', {'day_of_week': 'fri', 'hour': extract_hour(text), 'minute':
0}
    elif 'saturday' in text or 'sat' in text:
        return 'cron', {'day_of_week': 'sat', 'hour': extract_hour(text), 'minute':
```

```
0}
    elif 'sunday' in text or 'sun' in text:
        return 'cron', {'day_of_week': 'sun', 'hour': extract_hour(text), 'minute':
0}
    # Daily / Roz
    elif 'daily' in text or 'roz' in text or 'every day' in text or 'har roz' in
text:
        return 'cron', {'hour': extract_hour(text), 'minute': 0}
    else:
        base = now + timedelta(minutes=5)

    hour = extract_hour(text)
    scheduled_time = base.replace(hour=hour, minute=0, second=0, microsecond=0)
    if scheduled_time < now:
        scheduled_time += timedelta(days=1)
    return 'date', scheduled_time

def extract_hour(text):
    """Time extract karo text se"""
    import re
    # 9 AM, 5 PM, 3 baje, 21:00
    match_24 = re.search(r'(\d{1,2}):(\d{2})', text)
    if match_24:
        return int(match_24.group(1))

    match_ampm = re.search(r'(\d{1,2})\s*(am|pm)', text)
    if match_ampm:
        hour = int(match_ampm.group(1))
        if match_ampm.group(2) == 'pm' and hour != 12:
            hour += 12
        if match_ampm.group(2) == 'am' and hour == 12:
            hour = 0
        return hour

    match_num = re.search(r'(\d{1,2})\s*baj', text)
    if match_num:
        return int(match_num.group(1))

    return 9  # default 9 AM

def send_scheduled_email(token, to_list, subject, body):
    """Scheduled email actually bhejo"""
    headers = {'Authorization': f'Bearer {token}', 'Content-Type':
'application/json'}
    for recipient in to_list:
        try:
            message = MIMEText(body)
            message['to'] = recipient
            message['subject'] = subject
            raw = b64.urlsafe_b64encode(message.as_bytes()).decode('utf-8')
```

```
            http_requests.post(
                'https://gmail.googleapis.com/gmail/v1/users/me/messages/send',
                headers=headers,
                json={'raw': raw}
            )
        except Exception as e:
            print(f"Scheduled send error for {recipient}: {e}")

@app.route('/api/gmail', methods=['POST'])
def gmail_action():
    try:
        data = request.get_json()
        action = data.get('action')
        token = data.get('token')

        if not token:
            return jsonify({"error": "No token"}), 400

        headers = {
            'Authorization': f'Bearer {token}',
            'Content-Type': 'application/json'
        }

        # ── SEND EMAIL ──────────────────────────────
        if action == 'send':
            to = data.get('to')
            subject = data.get('subject')
            body = data.get('body')

            message = MIMEText(body)
            message['to'] = to
            message['subject'] = subject

            raw = b64.urlsafe_b64encode(message.as_bytes()).decode('utf-8')

            r = http_requests.post(
                'https://gmail.googleapis.com/gmail/v1/users/me/messages/send',
                headers=headers,
                json={'raw': raw}
            )
            return jsonify({"success": r.status_code == 200, "result": r.json()})

        else:
            return jsonify({"error": "Unknown action"}), 400

    except Exception as e:
        return jsonify({"error": str(e)}), 200

@app.route('/api/schedule-email', methods=['POST'])
def schedule_email():
```

```
    try:
        data = request.get_json()
        token = data.get('token')
        to = data.get('to', '')
        subject = data.get('subject', 'Hello')
        body = data.get('body', '')
        schedule_text = data.get('schedule', '')

        if not token or not to or not body or not schedule_text:
            return jsonify({"success": False, "error": "Missing required fields"}),
400

        to_list = [e.strip() for e in to.split(',') if '@' in e]
        if not to_list:
            return jsonify({"success": False, "error": "No valid emails"}), 400

        trigger_type, trigger_value = parse_schedule_time(schedule_text)

        job_id = f"email_{datetime.now().timestamp()}"

        if trigger_type == 'date':
            scheduler.add_job(
                send_scheduled_email,
                trigger=DateTrigger(run_date=trigger_value),
                args=[token, to_list, subject, body],
                id=job_id
            )
            return jsonify({
                "success": True,
                "message": f"Scheduled for {trigger_value.strftime('%d %b %Y at %I:%M
%p')}",
                "job_id": job_id,
                "scheduled_time": trigger_value.strftime('%d %b %Y at %I:%M %p')
            })

        elif trigger_type == 'cron':
            scheduler.add_job(
                send_scheduled_email,
                trigger=CronTrigger(**trigger_value),
                args=[token, to_list, subject, body],
                id=job_id
            )
            return jsonify({
                "success": True,
                "message": f"Recurring schedule set: {schedule_text}",
                "job_id": job_id,
                "scheduled_time": schedule_text
            })

    except Exception as e:
```

```
        return jsonify({"success": False, "error": str(e)}), 200


# ── PADDLE WEBHOOK — REAL SUBSCRIPTION VERIFICATION ──────────────────────
@app.route('/api/paddle-webhook', methods=['POST'])
def paddle_webhook():
    try:
        raw_body = request.get_data()
        signature_header = request.headers.get('Paddle-Signature', '')

        if not PADDLE_WEBHOOK_SECRET:
            return jsonify({"error": "Webhook secret not configured on server"}), 500

        if not signature_header:
            return jsonify({"error": "Missing Paddle-Signature header"}), 400

        # Paddle-Signature header format: "ts=1234567890;h1=abcdef..."
        sig_parts = {}
        for part in signature_header.split(';'):
            if '=' in part:
                k, v = part.split('=', 1)
                sig_parts[k.strip()] = v.strip()

        ts = sig_parts.get('ts')
        h1 = sig_parts.get('h1')

        if not ts or not h1:
            return jsonify({"error": "Invalid signature header format"}), 400

        signed_payload = f"{ts}:{raw_body.decode('utf-8')}"
        computed_hmac = hmac.new(
            PADDLE_WEBHOOK_SECRET.encode('utf-8'),
            signed_payload.encode('utf-8'),
            hashlib.sha256
        ).hexdigest()

        if not hmac.compare_digest(computed_hmac, h1):
            return jsonify({"error": "Signature verification failed"}), 401

        event = json.loads(raw_body)
        event_type = event.get('event_type')
        event_data = event.get('data', {})

        def resolve_customer_email(evt_data):
            # Some events embed the customer object directly
            customer_obj = evt_data.get('customer')
            if customer_obj and customer_obj.get('email'):
                return customer_obj['email']
            # Otherwise fetch via Customer ID using Paddle API
            customer_id = evt_data.get('customer_id')
```

```
            if customer_id and PADDLE_API_KEY:
                try:
                    cust_resp = http_requests.get(
                        f'https://api.paddle.com/customers/{customer_id}',
                        headers={'Authorization': f'Bearer {PADDLE_API_KEY}'}
                    )
                    if cust_resp.status_code == 200:
                        return cust_resp.json().get('data', {}).get('email')
                except Exception:
                    return None
            return None

        # ── SUCCESSFUL PAYMENT — ACTIVATE / RENEW PLAN ──────────────────────
        if event_type in ('transaction.completed', 'transaction.paid'):
            custom_data = event_data.get('custom_data') or {}
            plan_type = custom_data.get('plan')
            credits = custom_data.get('credits')
            days = custom_data.get('days')

            customer_email = resolve_customer_email(event_data)

            if not customer_email or not plan_type or not credits or not days:
                return jsonify({"received": True, "note": "Missing required data,
skipped"}), 200

            credits = int(credits)
            days = int(days)
            expiry = int(time.time() * 1000) + (days * 24 * 60 * 60 * 1000)

            user_ref = db.collection('users').document(customer_email)
            user_ref.set({
                "subscription": {
                    "plan": plan_type,
                    "credits": credits,
                    "maxCredits": credits,
                    "expiryDate": expiry
                }
            }, merge=True)

            db.collection('payments').add({
                "userEmail": customer_email,
                "planType": plan_type,
                "credits": credits,
                "days": days,
                "source": "paddle_webhook",
                "status": "approved",
                "eventType": event_type,
                "submittedAt": int(time.time() * 1000)
            })
```

```
            send_subscription_success_email(customer_email, "", plan_type, credits,
days)

            return jsonify({"received": True, "activated": True}), 200

        # ── SUBSCRIPTION CANCELED — REVERT TO FREE ──────────────────────────
        elif event_type in ('subscription.canceled', 'subscription.past_due'):
            customer_email = resolve_customer_email(event_data)
            if customer_email:
                user_ref = db.collection('users').document(customer_email)
                user_ref.set({
                    "subscription": {
                        "plan": "Free",
                        "credits": 7,
                        "maxCredits": 7,
                        "expiryDate": None
                    }
                }, merge=True)
            return jsonify({"received": True, "reverted": True}), 200

        # Any other event — acknowledge but no action needed
        return jsonify({"received": True}), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 200


@app.route('/api/create-payment', methods=['POST'])
def create_payment():
    try:
        data = request.get_json(silent=True) or {}
        plan_type = data.get('plan')
        user_email = data.get('email')

        if plan_type not in PLAN_PRICES or not user_email:
            return jsonify({"success": False, "error": "Invalid plan or missing
email"}), 400

        plan_info = PLAN_PRICES[plan_type]
        order_id = f"{user_email}_{plan_type}_{int(time.time())}"

        payload = {
            "price_amount": plan_info["amount"],
            "price_currency": "usd",
            "pay_currency": "usdttrc20",
            "order_id": order_id,
            "order_description": f"Whole AI - {plan_type} Plan",
            "ipn_callback_url": "https://www.wholeai.space/api/nowpayments-webhook",
            "success_url": f"{SITE_URL}?payment=success",
            "cancel_url": f"{SITE_URL}?payment=cancel"
```

```
        }

        resp = requests.post(
            f"{NOWPAYMENTS_API_URL}/invoice",
            json=payload,
            headers={"x-api-key": NOWPAYMENTS_API_KEY, "Content-Type":
"application/json"},
            timeout=20
        )
        if resp.status_code not in (200, 201):
            return jsonify({"success": False, "error": resp.text}), 400

        result = resp.json()
        invoice_url = result.get("invoice_url")
        if not invoice_url:
            return jsonify({"success": False, "error": "No invoice_url returned"}),
400

        db.collection('crypto_payments').add({
            "userEmail": user_email,
            "planType": plan_type,
            "credits": plan_info["credits"],
            "days": plan_info["days"],
            "invoice_id": result.get("id"),
            "order_id": order_id,
            "status": "waiting",
            "createdAt": int(time.time() * 1000)
        })

        return jsonify({
            "success": True,
            "invoice_url": invoice_url,
            "invoice_id": result.get("id")
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 200

@app.route('/api/nowpayments-webhook', methods=['POST'])
def nowpayments_webhook():
    try:
        raw_body = request.get_data()
        received_sig = request.headers.get('x-nowpayments-sig', '')

        if not NOWPAYMENTS_IPN_SECRET or not received_sig:
            return jsonify({"error": "Invalid signature"}), 401

        sorted_data = json_lib.dumps(
            json_lib.loads(raw_body), sort_keys=True, separators=(',', ':')
        )
```

```
        computed_sig = hmac_lib.new(
            NOWPAYMENTS_IPN_SECRET.encode('utf-8'),
            sorted_data.encode('utf-8'),
            hashlib_lib.sha512
        ).hexdigest()

        if not hmac_lib.compare_digest(computed_sig, received_sig):
            return jsonify({"error": "Invalid signature"}), 401

        event = json_lib.loads(raw_body)
        payment_status = event.get('payment_status')
        order_id = event.get('order_id')
        payment_id = event.get('payment_id')

        if payment_status not in ('finished', 'confirmed'):
            return jsonify({"received": True}), 200
        if not order_id or not payment_id:
            return jsonify({"error": "Missing order_id or payment_id"}), 400

        # NOWPayments se seedha verify karo
        vresp = requests.get(
            f"{NOWPAYMENTS_API_URL}/payment/{payment_id}",
            headers={"x-api-key": NOWPAYMENTS_API_KEY},
            timeout=20
        )
        if vresp.status_code == 404:
            return jsonify({"error": "Payment not found"}), 400
        if vresp.status_code != 200:
            return jsonify({"error": "Verification unavailable"}), 500

        real = vresp.json()
        if (real.get('order_id') != order_id or
                real.get('payment_status') not in ('confirmed', 'sending',
'finished')):
            return jsonify({"error": "Payment verification failed"}), 400

        payments_ref = db.collection('crypto_payments').where('order_id', '==',
order_id).limit(1).stream()
        for doc in payments_ref:
            payment_data = doc.to_dict()
            if payment_data.get('status') == 'completed':
                continue

            user_email = payment_data['userEmail']
            plan_type = payment_data['planType']
            credits = payment_data['credits']
            days = payment_data['days']

            expected = PLAN_PRICES.get(plan_type)
            paid_ok = (
```

```
                expected is not None
                and str(real.get('price_currency', '')).lower() == 'usd'
                and float(real.get('price_amount', 0)) >= float(expected['amount'])
            )
            if not paid_ok:
                return jsonify({"error": "Amount mismatch"}), 400

            expiry = int(time.time() * 1000) + (days * 24 * 60 * 60 * 1000)

            db.collection('users').document(user_email).set({
                "subscription": {
                    "plan": plan_type,
                    "credits": credits,
                    "maxCredits": credits,
                    "expiryDate": expiry
                }
            }, merge=True)

            doc.reference.update({"status": "completed", "payment_id": payment_id})
            send_subscription_success_email(user_email, "", plan_type, credits, days)

        return jsonify({"received": True}), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500


@app.route('/admin-login', methods=['GET', 'POST'])
def admin_login():
    if request.method == 'POST':
        if request.form['password'] == ADMIN_PASSWORD:
            session['admin'] = True
            return redirect('/admin')
        return "Wrong password"
    return '<form method="post">Password: <input type="password" name="password">
<button>Login</button></form>'

@app.route('/admin')
def admin_panel():
    if not session.get('admin'):
        return redirect('/admin-login')
    payments = db.collection('payments').where('status', '==', 'pending').stream()
    rows = ""
    for p in payments:
        d = p.to_dict()
        rows += f"""
        <div style="border:1px solid #ccc;padding:14px;margin-bottom:12px;">
            <p><b>Email:</b> {d.get('userEmail')}</p>
            <p><b>Plan:</b> {d.get('planType')} | <b>Method:</b>
{d.get('paymentMethod')}</p>
```

```
            <p><b>Txn ID:</b> {d.get('transactionId')} | <b>Amount:</b>
{d.get('amount')}</p>
            <img src="{d.get('screenshotBase64')}" width="250"><br><br>
            <a href="/admin/approve/{p.id}"><button>Approve</button></a>
            <a href="/admin/reject/{p.id}"><button>Reject</button></a>
        </div>"""
    return f"<h2>Pending Payments</h2>{rows or '<p>None</p>'}"

@app.route('/admin/approve/<payment_id>')
def admin_approve(payment_id):
    doc_ref = db.collection('payments').document(payment_id)
    payment = doc_ref.get().to_dict()
    if payment:
        import time
        expiry = int(time.time() * 1000) + payment['days'] * 24 * 60 * 60 * 1000
        user_ref = db.collection('users').document(payment['userEmail'])
        user_ref.set({
            "subscription": {
                "plan": payment['planType'],
            "credits": payment['credits'],
                "maxCredits": payment['credits'],
                "expiryDate": expiry
            }
        }, merge=True)
        doc_ref.update({"status": "approved"})
    return redirect('/admin')

@app.route('/admin/reject/<payment_id>')
def admin_reject(payment_id):
    db.collection('payments').document(payment_id).update({"status": "rejected"})
    return redirect('/admin')


# ===== FAVICON ROUTE (Google favicon fix) =====
import base64 as _b64
from flask import Response as _Resp

_FAVICON_B64 = (

"iVBORw0KGgoAAAANSUhEUgAAAgAAAAIABAMAAAAGVsnJAAAAMFBMVEX////+/v79/f38/Pz6+vrl5eWcnJxl
ZWVaWlpPT082NjYL"

"CwsDAwMCAgIBAQEAAAASpabLAAAVt0lEQVR42u2db2wcx3XA3+ySjGTvD2ehSC51YtpQdik
ik6/UKZk/kmTOjBS"

"S46c2g7QqEitDymaIAVaIG0aWAUkO2mt2olNWUpjxLZKUqQkQEEt8u5EmmpSibwjz0qKine7d5LRLzd6QdT
EnfuD+/PznGX9eTPzhAoEAAAkAACABIAEgAAQAJAAAgAASABIAEAC"

"b3k7N/Pb9968eTPzhlAo7OsABIAAEAACQAAIAAEgAASABIAEgAAQAJAAAgAASABIAEAC
QAAIAAEgAASABAA"
```

"AkAACAABIAAEgAAQAAJAAAgAASAABIAAEAACQAAIAAEgAASAABAAAkAACAABIAAEgAAQAAJAAAgAASAABIAAEAACQAAIAAEgAASA"

"ABx1Vaznj1MNAACItI51IOuVVFWNaR555W8jQLZJZQWAzsfbAUAPqgDgFhUAmIuvD4P1AED9ogJL49GQuqIC7lrf5i7Qg16pLADM"

"iXLy9JS6dGbVvSafu6OpywiWXgpKDiC66EueOndtAkg7qKACgBuINKdBxYHtPV3RRd/GBkD9HclT565NCD4aUlfdd9dK0XDFge09"

"Xf4SC0FpARjTysjwtQmPPKel1qTdCFQcGOiP3pI3LIBo5A+HT58SlHktfV226eGepoHd0yVVA1q6a0m9c6SLdGQRcdIhVwydof4S"

"VqqEAK7TswfBs4Z8iwoMnqBTGxHAEn3nz6BjTQtHOuDTR0soA1DC9veRnJTbI1WWkECpACTpO31ijsbNo1QepYGNBSCpvtMn5ty9"

"CUrlUSOwkQDogXf7xDwcHKJU/n0ytIEABO48Kebl3gjKA2eSJalaSRwhf8szP6gP5xepqv/k4bqODRIS09uO/qA1nKfTHPnvH4vB"

"jeEJ6oHXoADnVoRnSmEGSgAg8FFfeyGvxlv5zeRGsAFR18GTekFPKlfPunyOtwHGreMn6wp79Erz8bYw7/pxl4Dgg4dhosBnRfJX"

"3xQdDkCnhSrAihJIirMBBK9+qVorvHZtza+IjgYQrR4KF9Obi8LxNp+DAdCFF/9BL6oE5XfeTkjOBRB96I+gOHdOdD27r51nFbl2"

"gxQOC0W6s7r7byHsWADh/z05X2wZ883/UuNUFaDz314Yz/bbbrcEAFQNZVUC8ee1sjMBqL+py9IFemTqv+v1S0Ygix3c/bVtjgRA"

"Q98KZrIAZJvLDzAAbgBQYRjA6/JnFoFLHKeNOQKI/bKzKoMAeOoDFftrtzXWemUw/KHYfOjmhKc+kxQon/myz4kAgt+7kr5JpH2K"

"DHi7uwEgpoHWDgCTk/5hrXtKyyAC/yc6EIC+/Jm5tAOiPv9AU08X9YPbTSSgmqqCV0q+dG7Yo2d4oLeP34iAXxzk27Xp36ciDJ2i"

"yUlTtEefDND3jshi+na6/zjpvIhQ8k6fnL79lc+MG5Mq+31jUl1+rS89AWHzmYDjAIReFTOO/yhdSjvxlfTTdzIQUL6gOw2AcfPJ"

"dE0RlMqjNPX1rzxznb6bfv5IfGBMdRiApbSGmyiVR40sc9/X1QwzSMrzAYcBCHyrMV071pr3XaLv9HnSOQ5/kHQWAD2tCfRUfNXw"

"U0oNNcsqkre7fKU0g5wARN8WO011fIH6KaWx7DLwKkljPZQvzjsKQCCdCVQ+N/5x+y8PG1kILH89HbwHOOkAn3iA0byYGsWonf1K"

"iw9AdY9MBbLEST0z3+lNjQEZ9SeDXKrKB8DC6ERKE4XYoV4RgLpH5jTXlWxhwNlD/hT5ocnRGgeNBYLfC0+khjffDSpAycgUAfj9"

"riwDXFO8khpK5TUi4iIBxo7FFHkVZ78SUIDGLwcIAEC2Ab4Y+06vkkYHwo4BEE+jAS0DvY0A8aopKgHA9ayC554dmEmjA9QxKhD8"

"14lgigSflRSg+psfv0bh6yTrUJoeTNEhseJD0SESYGwJpwrAwd0KUHJ6BQy9kn024P3Bi6yS0Jqfhp2iAlVv

```
sVUVr/bPAMQvh1aa"

"RTUtawktPU8oKVjPaxzqymPX2PRVV4xt0MOdAtDquXs31miLGBrslZjvLF5rcYgEVI/UpwrAIsDC2H3Bj64R
523YlSIC8dGwMyRA"

"r0txA1se7hTA2Pr6/VarlGR/MbcHe9lyGy/eUpwgAWSMVVZhtn8aIPHmapO2Vil1PSm+QNWkyxEqMD3Prgmq
H9itAH1otVzQtSya"

"+P7AdErB9Y4AsHWS0W9S0TMDoJ7ORwAAWvofYe3E2LQTbIDhYv3g2sgTIqjVwfvtoeKurrXKEWdaokHGE1gg
igMkYPI8q85PTQOQ"

"O6vep2ft9gPUDTL+MNO6JjlABaZDNSkmkJgtgNC0Jwefxr2rlwmrLXAwAtYDkEcazDek7t0KxM+t+s1de+I5
vErh/b1u1hNwgBGk"

"VayXW980A7TqvgCQppzaD9BygDF61HUhbHsARpJxg4TZ/RKAeu8m3daXW/uBVDI6QKsmLB8OWN4LxP+TGehJ
n+wUIX7uXpvF7VqO"

"pkz4oJspy7Uk2R5AImaYb9TsnPat1oCmNpJrWa0fLZpvhH+vzvYqII22mX/gw273ahMoPEZyL2znPvMbT4wS
uwOg7KoYEu+VAe4P"

"j5sW8qjcrIvpC0kFzeYADCNudt/q+wFoVfyeBXhMzkecesw6YDSEwzYHQPQLzDvbPAOg3gOwcyGf0ur2x5kB
YdDuKpBgVkYKN7ok"

"iJ+7O/gROuW8cN5mOkJtSbE5AE01T2kQsluG6nvZMpoS+dXuhmS2guGYbnMAMGp+RXX9EoBx91cSj+XZj0s9
Zk9AHLO5CtCtbAs2"

"3wLt3F3XpzFfG96wn63gBZsDYOLd5AMlDKCuJA2D7fl240ZFp/kzjQftDcAwz16QRIOy0gkSSRP25OvJivEG
k0rRLTbvBsk8YWxg"

"JwANAwBQQanN34v5gEEm2RxAImSO9lU/ToCsWICY4M3fkZV2TfPtBy0GoGlmANKmBGh3F8IHu/IfyzU0mqFp
VLc3gBATDnKHAVaS"

"xuXfBwCA4TMTTczbWwXguuklkxsKgLTJp30sDYUYFdE8ICR+ewNghusCcSkAew+0UwDQpELql5AY5/mKnQFQ
FyvlnQBAbw90SBoI"

"rkIIRCTG0UhodpYAap73ru4GACAutedxHy1sUkMyjx/pWisL8r2sDYlRet5ks6RaAQCAuNVNj59LFBTNkdym
ETFNaraWACNurp60"

"Uns3jfQUlhOGus3clLBkYwACO+m56a7jRmRVLqjIGnbptGZnAFrQ/LpWTwC4C/WuG1ilsLMKqExARy66RCK0
mlscpXYGwE78S8W/"

"LmJ+5ZrFCxut7QWYoYBAipdXgYGoUUrsCyDl6ixewhJrCJm9jKAqWc6Q6fYF1d6DIdM6rhq3FW+LWTBpbwDm
wSCVrSjTXEg8YG8J"

"MF8eK+yVZ4FnFfGMEYvLk6x32oijAGjm9ltQeYkQVAHnAqDlDgAQAAIoZwCWhO80SstcAqiDVcCSl8dhhMkP
gMck9CRmRZkxl/Xe"

"JTcA5iB+PGiFE2c2JDXtdgYgmUO/1mxM1pykAmvtCC16eJUyUWIzABIx28Dil/ayi48lsDMA9m2BFragIwnz
HB1bvUjKLAGu4jWC"
```

```
"slq1ydYAZPPHqnDxReoXmBhD3NIaWzsv4HqQUQkLHAHqpIgQe16DFmsrtshbE2ynqNlZBdi1oMX7wppqLmJR
sTUAMK8DWpgr3hW6"

"btotKoisnbEVAELMM5lCoHgFDkngnLEAeZ81CsV6QuxGVEJ2S7ZWAXPtdFexnpDhYlYdWd0nWO0K7wyaHYGJ
YmWKcQOMz1oMwOL1"

"AZKb6QiXGoorMHHJ/Lmh1t6uMBBzgfFYkUnwwiqz+rY2YWsA1Y8wL3Cs2KndUbNVkWrD9pYAoVMyO0JXiuoG
jK1MJ7Aog60BEGZF"

"tPHQlaLeGK1i9iDdsmDdGU8jKAhMBeXrRVU48Qt2D9I+wd4qwK7knY4VteN/wW9+nLqpvVUAYJd5QwMZO19M
aVVMUqaabps7QuBy"

"m0vUa1LTi+YRDKlnnpZqF2OOoOYRRka3FpMBTZhi5EfyaDYHwG5yKi4D2hW/+WFyQwG7AxDcTE89VkQmzCQj
PgKzk9Z+3SCQGSYS"

"bGy5UFtopfUW5rBGSlrtLgEg7WLMFL1esA4sjjH2o/5x22eRgfr9TB0Xhwu2gvFhhp20adr2AKjIxkULzguv
72CTk962/sQpywEI"

"iXoGQHK0QBGIjzD2U1y03AZaD4CwO/4hMlygEVgcZhIJG6TT/hIAUk+EqXb99wvSAb05wmhAQ3/C8upav2Wm
7ndZq5C8/FwhBUUu"

"jTN52aTtC277SwCp2Me4QpFTEwUEBWhyhOn0hRsFZGAoPQAhwu4S15t/WsA+l9gWtvcg8d2yAwAA6WFHbJEf
F+DARV5gH6rpj4MT"

"ANSzGdAgVp2/GdSbp9mzWBs2JxwBICUDGtDkf+SdGX/mFJs/VLixB5wB4IbM2qrF0bfyFAF9xzCrASSxT3YE
AHD1sB67sWU4TxGY"

"GU5BVte/4AwJgIZud6o3mJ8I6DvSENuecAgAEmGNAOhb3shLBGZOpAATf7VbcQqA23tT+qvIqZN5ZACaax5J
4SXt7QIeF5fTBtOc"

"iqV8zsj5xMCkmubAOU6HTnLZL0A+TDOqa/qXz+asBDM//27KdIowm282yvVTASAf7E0JXRmJE2/lqATRHW+0
pQwe3N355WNdVxWg"

"+q9SRZgon6c5CXGSvpjm4FXl+XkHnThp3ExzVrYofjUZygFe4G0l9WGh8lzIQSdOptMB0BuOn8xhpnS6+Xgi
1Wdw7+WkAbxOnU13"

"OiJRHh1fWutJ//JT6U4e5nbwsPjXfLguxOF/Um7eXNLqdeHBbK8j0PrCi62p4RMx+jfNDzpJAjIcPe6Bw2o2
GTD8xjFIF/rmd/g4"

"LwDJ33an67a94uHAUkZrpvuTx+R0u8LIzh867ehtGvha2kmMduHweDLD6dtLgeVjctqcc+IneJ28zQ/AOqX0
AWcfGTpjpDt/3pik"

"d45InrQPKX/udxwAY/nJtIM3Og6DJ6iecgT99RA9exC8ad1d8QFOTgBPADT0swxnpHph8+FxujS5yrOLTvrp
8rH9kCHnovJFbhpA"

"Cbe92boxGE4fBPHqHxzYuasL5jQgFCQNSDsk3xy7dt6XfrAgCm/ICq9q8gMAwatfypAFWmzzVxzwtvffuzEy
57824anLEDRSml8R"

"wYEAdH0onCkO5tXDFfvd7m1uGpfUeVW9OSEomcaKonBcUZwIAIKXnsuYCJy0Uz8AbNo60zkOAB0wp2X6qvKp
k/wEgCuAbCIAALWN"
```

"VIsJrnAjkWKhzF/jKwBcAWQVgZVBHo1La8wb8hUAvgDWEIGcLs4CwBdAlo4g54trF8AdQGZfIHcB4OgD8AcAwV/vq9KKqV7bw9/j"

"KgC8Aej04MW5Ip73zZ6VuAoA77S6YvjpcHXhSiDOfGM3XwHgLQFgLL79XOF2UPnUyYjsbAAQrTkOM1Xgs97I6w0+zvXjnkvMMzuo"

"K4Xqz8Fe3u0HfvGAe4E++qJYEAFR+RMjwL16hH/Sy6jr0CvV+feFpO3qWRd/Aajg/gvgCf6zerEAA/j+3/HuAUpiAwCg9deHWvJO"

"geadOfRspASV428DKKVJ+hrkKcweeIb6S1G3kgCgS8tH8iPghaHxkrS/RADoVH4EPDA0fp1uJABG4M5BaM/9/Q+eSaobCsDHBHI7"

"aol0lLD9JQNA9cCdIxkmfthwKQydyWUpicMAUD20fEQR1zQEHqViaFwvWftLCIDqgeVjfdAhrSH+lYfHS/f+SwqAGgH6o6dAzGIL"

"vQo8+gItmf6XaCywyunyd1w+9d6Etyb9QSmeBn/ll3sen8rFUjgnHmC+5jzVw+dOgDd1KkTwUT8Mbh9oDbaXtEYlBgCxBd/yy/5r"

"E4TZZE4oVBzwfrovekuGDQ0AYO6WknzJf/MnzO3K/d5Hu/Qr7aWuTukBgBHwSskz755ZPSVGKv9tc5cRKKn2rxsAACMg+F59YvUv"

"Cw98pAc98jrUpWIdfhOEDvAzISI65+lYj6qsDwAAqGVvyOL6VATPGEEACGBdLs3q9LhOAyBxOIoBVQABIAAnGcEyByChCiAABIAA"

"EAACQAAIAAEgAASAABBAWQ6HJZSAMo8HaCgBCKC8jSBKQLkbQZQABIAAEAACQAAIAAEgAASwrhfflaLRcM5fzXxMu+BzLIA5ryfj"

"/35h/kh2Zi5mqsOhAOa8I1OhTAcDLZk+UeMbGb6n1nb0+znuIuCZTU780elTFpTT0/RiUHGiBMz81/HzmYVXD5o+ZkkycXkc/omj"

"nPLbJvjbPlG2pKNSKn/od962ORo+9o9FJZFapaZ7PvGzW5LTbIC+/FBr0KKyxOrn+3hZAW6OUORlOWgZzNr3WnnVk5cRNLaOGtaV"

"thA53yg7TAKW3wpbqE87/l1zmAQsXnLFLCwufrPVaTbgcr2lPE9RZwGgWy0+F8x1UXMWAErDVpZnaJrTACxYLFEOAwDGtLUVdoWd"

"ZQOccyEABMBnjCV0Wjt845ZblZcEEJe1xSUkyVEAhITb0lcmkAaHDYZuWSsBlDhtLCClnL9d1NXQTxwGoPqzlp6R69o+7TAAwu0/"

"VawrTbzK67RBbvEAcrvnpHWltTR3ceuuuAVF9SFtwioBICe4HbPBb2YoeOm5OmtGMGILx3NWeB61deikMlf8mJCO61df53fMBr+p"

"MXFmSPsJ2Zbp3zEzGtKY8YtTlX/Zyy/LEse5wVaZbL6Wqxmg8xlrOLjrLxZkcJ4KAOhi8kzGU5SuvWzOJvf9TF9Ob+4KcFwiwTWb"

"XDSSWXVTssll5jjNc4kI1xUintrMRpD5B814DAfZ5twlMkC8mf6T0kEqmE0OASAABIAAEAACQAAIAAEgAARQBgAwrS5mkEAbgADQ"

"CKIRRBVAAAgAASAABIAAEAACQAAIAAEggHIbDqMElHs8ACUAAZS5EZRQAsrcCGplrgKO3FVgrqHcbUCZqwDb

```
CZByAyBtiq/+SH1l"

"JwFuEwBXIykzAHKjKdOibOZRBgB0n9kIblooMwBE3Cev+nRDkcoMgDDrWgVAiPfKZdcNDly8/6FuYL1MwPoB
qN+7595LFyr2JsoO"

"gHC19d5vy5En5LIDAPVP1z9ytwd4ehHKD4D7Nz3jPgAAT3Lgu+51qwbXnaPZr2jNMy95GyHmf/Rww7p5wpw3
Tma9PMFj5NooVAx+"

"vleEcpQAMBa3vhQG4dHORbk8AYCRkACALqxj+9cXAFA/AHglKFsANrgwLI4AEAACQAAIAAEgAASAABAAAkAA
CAABIAAEgAAQAAJA"

"AAgAASAABIAAEAACQAAIAAEgAASAABAAAkAACAABIAAEgAAQAAJAAAgAASAABIAAEAACQAAIAAEgAASAABAA
AkAACAABIAAEgAAQ"
    "AAJAAAgAASAABIAAEAACQAAIAAEgAASAABCAs67/BxTEOsuQj/foAAAAAElFTkSuQmCC"
)
_FAVICON_BYTES = _b64.b64decode(_FAVICON_B64)

@app.route("/favicon-v2-512.png")
def _favicon_v2_512():
    r = _Resp(_FAVICON_BYTES, mimetype="image/png")
    r.headers["Cache-Control"] = "public, max-age=86400"
    return r
# ===== END FAVICON ROUTE =====



# ============================================================================
# ===== SITE PUBLISHING + FREE SUBDOMAIN + CUSTOM DOMAINS (Pro / Heavy Pro) ===
# ============================================================================
# Environment variables (Vercel > Settings > Environment Variables):
#   VERCEL_TOKEN          (required for custom domains) - Vercel access token
#   VERCEL_PROJECT_ID     (required for custom domains) - project that serves this
app
#   VERCEL_TEAM_ID        (optional)  - only if the project lives in a Vercel team
#   SITES_BASE_DOMAIN     (optional)  - default: wholeai.space  (free sites:
name.wholeai.space)
#   SITES_WILDCARD_READY  (optional)  - "0" (default) = links look like
wholeai.space/name  (no DNS work needed)
#                                       "1" = links look like name.wholeai.space
(needs *.wholeai.space on Vercel)
#   CUSTOM_DOMAIN_CNAME   (optional)  - default: cname.vercel-dns.com
#   CUSTOM_DOMAIN_A       (optional)  - default: 76.76.21.21
#   EXTRA_MAIN_HOSTS      (optional)  - comma separated extra hosts that are the main
app
import mimetypes as _mimetypes
import re as _re_sites
from firebase_admin import auth as _fb_auth

VERCEL_TOKEN = os.environ.get("VERCEL_TOKEN", "").strip()
VERCEL_PROJECT_ID = os.environ.get("VERCEL_PROJECT_ID", "").strip()
```

```
VERCEL_TEAM_ID = os.environ.get("VERCEL_TEAM_ID", "").strip()
SITES_BASE_DOMAIN = os.environ.get("SITES_BASE_DOMAIN",
"wholeai.space").strip().lower()
SITES_WILDCARD_READY = os.environ.get("SITES_WILDCARD_READY", "0").strip() == "1"
CUSTOM_DOMAIN_CNAME = os.environ.get("CUSTOM_DOMAIN_CNAME", "cname.vercel-
dns.com").strip()
CUSTOM_DOMAIN_A = os.environ.get("CUSTOM_DOMAIN_A", "76.76.21.21").strip()
# The real app (Flask on Vercel) lives on www.<base>. The bare domain (wholeai.space)
may point
# somewhere else (for example Firebase Hosting, which shows "Site Not Found"), so
free links must use www.
SITES_PUBLIC_HOST = os.environ.get("SITES_PUBLIC_HOST", "www." +
SITES_BASE_DOMAIN).strip().lower()
_MAIN_HOSTS = {SITES_BASE_DOMAIN, "www." + SITES_BASE_DOMAIN, SITES_PUBLIC_HOST,
"localhost", "127.0.0.1"}
for _h in os.environ.get("EXTRA_MAIN_HOSTS", "").split(","):
    if _h.strip():
        _MAIN_HOSTS.add(_h.strip().lower())

PAID_PLANS = ("Pro", "Heavy Pro")
SITE_MAX_BYTES = 900 * 1000  # Firestore document limit is 1 MiB
_RESERVED_SLUGS = {
    "www", "app", "api", "admin", "mail", "ftp", "cdn", "static", "assets",
"dashboard", "login",
    "signup", "support", "help", "blog", "docs", "status", "billing", "pay",
"payments", "root",
    "wholeai", "whole", "ai", "test", "dev", "staging", "s", "sites", "domains",
"cname", "admin-login",
    "favicon", "__site-icon", "sitemap", "robots", "static", "index",
"google13d17d96d6c0eb30", "pricing", "privacy", "terms", "about", "contact"
}
_site_cache = {}
_SITE_CACHE_TTL = 30


# ---------- helpers ----------
def _sites_user_email():
    """Verify the Firebase ID token sent by the frontend and return the lower-cased
email."""
    header = request.headers.get("Authorization", "")
    if not header.startswith("Bearer "):
        return None
    try:
        decoded = _fb_auth.verify_id_token(header[7:].strip())
        email = (decoded.get("email") or "").strip().lower()
        return email or None
    except Exception as e:
        print("[sites] token verify failed:", e)
        return None
```

```
def _sites_user_plan(email):
    """Real plan from Firestore (server-side - the browser can not fake this)."""
    try:
        snap = db.collection("users").document(email).get()
        if not snap.exists:
            return "Free"
        sub = (snap.to_dict() or {}).get("subscription") or {}
        plan = sub.get("plan") or "Free"
        if plan in PAID_PLANS:
            expiry = sub.get("expiryDate")
            if expiry and int(expiry) < int(time.time() * 1000):
                return "Free"
        return plan
    except Exception as e:
        print("[sites] plan lookup failed:", e)
        return "Free"


_owner_plan_cache = {}
_OWNER_PLAN_TTL = 120  # seconds


def _owner_is_paid_cached(email):
    """True if the site owner currently has an active Pro / Heavy Pro plan (cached,
saves Firestore reads)."""
    if not email:
        return False
    now = time.time()
    hit = _owner_plan_cache.get(email)
    if hit and now - hit[0] < _OWNER_PLAN_TTL:
        return hit[1]
    paid = _sites_user_plan(email) in PAID_PLANS
    _owner_plan_cache[email] = (now, paid)
    return paid


def _slugify(text):
    s = _re_sites.sub(r"[^a-z0-9]+", "-", (text or "").lower()).strip("-")
    s = s[:40].strip("-")
    return s or "my-site"


def _site_public_url(slug):
    if SITES_WILDCARD_READY:
        return "https://%s.%s" % (slug, SITES_BASE_DOMAIN)
    return "https://%s/%s" % (SITES_PUBLIC_HOST, slug)


def _site_fallback_url(slug):
```

```
    return "https://%s/%s" % (SITES_PUBLIC_HOST, slug)


ICON_MAX_BYTES = 100 * 1000
_ICON_RE = _re_sites.compile(r"^([A-Za-z0-
9+/=]+)$")
ICON_ROUTE = "__site-icon"


def _clean_icon(raw):
    """Validate an uploaded logo (data URL). Returns the data URL, or None if
invalid."""
    raw = (raw or "").strip()
    m = _ICON_RE.match(raw)
    if not m:
        return None
    try:
        size = len(_b64.b64decode(m.group(2), validate=True))
    except Exception:
        return None
    if size > ICON_MAX_BYTES:
        return None
    return raw


def _site_icon_response(site):
    raw = (site.get("icon") or "")
    m = _ICON_RE.match(raw)
    if not m:
        return Response(b"", status=404)
    resp = Response(_b64.b64decode(m.group(2)), mimetype="image/" + m.group(1))
    resp.headers["Cache-Control"] = "public, max-age=300"
    return resp


def _inject_head(html, site, base_path):
    """Put the real project name as tab title and the user's own logo (or no logo) as
favicon."""
    import html as _html_mod
    name = _html_mod.escape((site.get("projectName") or "My Site").strip() or "My
Site", quote=True)
    if site.get("icon"):
        icon_href = (base_path.rstrip("/") + "/" + ICON_ROUTE) if base_path else ("/"
+ ICON_ROUTE)
        icon_tag = '<link rel="icon" href="%s?v=%s">' % (icon_href,
int(site.get("iconUpdatedAt") or 0))
    else:
        # empty icon: browser shows its own default, never the Whole AI logo
        icon_tag = '<link rel="icon" href="data:,">'
    html = _re_sites.sub(r"<title[^>]*>.*?</title>", "", html, flags=_re_sites.I |
```

```
_re_sites.S)
    html = _re_sites.sub(r"<link[^>]+rel=[\"']?(?:shortcut\s+)?(?:icon|apple-touch-
icon)[\"']?[^>]*>", "", html, flags=_re_sites.I)
    tags = "<title>%s</title>%s" % (name, icon_tag)
    if _re_sites.search(r"<head[^>]*>", html, flags=_re_sites.I):
        return _re_sites.sub(r"(<head[^>]*>)", lambda m: m.group(1) + tags, html,
count=1, flags=_re_sites.I)
    if _re_sites.search(r"<html[^>]*>", html, flags=_re_sites.I):
        return _re_sites.sub(r"(<html[^>]*>)", lambda m: m.group(1) + "<head>" + tags
+ "</head>", html, count=1, flags=_re_sites.I)
    return "<!doctype html><head><meta charset=\"utf-8\">" + tags + "</head>" + html


def _reserve_slug(base, doc, owner=None):
    """Create sites/<slug>. If taken: base-2, base-3 ... Returns the slug used."""
    for n in range(1, 40):
        candidate = base if n == 1 else "%s-%d" % (base[: 38 - len(str(n))].strip("-
"), n)
        doc["slug"] = candidate
        try:
            db.collection("sites").document(candidate).create(doc)
            return candidate
        except Exception as ce:
            if "already exists" in str(ce).lower() or "AlreadyExists" in
type(ce).__name__:
                if owner:
                    # my own old link (redirect stub) can be taken back
                    snap = db.collection("sites").document(candidate).get()
                    old = snap.to_dict() if snap.exists else None
                    if old and old.get("redirectTo") and old.get("formerOwner") ==
owner:
                        db.collection("sites").document(candidate).set(doc)
                        return candidate
                continue
            raise
    return None


def _move_site(site, new_slug):
    """Move a site to a new link. The old link stays alive as a redirect."""
    old_slug = site["slug"]
    now = int(time.time() * 1000)
    new_doc = dict(site)
    new_doc.update({"slug": new_slug, "updatedAt": now})
    used = _reserve_slug(new_slug, new_doc, owner=site.get("owner"))
    if not used:
        return None
    # old link -> redirect stub (no owner / projectId, so it never shows in lists)
    db.collection("sites").document(old_slug).set({
        "slug": old_slug, "redirectTo": used, "formerOwner": site.get("owner"),
```

```
"createdAt": now, "updatedAt": now
    })
    # older stubs that pointed at old_slug now point straight at the newest link
    try:
        for doc in db.collection("sites").where("redirectTo", "==",
old_slug).stream():
            doc.reference.update({"redirectTo": used})
    except Exception as e:
        print("[sites] stub repoint failed:", e)
    _site_cache_clear()
    return new_doc


def _site_payload(d):
    return {
        "icon": d.get("icon") or "",
        "slug": d.get("slug"),
        "projectId": d.get("projectId"),
        "projectName": d.get("projectName"),
        "url": _site_public_url(d.get("slug")),
        "fallbackUrl": _site_fallback_url(d.get("slug")),
        "customDomain": d.get("customDomain") or "",
        "domainStatus": d.get("domainStatus") or "",
        "updatedAt": d.get("updatedAt") or 0,
    }


def _vercel_params():
    return {"teamId": VERCEL_TEAM_ID} if VERCEL_TEAM_ID else {}


def _vercel(method, path, json_body=None):
    if not VERCEL_TOKEN or not VERCEL_PROJECT_ID:
        return 503, {"error": {"message": "Custom domains are not configured on the
server yet."}}
    try:
        r = requests.request(
            method, "https://api.vercel.com" + path,
            headers={"Authorization": "Bearer " + VERCEL_TOKEN, "Content-Type":
"application/json"},
            params=_vercel_params(), json=json_body, timeout=25
        )
        try:
            body = r.json()
        except Exception:
            body = {}
        return r.status_code, body
    except Exception as e:
        return 502, {"error": {"message": "Could not reach Vercel: %s" % e}}
```

```
_DOMAIN_RE = _re_sites.compile(r"^(?=.{4,253}$)(?!-)([a-z0-9-]{1,63}(?<!-)\.)+[a-z]
{2,63}$")


def _clean_domain(raw):
    d = (raw or "").strip().lower()
    d = _re_sites.sub(r"^https?://", "", d)
    d = d.split("/")[0].split("?")[0].split(":")[0].strip(".")
    return d


def _domain_is_apex(domain):
    parts = domain.split(".")
    if len(parts) == 2:
        return True
    if len(parts) == 3 and parts[-2] in ("co", "com", "org", "net", "gov", "ac",
"edu") and len(parts[-1]) == 2:
        return True
    return False


def _dns_records(domain, verification=None):
    records = []
    if _domain_is_apex(domain):
        records.append({"type": "A", "name": "@", "value": CUSTOM_DOMAIN_A})
    else:
        sub = domain.split(".")[0] if len(domain.split(".")) > 2 else domain
        # host label = everything before the registrable domain
        labels = domain.split(".")
        cut = 3 if (len(labels) >= 4 and labels[-2] in ("co", "com", "org", "net",
"gov", "ac", "edu") and len(labels[-1]) == 2) else 2
        host_label = ".".join(labels[:-cut]) or sub
        records.append({"type": "CNAME", "name": host_label, "value":
CUSTOM_DOMAIN_CNAME})
    for v in (verification or []):
        if (v.get("type") or "").upper() == "TXT":
            records.append({"type": "TXT", "name": v.get("domain", "_vercel"),
"value": v.get("value", "")})
    return records


def _get_site_by_slug(slug):
    now = time.time()
    hit = _site_cache.get(("slug", slug))
    if hit and now - hit[0] < _SITE_CACHE_TTL:
        return hit[1]
    snap = db.collection("sites").document(slug).get()
    data = snap.to_dict() if snap.exists else None
    _site_cache[("slug", slug)] = (now, data)
```

```
    return data


def _get_site_by_domain(domain):
    now = time.time()
    hit = _site_cache.get(("dom", domain))
    if hit and now - hit[0] < _SITE_CACHE_TTL:
        return hit[1]
    data = None
    for doc in db.collection("sites").where("customDomain", "==",
domain).limit(1).stream():
        data = doc.to_dict()
    _site_cache[("dom", domain)] = (now, data)
    return data


def _site_cache_clear():
    _site_cache.clear()


def _normalize_files(files):
    out = []
    for f in (files or []):
        name = str((f or {}).get("name") or "").strip().lstrip("/")
        content = (f or {}).get("content")
        if not name or ".." in name or not isinstance(content, str):
            continue
        out.append({"name": name, "content": content})
    return out


def _render_site(site, path, base_path=""):
    if (path or "").lstrip("/") == ICON_ROUTE:
        return _site_icon_response(site)
    files = {f["name"]: f["content"] for f in (site.get("files") or [])}
    path = (path or "").lstrip("/")
    if path == "" or path.endswith("/"):
        path = path + "index.html"
    body = files.get(path)
    if body is None and path == "index.html":
        body = site.get("html") or ""
    if body is None and "." not in path.split("/")[-1]:
        body = files.get("index.html") or site.get("html")  # SPA style fallback
        path = "index.html"
    if body is None:
        return Response("<!doctype html><title>Not found</title><body style=\"font-
family:system-ui;"
                        "text-align:center;padding:15vh 20px\"><h1>404</h1><p>Page
not found.</p></body>",
                        status=404, mimetype="text/html")
```

```
    mime = _mimetypes.guess_type(path)[0] or "text/html"
    if mime == "text/html":
        body = _inject_head(body, site, base_path)
    resp = Response(body, mimetype=mime)
    resp.headers["Cache-Control"] = "public, max-age=60"
    resp.headers["X-Content-Type-Options"] = "nosniff"
    return resp


def _redirect_stub(site, path):
    target = _site_public_url(site["redirectTo"]).rstrip("/") + "/" + (path or
"").lstrip("/")
    if request.query_string:
        target += "?" + request.query_string.decode("utf-8", "ignore")
    return redirect(target, code=301)


def _unavailable_page(title, text):
    html = ("<!doctype html><meta name=viewport content='width=device-width,initial-
scale=1'>"
            "<title>%s</title><body style=\"margin:0;font-family:system-ui,sans-
serif;background:#fff;color:#0a0a0a;"
            "display:flex;min-height:100vh;align-items:center;justify-
content:center;text-align:center;padding:24px\">"
            "<div><h1 style=\"font-size:22px;margin:0 0 8px\">%s</h1><p
style=\"color:#555;margin:0\">%s</p></div></body>"
            % (title, title, text))
    return Response(html, status=404, mimetype="text/html")


# ---------- serve published sites by Host header ----------
from flask import Response  # noqa: E402  (already imported above as needed)


@app.before_request
def _serve_hosted_sites():
    host = (request.host or "").split(":")[0].lower()
    if not host or host in _MAIN_HOSTS or host.endswith(".vercel.app"):
        return None
    try:
        if host.endswith("." + SITES_BASE_DOMAIN):
            slug = host[: -(len(SITES_BASE_DOMAIN) + 1)]
            if not slug or "." in slug:
                return None
            site = _get_site_by_slug(slug)
            if not site:
                return _unavailable_page("Site not found", "This site has not been
published yet.")
            if site.get("redirectTo"):
                return _redirect_stub(site, request.path)
```

```
            return _render_site(site, request.path)
        site = _get_site_by_domain(host)
        if site:
            # Custom domain is a Pro / Heavy Pro feature. If the owner's plan
expired,
            # send visitors to the free link. The site and its data stay untouched,
            # and the domain works again as soon as the owner renews.
            if not _owner_is_paid_cached(site.get("owner")):
                target = _site_public_url(site.get("slug")).rstrip("/") + "/" +
request.path.lstrip("/")
                if request.query_string:
                    target += "?" + request.query_string.decode("utf-8", "ignore")
                return redirect(target, code=302)
            return _render_site(site, request.path)
    except Exception as e:
        print("[sites] serve error:", e)
    return None


@app.route("/<slug>/", defaults={"subpath": ""})
@app.route("/<slug>/<path:subpath>")
def _serve_site_by_path(slug, subpath):
    """Free link: wholeai.space/<name>  (Flask redirects /name -> /name/ so relative
files keep working)."""
    slug = (slug or "").lower()
    if not _re_sites.match(r"^[a-z0-9][a-z0-9-]{0,39}$", slug) or slug in
_RESERVED_SLUGS:
        return _unavailable_page("Page not found", "This page does not exist.")
    site = _get_site_by_slug(slug)
    if not site:
        return _unavailable_page("Site not found", "This site has not been published
yet.")
    if site.get("redirectTo"):
        return _redirect_stub(site, subpath)
    return _render_site(site, subpath, base_path="/" + slug)


# ---------- API ----------
@app.route("/api/sites/publish", methods=["POST"])
def sites_publish():
    email = _sites_user_email()
    if not email:
        return jsonify({"error": "Please log in again."}), 401
    data = request.get_json(silent=True) or {}
    project_id = str(data.get("projectId") or "").strip()
    project_name = str(data.get("projectName") or "My Site").strip()[:80]
    html = data.get("html") if isinstance(data.get("html"), str) else ""
    files = _normalize_files(data.get("files"))
    if not project_id:
        return jsonify({"error": "Missing project."}), 400
```

```
    if not html.strip() and not files:
        return jsonify({"error": "Nothing to publish yet. Build the project
first."}), 400
    total = len(html.encode("utf-8")) + sum(len(f["content"].encode("utf-8")) for f
in files)
    if total > SITE_MAX_BYTES:
        files_only_html = [f for f in files if f["name"].lower().endswith((".html",
".css", ".js"))]
        total2 = len(html.encode("utf-8")) + sum(len(f["content"].encode("utf-8"))
for f in files_only_html)
        if total2 > SITE_MAX_BYTES:
            return jsonify({"error": "Project is too large to publish (limit about
900 KB)."}), 413
        files = files_only_html
    now = int(time.time() * 1000)
    try:
        existing = None
        for doc in db.collection("sites").where("owner", "==",
email).where("projectId", "==", project_id).limit(1).stream():
            existing = doc
        if existing is not None:
            d = existing.to_dict()
            existing.reference.update({"html": html, "files": files, "projectName":
project_name, "updatedAt": now})
            d.update({"html": html, "files": files, "projectName": project_name,
"updatedAt": now})
            # link follows the project name automatically (unless the user typed a
custom link by hand)
            want = _slugify(project_name)
            if d.get("slugAuto", True) and want != d.get("slug") and want not in
_RESERVED_SLUGS and len(want) >= 3 \
                    and not _re_sites.match(r"^%s-\d+$" % _re_sites.escape(want),
d.get("slug") or ""):
                moved = _move_site(d, want)
                if moved:
                    d = moved
            _site_cache_clear()
            return jsonify({"ok": True, "site": _site_payload(d), "updated": True})
        custom_slug = bool(str(data.get("slug") or "").strip())
        base = _slugify(data.get("slug") or project_name)
        if base in _RESERVED_SLUGS:
            base = base + "-site"
        doc = {
            "owner": email, "projectId": project_id, "projectName": project_name,
            "html": html, "files": files, "customDomain": "", "domainStatus": "",
"icon": "",
            "slugAuto": not custom_slug, "createdAt": now, "updatedAt": now,
        }
        used = _reserve_slug(base, doc, owner=email)
        if used:
```

```
            _site_cache_clear()
            return jsonify({"ok": True, "site": _site_payload(doc), "updated":
False})
        return jsonify({"error": "Could not reserve a link. Try renaming the
project."}), 409
    except Exception as e:
        print("[sites] publish error:", e)
        return jsonify({"error": "Publish failed. Please try again."}), 500


@app.route("/api/sites/list", methods=["GET"])
def sites_list():
    email = _sites_user_email()
    if not email:
        return jsonify({"error": "Please log in again."}), 401
    try:
        items = [_site_payload(doc.to_dict()) for doc in
db.collection("sites").where("owner", "==", email).stream()]
        items.sort(key=lambda x: x.get("updatedAt", 0), reverse=True)
        return jsonify({"ok": True, "plan": _sites_user_plan(email), "sites": items})
    except Exception as e:
        print("[sites] list error:", e)
        return jsonify({"error": "Could not load sites."}), 500


def _owned_site(email, slug):
    site = _get_site_by_slug((slug or "").strip().lower())
    if not site or site.get("owner") != email:
        return None
    return site


@app.route("/api/domains/add", methods=["POST"])
def domains_add():
    email = _sites_user_email()
    if not email:
        return jsonify({"error": "Please log in again."}), 401
    if _sites_user_plan(email) not in PAID_PLANS:
        return jsonify({"error": "Custom domains are available on Pro and Heavy
Pro.", "upgrade": True}), 403
    data = request.get_json(silent=True) or {}
    site = _owned_site(email, data.get("slug"))
    if not site:
        return jsonify({"error": "Project is not published yet."}), 404
    domain = _clean_domain(data.get("domain"))
    if not _DOMAIN_RE.match(domain):
        return jsonify({"error": "Enter a valid domain, for example example.com"}),
400
    if domain == SITES_BASE_DOMAIN or domain.endswith("." + SITES_BASE_DOMAIN) or
domain.endswith(".vercel.app"):
```

```
        return jsonify({"error": "This domain can not be used."}), 400
    for doc in db.collection("sites").where("customDomain", "==",
domain).limit(1).stream():
        if doc.id != site["slug"]:
            return jsonify({"error": "This domain is already connected to another
project."}), 409
    old = site.get("customDomain")
    if old and old != domain:
        _vercel("DELETE", "/v9/projects/%s/domains/%s" % (VERCEL_PROJECT_ID, old))
    status, body = _vercel("POST", "/v10/projects/%s/domains" % VERCEL_PROJECT_ID,
{"name": domain})
    if status not in (200, 201):
        err = (body.get("error") or {})
        code = err.get("code", "")
        if status == 409 or code in ("domain_already_in_use",
"domain_already_exists"):
            # may already be attached to this very project - fall through and read it
            status, body = _vercel("GET", "/v9/projects/%s/domains/%s" %
(VERCEL_PROJECT_ID, domain))
            if status != 200:
                return jsonify({"error": "This domain is already used by another
account."}), 409
        else:
            return jsonify({"error": err.get("message") or "Could not add the
domain."}), 502 if status >= 500 else 400
    verified = bool(body.get("verified"))
    records = _dns_records(domain, body.get("verification"))
    db.collection("sites").document(site["slug"]).update({
        "customDomain": domain, "domainStatus": "active" if verified else "pending",
"updatedAt": int(time.time() * 1000)
    })
    _site_cache_clear()
    return jsonify({"ok": True, "domain": domain, "status": "active" if verified else
"pending", "records": records})


@app.route("/api/domains/verify", methods=["POST"])
def domains_verify():
    email = _sites_user_email()
    if not email:
        return jsonify({"error": "Please log in again."}), 401
    if _sites_user_plan(email) not in PAID_PLANS:
        return jsonify({"error": "Custom domains are available on Pro and Heavy
Pro.", "upgrade": True}), 403
    data = request.get_json(silent=True) or {}
    site = _owned_site(email, data.get("slug"))
    if not site or not site.get("customDomain"):
        return jsonify({"error": "No domain connected to this project."}), 404
    domain = site["customDomain"]
    _vercel("POST", "/v9/projects/%s/domains/%s/verify" % (VERCEL_PROJECT_ID,
```

```
domain))
    s1, proj = _vercel("GET", "/v9/projects/%s/domains/%s" % (VERCEL_PROJECT_ID,
domain))
    s2, cfg = _vercel("GET", "/v6/domains/%s/config" % domain)
    verified = bool(proj.get("verified")) if s1 == 200 else False
    misconfigured = bool(cfg.get("misconfigured", True)) if s2 == 200 else True
    status = "active" if (verified and not misconfigured) else "pending"
    if status != site.get("domainStatus"):
        db.collection("sites").document(site["slug"]).update({"domainStatus":
status})
        _site_cache_clear()
    return jsonify({
        "ok": True, "domain": domain, "status": status, "verified": verified,
"misconfigured": misconfigured,
        "records": _dns_records(domain, proj.get("verification") if s1 == 200 else
None)
    })


@app.route("/api/domains/remove", methods=["POST"])
def domains_remove():
    email = _sites_user_email()
    if not email:
        return jsonify({"error": "Please log in again."}), 401
    data = request.get_json(silent=True) or {}
    site = _owned_site(email, data.get("slug"))
    if not site:
        return jsonify({"error": "Project not found."}), 404
    domain = site.get("customDomain")
    if domain:
        _vercel("DELETE", "/v9/projects/%s/domains/%s" % (VERCEL_PROJECT_ID, domain))
    db.collection("sites").document(site["slug"]).update({"customDomain": "",
"domainStatus": ""})
    _site_cache_clear()
    return jsonify({"ok": True})


@app.route("/api/sites/rename", methods=["POST"])
def sites_rename():
    email = _sites_user_email()
    if not email:
        return jsonify({"error": "Please log in again."}), 401
    data = request.get_json(silent=True) or {}
    site = _owned_site(email, data.get("slug"))
    if not site:
        return jsonify({"error": "Website not found."}), 404
    name = str(data.get("projectName") or site.get("projectName") or "My
Site").strip()[:80]
    if not name:
        return jsonify({"error": "Website name can not be empty."}), 400
```

```
    old_slug = site["slug"]
    raw_new = str(data.get("newSlug") or "").strip()
    new_slug = _slugify(raw_new) if raw_new else old_slug
    now = int(time.time() * 1000)
    try:
        if new_slug == old_slug:
            db.collection("sites").document(old_slug).update({"projectName": name,
"updatedAt": now})
            site.update({"projectName": name, "updatedAt": now})
            want = _slugify(name)
            if site.get("slugAuto", True) and want != old_slug and want not in
_RESERVED_SLUGS and len(want) >= 3 \
                    and not _re_sites.match(r"^%s-\d+$" % _re_sites.escape(want),
old_slug):
                moved = _move_site(site, want)
                if moved:
                    site = moved
            _site_cache_clear()
            return jsonify({"ok": True, "site": _site_payload(site)})
        if len(new_slug) < 3:
            return jsonify({"error": "Link name must be at least 3 characters."}),
400
        if new_slug in _RESERVED_SLUGS:
            return jsonify({"error": "This link name is reserved. Please choose
another."}), 400
        taken = db.collection("sites").document(new_slug).get()
        if taken.exists:
            t = taken.to_dict() or {}
            mine_stub = t.get("redirectTo") and t.get("formerOwner") == email
            if not mine_stub:
                return jsonify({"error": "This link name is already taken. Try
another one."}), 409
        site = dict(site)
        site.update({"projectName": name, "slugAuto": False})
        db.collection("sites").document(old_slug).update({"projectName": name,
"slugAuto": False})
        moved = _move_site(site, new_slug)
        if not moved:
            return jsonify({"error": "This link name is already taken. Try another
one."}), 409
        return jsonify({"ok": True, "site": _site_payload(moved)})
    except Exception as e:
        print("[sites] rename error:", e)
        return jsonify({"error": "Rename failed. Please try again."}), 500

@app.route("/api/sites/icon", methods=["POST"])
def sites_icon():
    email = _sites_user_email()
    if not email:
        return jsonify({"error": "Please log in again."}), 401
```

```
    data = request.get_json(silent=True) or {}
    site = _owned_site(email, data.get("slug"))
    if not site:
        return jsonify({"error": "Publish the website first, then add a logo."}), 404
    raw = data.get("icon")
    now = int(time.time() * 1000)
    if not raw:
        icon = ""
    else:
        icon = _clean_icon(raw)
        if icon is None:
            return jsonify({"error": "Logo must be a PNG, JPG or WebP image under 100
KB."}), 400
    try:
        db.collection("sites").document(site["slug"]).update({"icon": icon,
"iconUpdatedAt": now, "updatedAt": now})
        site.update({"icon": icon, "iconUpdatedAt": now})
        _site_cache_clear()
        return jsonify({"ok": True, "site": _site_payload(site)})
    except Exception as e:
        print("[sites] icon error:", e)
        return jsonify({"error": "Could not save the logo. Please try again."}), 500

# ===== END SITE PUBLISHING / CUSTOM DOMAINS =====


if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=False, threaded=True)
```